SRE / sysreseval Documentation

IUT d'Orsay, Université Paris-Saclay

UTILISATION

1 Overview **1**
- 1.1 Student Workflow . . . 1
- 1.2 Monitoring of a session . . . 2
- 1.3 Running exams . . . 2
- 1.4 Post-exam workflow . . . 2

2 Exam Administration **3**
- 2.1 Lifecycle at a glance . . . 3
- 2.2 Setting up an exam — `sre set-exam` . . . 4
- 2.3 Monitoring an exam . . . 5
  - 2.3.1 Clean up after an exam . . . 5
- 2.4 Exam-only labs . . . 6
- 2.5 Post-exam workflow . . . 6
  - 2.5.1 Inspect a single archive . . . 6
  - 2.5.2 Re-grade with a corrected project file . . . 6
  - 2.5.3 Per-student PDF reports + summary spreadsheet . . . 7
  - 2.5.4 Export to a spreadsheet . . . 7

3 Lab Authoring Guide **9**
- 3.1 The authoring workflow on the CLI . . . 9
  - 3.1.1 `sre check [-p] <lab> [<state>]` . . . 9
  - 3.1.2 `sre start --debug-project [-p] <lab> [--xauth-file <path>]` . . . 10
  - 3.1.3 `sre connect <running_lab> <device>` and `sre exec <running_lab> <device> <command…>` . . . 10
  - 3.1.4 `sre state <running_lab> <state_name>` . . . 11
  - 3.1.5 `sre eval <running_lab>` . . . 11
  - 3.1.6 `sre wipe` . . . 11
- 3.2 Identifying a running lab and where its files live . . . 11
- 3.3 The four classes at a glance . . . 12
- 3.4 `Data` class . . . 13
  - 3.4.1 IP helper functions (from `/opt/sre/lib/ips.py`) . . . 13
  - 3.4.2 IPv6 labs . . . 14
  - 3.4.3 `compute_pre_generate` and `compute_post_generate` . . . 15
- 3.5 `Flavor` class . . . 16
- 3.6 `NetScheme` class . . . 17
  - 3.6.1 Declarations and accessors . . . 18
  - 3.6.2 `Machine` constructor parameters . . . 19
  - 3.6.3 `Network` parameters . . . 19
  - 3.6.4 Switches and VLANs . . . 20
  - 3.6.5 Instructor-only texts — `instructor()` . . . 20

3.6.6 Numbered sections, references and table of contents — `info_subtitle()`, `info_toc()` 22
3.7 State methods . . . 24
3.7.1 Lifecycle states: `save` / `restore` . . . 25
3.7.2 `@sre_state` decorator . . . 25
3.7.3 Per-machine operations . . . 25
3.7.4 File transfer between host and container . . . 26
3.7.5 Host-side operations . . . 26
3.7.6 Switch operations . . . 27
3.7.7 Multi-step state setup — the `step` parameter . . . 27
3.7.8 Reading command results — `multi_pass=True` . . . 27
3.7.9 Network config helpers (from `/opt/sre/lib/net_config.py`) . . . 28
3.7.10 State helpers (from `/opt/sre/lib/state_helpers.py`) . . . 28
3.8 `Grade` class . . . 29
3.8.1 The grading lifecycle . . . 30
3.8.2 Grade parts . . . 31
3.8.3 Self-eval vs instructor-eval scope . . . 32
3.8.4 Cheat answers . . . 33
3.8.5 Letter grades vs numeric marks . . . 33
3.8.6 Bonus elements . . . 33
3.8.7 Overriding `mark_exo_eval()` to adjust the overall mark . . . 33
3.8.8 Grade methods . . . 36
3.9 Grading Library Reference . . . 37
3.9.1 DHCP helpers (from `/opt/sre/lib/dhcp.py`) . . . 37
3.9.2 Mail helpers (from `/opt/sre/lib/smtp.py`) . . . 37
3.9.3 TLS helpers (from `/opt/sre/lib/tls.py`) . . . 38
3.9.4 DNS helpers (from `/opt/sre/lib/dns.py`) . . . 39
3.9.5 TCP port helpers (from `/opt/sre/lib/grade_helpers.py`) . . . 39
3.9.6 OSPF helpers (from `/opt/sre/lib/frr.py`) . . . 40
3.9.7 BGP helpers (from `/opt/sre/lib/frr.py`) . . . 40
3.9.8 Standard machine wiring (from `/opt/sre/lib/std.py`) . . . 42
3.9.9 Miscellaneous generators (from `/opt/sre/lib/utils.py`) . . . 42
3.9.10 Network inspection helpers (from `/opt/sre/lib/net_config.py`) . . . 42
3.9.11 Switch helpers (from `/opt/sre/lib/switch.py`) . . . 44
3.9.12 Ping helper (from `/opt/sre/lib/ping.py`) . . . 44
3.9.13 SSH helpers (from `/opt/sre/lib/ssh.py`) . . . 44
3.9.14 Packet capture helpers (from `/opt/sre/lib/pcap_gen.py`) . . . 46
3.10 Module-level Attributes . . . 47
3.10.1 Lab identity and presentation . . . 47
3.10.2 Student self-evaluation . . . 48
3.10.3 Automatic evaluation . . . 48
3.10.4 Archives and recordings . . . 49
3.10.5 State machine and flavors . . . 49
3.10.6 Networking . . . 49
3.10.7 Filesystem and export . . . 50
3.10.8 Schema rendering . . . 50
3.11 Complete Minimal Example . . . 50

4 Hubs, switches and VLANs 53
4.1 In the GUI . . . 53
4.1.1 The Networks tab . . . 53
4.1.2 The console of a manageable switch . . . 54
4.2 In a lab . . . 55
4.2.1 Declaring the kind of a network . . . 55
4.2.2 Running switch commands from a state . . . 56

4.2.3 Grading the configuration of a switch . . . 57
4.2.4 A complete example . . . 57
4.3 On the command line . . . 58
4.4 Limits . . . 58
4.5 What a host needs . . . 58
4.6 Tests . . . 59

5 Translating a Lab **61**
5.1 Internationalization with `tr()` / `make_tr()` . . . 61
5.2 Marking strings that must NOT be translated: `no_tr()` . . . 61
5.3 Centralizing translations with `_TRANSLATIONS` . . . 61
5.3.1 Where to put `_TRANSLATIONS` . . . 62
5.3.2 Dynamic strings with format placeholders . . . 62
5.4 Translation toolchain . . . 63
5.4.1 1. `prepare-sre-translations` — migrate and scaffold . . . 63
5.4.2 2. `check-sre-translations` — verify consistency . . . 64
5.4.3 3. `add-sre-translations` — machine-translate missing strings . . . 64
5.5 Complete translation workflow . . . 65
5.6 Switching the default language: `--change-default-language` . . . 65
5.6.1 Prerequisites . . . 66
5.6.2 Example: switching to English as a pivot before translating . . . 66

6 Installation and Setup **67**
6.1 Prerequisites . . . 67
6.2 Deployment layout . . . 67
6.3 Debian package . . . 68
6.4 Updating . . . 70
6.5 Running the installer . . . 70
6.6 Install manually . . . 71
6.7 Post-install steps . . . 73
6.7.1 1. Deploy and verify . . . 73
6.7.2 2. Enable X11 access for lab virtual machines . . . 74
6.7.3 3. Restricting student access to Docker . . . 74
6.7.4 4. Sharing evaluation archives for `sre watch` . . . 75
6.7.5 5. Pre-loading Docker images . . . 75
6.8 Reference . . . 76
6.8.1 Build targets . . . 76
6.8.2 Tests . . . 76
6.8.3 Building the documentation . . . 76
6.8.4 Network plugin for the switch types . . . 77
6.8.5 Dependencies . . . 77

7 GUI Reference — `sysreseval` **79**
7.1 Launching . . . 79
7.2 Opening a project . . . 79
7.3 Saving and restoring a project . . . 79
7.4 Project tabs . . . 80
7.5 Instructor mode . . . 80
7.6 Exam mode . . . 81
7.7 Settings . . . 81

8 CLI Reference — `sre` **83**
8.1 Global options . . . 83
8.2 Dual commands . . . 83

8.2.1 sre start [-d data] [-p] [--flavor … | --flavor-json … | --set-flavor-name …] [--xauth-file <file>] [--instructor-mode] <lab> [data_version] . . . 83
8.2.2 sre stop <running_lab> . . . 84
8.2.3 sre save [-o <file>] [--full-images] <running_lab> . . . 84
8.2.4 sre restore [--instructor-mode] <save_file | -> . . . 84
8.2.5 sre wipe . . . 85
8.2.6 sre connect [--shell <shell>] [--exec <argument…>] [--no-records] <running_lab> <device> . . . 85
8.2.7 sre eval [-p path] [--auto-eval] <running_lab> . . . 85
8.2.8 sre state <running_lab> <state_name> . . . 85
8.3 Admin-only commands . . . 85
8.3.1 sre exec [--shell <shell>] <running_lab> <device> <command…> . . . 85
8.3.2 sre set-instructor-mode <running_lab> and sre remove-instructor-mode <running_lab> . . . 86
8.3.3 sre eval-all [--display-grades / --no-display-grades] . . . 86
8.3.4 sre check [-p] <lab> [<state>] . . . 86
8.3.5 sre watch [--timeout <sec>] [--interval <sec>] [-H <regexp>] [-S <time>] [-L] <dir…> . . . 86
8.3.6 sre preload-images [--random-delay <sec>] <file_or_dir…> . . . 87
8.3.7 sre make-titles [-o <file> | -r] <directory> . . . 87
8.3.8 sre update [--main | --ref <tag|branch|commit>] [--check | --force] [-y] [--venv] . . . 88
8.4 Exam management . . . 89
8.4.1 sre set-exam [options] . . . 89
8.4.2 sre del-exam . . . 90
8.4.3 sre save-records -d <dir> [--only-last-record] . . . 90
8.5 Post-exam management . . . 90
8.5.1 sre cat [options] <file…> . . . 90
8.5.2 sre check-eval [-s <srelab>] <file…> . . . 90
8.5.3 sre re-eval -s <srelab> -p <prefix> [-d <outdir>] [-r] <file_or_dir…> . . . 90
8.5.4 sre sheet -o <output.ods> [-r] [-S <time>] [-F <time>] [--separate-instances] <file_or_dir…> . . . 90
8.5.5 sre outline [-o <summary.ods>] [-d <pdf_dir>] [-r] [-S <time>] [-F <time>] [--lang <lang>] [--no-timeline] [--remaining-time] [--separate-instances] [--users-file <file>] <file_or_dir…> . . . 91
8.6 Internal commands (GUI-only) . . . 91
8.6.1 sre list [--with-titles] . . . 91
8.6.2 sre export <running_lab> [--sep <N>] [--curved] [--shapes] [--reverse] [--random-seed <N>] . . . 92
8.6.3 sre pre-start-exam . . . 92
8.6.4 sre start-exam . . . 92
8.6.5 sre eval-exam . . . 92
8.6.6 sre end-exam . . . 92

9 Exam Reference **93**
9.1 exam.json — single source of truth . . . 93
9.2 GUI exam logic . . . 94
9.3 Commands . . . 94

10 Runtime & Internals **95**
10.1 Architecture . . . 96
10.2 Filesystem layout . . . 98
10.2.1 Running lab directory . . . 98

10.3 Configuration . . . 99
10.3.1 Constants (`src/SRE/params.py`) . . . 99
10.3.2 Environment variables . . . 100
10.3.3 Locale . . . 100
10.4 Dynamic module loading . . . 100
10.5 Data serialization . . . 100
10.6 Test execution . . . 101
10.7 Switches . . . 101
10.8 Course text (`informations`) . . . 102
10.9 Progress reporting . . . 103
10.10 Security model . . . 103
10.11 Archive format . . . 103
10.12 Save file format . . . 104

CHAPTER

ONE

OVERVIEW

SysResEval (or **SRE**, for *Système Réseaux Évaluation*) is a software suite for managing exercise sessions and evaluations in network and system administration courses. It is built on top of the Kathara framework, using Docker containers and VDE networking.[1]

It was developed at **IUT d'Orsay, Université Paris-Saclay** (France).

SysResEval can be used by students autonomously, in supervised exercise sessions, or in time-limited evaluations.

Students interact only with a GUI, `sysreseval`, which calls the `sre` CLI on their behalf through the small `sre-wrapper` setuid helper The instructor can also use the CLI to set up an exam, start a project for a student, connect to a machine, launch an eval, etc…

1.1 Student Workflow

The student starts the **sysreseval** GUI, then chooses and opens a *lab*.[2] Each lab presents several tabs:

- **Schema** — the network topology. By default, machines that disallow connection are drawn in red, but every lab can override colors and shapes for machines and networks. Some machines may be hidden from the diagram entirely.
- **Informations** — general background and instructions, written in Markdown.
- **Questions** — a list of items the student must complete. Each question is one of:
  - a Markdown text describing a task to perform on the machines (e.g. *configure the network on m1*),
  - a form whose fields capture specific facts (e.g. *what is the MTU of `eth0` on m1?*); fields may be free-text validated by a regexp (e.g. an IPv4 address), a multi-line text area, a dropdown, or a checkbox,
  - a free-form multi-line text answer.
- **Terminals** — an embedded shell for each machine the student is allowed to connect to. A lab may expose these as plain root shells or, for example, as a `login` prompt that restricts students to a particular user account.
- **Machines** — a status table for each machine: state, NAT network, exposed ports. The students can use it to launch separate terminals sessions to a machine (useful when several terminal sessions are needed on one machine)

[1] A first version was made with Marionnet and then rebuilt from scratch on Kathara / Docker.
[2] The word *lab* comes from Kathara (and Netkit). We use *lab* and *project* interchangeably in this documentation.

- **Networks** — the networks of the lab with their type: hub, switch or manageable switch. A manageable switch has VLANs and a console, which the students may open from this tab when the lab allows it (see *Hubs, switches and VLANs*).
- **Evaluations** — lets the student trigger an automated evaluation of their work and view the resulting grade table.
- **Apply Configuration** — lets the student put the project into a predefined state, for example a partial correction.

The **Questions**, **Evaluations**, and **Apply Configuration** tabs are optional and are shown only when the lab defines them.

Multiple labs can be open at once, letting a course be structured by topic instead of by session.

1.2 Monitoring of a session

During an exercices session, an instructor with the `sre watch` tool can monitor a whole class:

- consult the last evaluation result of any student (evaluations run periodically — the interval is a per-project parameter, typically one minute — even when the student does not trigger a self-evaluation).
- be alerted about any error during evaluations or by the lack of a recent evaluation
- display a list of all "grade elements" and the max, min and average grade of the students

The grade elements shown to the instructor can differ from those the student sees during a self-evaluation. For example, on a static routing exercise, the instructor may see detailed items (IPv4 address, default route, indirect route toward network A, etc.) while the student only sees a single "Configuration of machine m1".

1.3 Running exams

sysreseval manages time-limited exams. The instructor configures the exam and sysreseval starts each student's project — either immediately or at a scheduled time — displays a countdown, runs periodic evaluations (by default every 60 seconds), and shows a banner when the allotted time is up.

The instructor can change the duration during the exam, or even after it ends — the virtual machines are not stopped at the end (in practice, it is often easier to set a single duration for the whole class with a `dsh` call, then adjust it individually for students with special accommodations)

The student's final mark is the best overall grade across their evaluations.

1.4 Post-exam workflow

Each evaluation produces an archive file containing all of its data, readable via `sre cat`. If a student questions their score, the instructor can give a full explanation — for example, for a question about routing, by showing the output of `ip route` on m1 to reveal which routes were in place at the time. During exams, all terminal sessions are also recorded as asciinema files.

After the exam, the instructor can re-run the evaluation script — for instance to adjust the relative weight of items, or to fix a bug in the script — and then generate a per-student PDF with a detailed item-by-item breakdown, plus a recap ODS spreadsheet.

CHAPTER

TWO

EXAM ADMINISTRATION

2.1 Lifecycle at a glance

The model is **declarative**: the instructor writes a single file, `/var/lib/sre/exam.json`, and the GUI on each student PC does the rest. Two invariants make this work:

- **`exam.json` is the trigger.** Its presence puts the GUI in exam mode; its contents determine the current phase, the countdowns, and what to fire next. The instructor's only job is to create, edit, or delete this file — via `sre set-exam` and `sre del-exam`. The GUI polls it once per second and fires `pre-start-exam`, `start-exam`, `eval-exam`, and `end-exam` at the appropriate moments.
- **Every exam command is idempotent.** Each one can be re-run safely, which is what lets the GUI recover from transient failures and react to mid-exam edits. The instructor can extend the duration, add or remove a lab, or push back the end time at any point by editing `exam.json` (via `sre set-exam`); the change propagates within one second.

```
[Instructor]                          [GUI on each student PC, polling exam.json @ 1 Hz]
     |                                          |
     |  sre set-exam --labs tp1 \               |
     |    --start-after 09:00 \                 |  ← exam.json appears
     |    --end-before  11:00 \                 |
     |    --duration    120                     |
     |    --eval-interval 60                    |
     |                                          |
     |                              Phase: WAITING
     |                              • shows logo + countdown to 09:00
     |                              • ~60 s before start_after, sysreseval GUI fires
     |                                sre pre-start-exam (pulls images,
     |                                starts containers, writes pre_start_date)
     |                                          |
     +-- 09:00 ------------------------------->|
     |                              Phase: ACTIVE
     |                              • once projects are up, sysreseval GUI fires
     |                                sre start-exam (stamps started_at)
     |                              • shows remaining-time countdown
     |                              • fires sre eval-exam every 60 s
     |                              • student works, may self-grade
     |                                          |
     +-- 11:00 (or started_at + 120 min) ---->|
     |                              Phase: ENDED
```

(continues on next page)

(continued from previous page)

```
    |                              • sysreseval GUI fires sre end-exam once
    |                                (final concurrent eval + stamps ended_at)
    |                              • shows "That's All Folks"
    |                                        |
    |  sre del-exam                          |
    |  (removes exam.json, wipes projects)   |
    |  sre sheet / outline / re-eval         |  ← post-exam grading
```

2.2 Setting up an exam — `sre set-exam`

```
# Minimal - one lab, hard end time. No start time means "open immediately".
sre set-exam --labs s4/tp_ssh --end-before 11:00

# With some more options example
sre set-exam \
  --labs s4/tp_ssh s4/tp_dhcp:hard \
  --start-after  2026-06-15T09:00 \
  --end-before   2026-06-15T11:00 \
  --duration     90 \
  --eval-interval 30
```

In this case, each student gets 90 minutes, but every exam ends at 11:00 — even for students who start after 9:30.

Behaviour:

- **Incremental updates.** Only the options provided on the command line are written; everything else is preserved. `sre set-exam --duration 150` extends an in-progress exam without touching `labs` or `start_after`.
- **First-time requirements.** If `exam.json` does not yet exist, `--labs` is required, and so is at least one of `--end-before` / `--duration` (otherwise the exam would never end).
- **Date formats.** Both `--start-after` and `--end-before` accept full datetimes (`2026-06-01T09:00`, `2026-06-01 09:00`, `2026-06-01T09:00:00`) and time-only (`09:00`, `09:00:00`) — time-only is combined with today's date.
- **Lab validation.** Each lab name is checked against `sre list` (`get_lab_list(include_exam_only_labs=True)`); absolute paths must lie under `params.authorized_src_dir` (`/opt/sre/lab/` or `/home/admin1/`, etc..).
- **Resetting `start_after` resets state.** Changing `--start-after` clears `pre_start_date` and `started_at` so the GUI re-triggers `pre-start-exam` and `start-exam` for the new run.
- **Mis-exam edits.** Anything in `exam.json` may be changed while the exam is running; the GUI picks up changes within 1 s.
- **relative duration change**. During an exam, `--duration +X` or `--duration -X` add or substract X minutes.
-

Option	Effect
`--labs <lab[:fl ...>`	Authorised labs; `:flavor` selects a named `Flavor` preset (e.g. `tp1:hard`). Required when creating.
`--start- <dt>`	Exam opens after this moment. If omitted, the exam is considered open immediately.
`--end-be <dt>`	Exam closes after this moment, regardless of `started_at`.
`--durati <min>`	Hard cap on duration; the exam ends at `started_at + duration` *or* `end_before`, whichever fires first. Only effective once `started_at` is set. When updating an existing `exam.json` that already has a `duration` field, prefix the value with `+` or `-` to adjust the current duration (e.g. `--duration +30` adds 30 min, `--duration -15` subtracts 15 min).
`--eval-i <sec>`	Automatic-eval period (default 60).
`--record <bool>`	Record terminal sessions (`true`/`false`/`0`/`no`). Default `true`.

See *Exam Reference* for how the GUI drives the exam (phase computation and per-phase actions).

2.3 Monitoring an exam

Once each student machine mirrors its archives into a shared directory subtree on the instructor's machine (see the Installation chapter), `sre watch <directory>` provides a live dashboard of the exam:

- **Remaining time and latest grade** per student.
- **Inactivity alerts** when no new archive has appeared for a student in the last X seconds — usually a sign that `sysreseval` crashed or the machine was shut down.
- **Evaluation-error alerts** flagging any error raised by the last eval. A well-written lab should never produce errors; if a command is legitimately allowed to fail, pass `allow_error=True` to `Grade.cmd()` so the failure is not reported.

The dashboard shows **one row per running project instance** (hostname, lab and start time), so a student who opened the same lab twice gets two rows, each with its own grade and alerts, instead of one row whose grade jumps between the two. A lab that was stopped and started again also keeps its old row, with an inactivity alert, until you dismiss it (`P` key) or hide it (`S` key). To see a single row per student and lab, the most recently started instance, run `sre watch -L <directory>` or press `L` in the dashboard. See *CLI Reference* for the other options and keys.

2.3.1 Clean up after an exam

```
sre del-exam
```

Removes `exam.json` and wipes every running project. Archive directories are preserved.

Note: in exam mode, quitting the GUI does not stop the Docker containers (unlike normal mode), so the instructor can keep them running afterwards — for instance to re-run a corrected version of the project if a bug surfaces during the exam.

2.4 Exam-only labs

Lab names containing any substring from `params.exam_only_affix = ["_EXAM_", "_OLD_", "_DRAFT_", "_TESTS_"]` are hidden from `sysreseval`'s lab picker. This is how you keep an exam lab invisible until it appears in `exam.json`:

- `/opt/sre/lab/s4/ssh_EXAM_final.py` — hidden from listings, but allowed in `--labs`.
- `/opt/sre/lab/_DRAFT_/work_in_progress.py` — hidden in any subdirectory whose name contains a flagged affix.

`sre set-exam --labs ...` and all the commands from privileged users (`sre start...`) accept hidden labs of course.

2.5 Post-exam workflow

2.5.1 Inspect a single archive

```
sre cat --grades --answers /var/lib/sre/archives/20260615110000_*.zst
```

Field option	Shows
`--grades`	Per-element grades, total grade, max
`--answers`	Student answers (incl. login, hostname, fullname, email, exam timing)
`--errors`	Errors raised during grading
`--tests`	Raw test results per machine (large)
`--data`	Serialized `Data` instance for the lab
`--files` / `--extract-files`	List or extract files saved into the archive
`--json`	One JSON dict per file on stdout (script-friendly)

2.5.2 Re-grade with a corrected project file

If a grading bug surfaces during the exam, fix-it and re-grade every archive:

```
sre re-eval -s /opt/sre/lab/_EXAM_/ssh_corrected.py \
            -p corrected \
            -d /tmp/regraded \
            -r /home/a/results/
```

Output archives appear in `/tmp/regraded/` with the `corrected` prefix. Use `sre check-eval -s ...` first to preview the diff without writing any file.

This only works if the corrected project file does not require any **new** command to be run on the virtual machines: re-evaluation replays the grading logic against the test output already stored in the archive, so a command that was never executed during the exam (say, `ip route` on `m1`) cannot be recovered after the fact. Within that limit, fixing a grading bug post-exam is straightforward.

To preview the impact of the correction on a single archive without writing anything, use:

```
sre check-eval --srelab corrected_project.py archive_file
```

which prints the diff between the stored grades and the re-graded ones.

The overall grade is computed by the `Grade0.mark_exo_eval()` method, which can be overridden to rescale or cap the final mark. For example, to multiply every mark by 1.1 and round up to the next 0.1, override `mark_exo_eval()` in the lab's `Grade` class:

```
def mark_exo_eval(self):
    if not self._grade_list or self._total_max == 0:
        return None
    import math
    return min(
        self._maximum_mark,
        math.ceil(10 * 1.1 * self._maximum_mark * self._total_grade / self._total_max) /␣
↪10,
    )
```

2.5.3 Per-student PDF reports + summary spreadsheet

To create per-student PDF reports and a grade spreadsheet:

```
sre outline \
  -o /tmp/summary.ods \
  -d /tmp/reports \
  -r /var/lib/sre/archives/
```

Useful options:

- `--users-file <file>` — auxiliary `LOGIN NAME EMAIL` file (whitespace- or CSV-separated, `#` for comments) that supersedes the names and emails embedded in the archives. Handy when student logins are pseudonyms.
- `--remaining-time` — include each student's remaining exam time (from `exam_time_remaining` in the archive) in the summary.
- `--no-timeline` - omit the evaluation history table from PDF reports
- `--no-parts` - do not group PDF grade rows by GradePart (flat list, no subtotals)
- `--separate-instances` - one PDF report and one ODS row per running project instance instead of one per student. By default a student who opened the same lab several times gets a single report: the best grade is taken across all instances and the evaluation history shows one table per instance, headed by its start time. With this option each instance gets its own report (the PDF name ends with the instance start timestamp) and its own row in the summary, with a `project_start` column.

2.5.4 Export to a spreadsheet

To get a full sheet of all grades elements of all evaluations with per-question summary:

```
sre sheet -o /tmp/exam-results.ods -r /var/lib/sre/archives/
```

Each archive is one row of the lab's sheet, with a `project_start` column identifying the running project instance it comes from; rows are sorted by login, hostname, project start and evaluation date. The `Sessions` sheet keeps the best score per student; add `--separate-instances` to get one line per running project instance instead, for a student who opened the same lab several times.

CHAPTER

THREE

LAB AUTHORING GUIDE

A lab is a Python module that defines three classes: `Data`, `NetScheme`, and `Grade`, each inheriting from the corresponding `*0` base class in `lib_sre.py`. An optional `Flavor` class can be defined to parameterize the lab at start time.

A lab is usually a single `.py` file.

Directory labs

SRE also supports **directory labs** — a subdirectory containing a `srelab.py` plus per-state file trees (`initial/`, `state1/`, …) that are copied into the containers at the matching state transition. This is the Kathara layout, but in practice it's rarely needed: most setup is better expressed in code, since `Data`-derived values (random IPs, secrets, etc.) can then be injected into the generated files. The directory form is mainly useful when you need to ship payload that can't reasonably be generated at runtime — a pre-built apt package, a large data file, or anything else you don't want to download from the Internet on every lab start.

```
/opt/sre/lab/s4/tp_ssh/
+-- srelab.py          ← the module
+-- initial/           ← files injected at startup
|   +-- router/        ← files for the machine named "router"
|   |   +-- etc/network/interfaces
|   +-- all/           ← files copied to every machine
|       +-- etc/motd
+-- state1/            ← files injected when "sre state <lab> state1" is called
    +-- router/
        +-- etc/...
```

3.1 The authoring workflow on the CLI

While iterating on a `srelab.py`, you typically alternate between editing the module and exercising it with a handful of `sre` subcommands. Full reference lives in *CLI reference*; this section just names the commands you'll reach for as an author and explains the one option that exists for authors only.

3.1.1 `sre check [-p] <lab> [<state>]`

Static validation, no containers deployed. Imports the module, runs `Data.compute_pre_generate()` → `Data.generate()` → `data.compute_post_generate()`, checks that the `Data` instance survives the `data.json` round trip (`to_json()` / `from_json()`, `pack()` / `unpack()`, as `sre start` and `sre eval` do), instantiates `NetScheme`, applies `initial()` (op registration only), then calls `Grade.grade()` once. Catches import errors, missing fields, bad topology, and exceptions raised at registration time. With a trailing `<state>` argument, the named `@sre_state` method is also exercised. Run this first after every non-trivial edit — it's

much faster than starting a real lab. The numbered sections of `informations` are resolved and printed as an outline, and the `@@ref@@` references of the question texts are checked (see *Numbered sections*).

3.1.2 `sre start --debug-project [-p] <lab> [--xauth-file <path>]`

The author counterpart to `sre start`. Privileged only and incompatible with `--user`. Deploys the lab exactly like a normal start, then drops a `.private/debug_project` marker that the rest of the runtime keys on. Effect:

- **Topology**: every machine is shown in the GUI's *Machines* tab — even those declared `hidden=True` or `allow_connection=False`. *Connect* buttons are exposed for all of them so you can step inside infrastructure machines (DNS, monitors, helpers) to inspect their state. A *Full Schema* tab, right after *Schema*, draws the topology with the hidden machines too: they, their links and the networks only they use are in red (the *Schema* tab stays the student's view).
- **Restrictions lifted**: user-mode guards on `sre state` and `sre connect` (which normally refuse to touch hidden machines or apply `user_allowed=False` states) are bypassed.
- **Grading**: when you `sre eval` a debug project, marks are always shown and every grade scope is surfaced — `no_mark_on_self_grade`, `hide_potential_penalty_grades_in_self_grade` and the periodic-eval gating are all ignored.
- **Operations log**: every state application (`initial` at start, `sre state`, the `save`/`restore` lifecycle states) and every evaluation appends what it executed to `<project dir>/operations.log`: one header per run, then `stepN - on <machine|host> : <operation>` lines with the command output and exit code. File writes, appends and copies show the path, mode, owner and size, followed by the content written (text only, capped like command outputs). Each evaluation ends with its grade elements (`grade - [<part>] <title> : <grade> / <max>`, with `(self-eval only)` / `(exo-eval only)` when the scope is restricted) and the totals and marks of both scopes. The GUI shows it live in the *Log* tab. Never written for a normal project.
- **Instructor texts**: the texts wrapped in *`instructor()`* are kept in `info.json`, as for a project in instructor mode, and the GUI shows its *Instructor mode* button on the project to display or hide them.
- **`-p` is implicit**: `--debug-project` always treats the lab argument as a filesystem path, so you can point it directly at the `.py` you're editing (e.g. `~/labs/draft.py`) without copying it into `/opt/sre/lab/` first.

The marker survives container restarts and is cleared when the project is `sre stop`ped or `sre wipe`d.

`--xauth-file <path>` is the other authoring-time option on `sre start`. By default, machines marked `x11_host=True` pick up the host's X magic cookie from the `SRE_XAUTH_COOKIE` environment variable (the sre-wrapper sets this for the GUI flow). When you start a project by hand from a shell where that variable isn't set — typical for `--debug-project` — pass `--xauth-file ~/.Xauthority` to source the cookie from a file instead. Privileged only.

3.1.3 `sre connect <running_lab> <device>` and `sre exec <running_lab> <device> <command…>`

`sre connect` opens an interactive shell inside `<device>` (using the machine's configured `shell`), or the management console when `<device>` is a managed switch (a network with `'mode': 'managed'`, see *Switches and VLANs*). `sre exec` runs a one-shot command and returns its stdout/stderr/exit code — useful for scripting checks against a running lab without holding a TTY. On a managed switch `sre exec <running_lab> <network> <command…>` runs one console command (`vlan/print`, `port/setvlan @pc1 10`). `sre exec` is privileged only; both honor the debug-project marker, so they can target hidden machines when the project was started with `--debug-project`.

`sre connect --exec <argument…>` is a hybrid: launches the machine's shell, runs the command, exits. Unlike `sre exec` it goes through the shell launcher and is available to students.

3.1.4 `sre state <running_lab> <state_name>`

Applies a non-`initial` `@sre_state` method against the live containers. The main use case during authoring is exercising the instructor's "fully-configured" state (often called `final`) so you can run `sre eval` against a known-good configuration and verify the grading code reports the expected full marks. In a non-debug project, students can only apply states declared with `@sre_state(user_allowed=True)` *and* only when the module sets `allow_user_states = True` — `--debug-project` lifts both guards.

3.1.5 `sre eval <running_lab>`

Runs an evaluation against the live containers — equivalent to pressing the *Start evaluation* button in the GUI. The evaluation also refreshes the project's view of the lab module, so any edits to `informations`, questions, or grade rubric take effect on the next display without needing to restart the lab.

3.1.6 `sre wipe`

Stops every running Kathara lab and removes everything under `/var/lib/sre/projects/`. The reset button between iterations: when a lab is misbehaving and `sre stop <running_lab>` isn't enough (orphaned bridges, broken Kathara state, leftover marker files), `sre wipe` brings the host back to a clean slate. It does **not** touch `exam.json` or archive directories.

A typical edit-test cycle:

```
sre check -p ~/labs/draft.py final            # validate module + the 'final' state
sre start --debug-project -p ~/labs/draft.py  # deploy, with debug visibility
sre connect <running_lab> router              # poke around
sre state <running_lab> final                 # apply the instructor's known-good state
sre eval <running_lab>                        # confirm grade == max
sre wipe                                       # reset before the next edit
```

3.2 Identifying a running lab and where its files live

Every running project lives in its own subdirectory of `/var/lib/sre/projects/`. The directory name encodes the launch timestamp, the lab name, and the student username:

```
/var/lib/sre/projects/{YYYYmmddHHMMSS}@@@{lab_name}@@@{username}/
```

e.g. `20260606143215@@@s4@ssh@@@alice`. List the directory to see what's currently running:

```
ls /var/lib/sre/projects/
```

Partial names are accepted. Every subcommand that takes a `<running_lab>` argument resolves it by substring match against the directory listing — you only need to type enough of the name to uniquely identify one project. If your substring matches several running labs, `sre` prints the candidates and exits so you can disambiguate; if it matches none, you get `no running lab matches '...'`. In practice the lab name itself (or even a fragment of it) is usually enough:

```
sre connect ssh router        # works if only one ssh project is running
sre connect 20260606 router      # disambiguate by timestamp prefix
sre connect alice router      # disambiguate by username
```

Inside the project directory you'll find:

Path (relative to the project dir)	Contents
`info.json`	Public metadata read by the GUI: lab title, `informations`, machine list, question list. Written at `sre start` and refreshed on `sre eval`.
`answers/answers.json`	Student answers to `question_text` / `question_form` blocks.
`answers/cheat.json`	Optional instructor-provided cheat answers keyed by state name (used by `cheat_answers=`).
`shared/`	Bind-mounted into every container as `/home/sre/{running_lab_name}/` when the module sets `shared_path = True`. The host-side path for file exchange.
`.private/`	Mode `0o700`. Holds everything the runtime needs but students shouldn't touch.
`.private/data.json`	The serialized `Data` instance, reloaded on every later operation.
`.private/srelab`	Symlink to the lab's `.py` file (whatever `sre start` was pointed at).
`.private/files/`	Target of `self.cp_from_host()` / `self.cp_to_host()` — also where `cp_from_host` resolves relative source paths.
`.private/records/`	Asciinema recordings of terminal sessions, if `record_sessions` is enabled.
`.private/eval_in_progres`	PID lock created during `sre eval` to prevent concurrent evaluations.
`.private/debug_project`	Marker dropped by `sre start --debug-project` (see above).
`.private/instructor_mode`	Marker of the instructor mode (`sre start --instructor-mode`, `sre set-instructor-mode`; see *Instructor-only texts*).
`.private/auto_eval.log`	One line per student-triggered self-eval; drives the `self.auto_eval_count` counter exposed to `Grade.grade()`.

Evaluation archives are **not** stored under the project directory — they go to `/var/lib/sre/archives/` (plus any extra `archive_dirs` declared at module level), named `{YYYYmmddHHMMSS}_{running_lab_name}.zst`.

3.3 The four classes at a glance

A lab module defines up to four classes, each with a narrow role:

- **`Data`** — a `@dataclass` that holds every per-instance parameter of the lab (IP addresses, secrets, ports, randomized values). `Data.generate()` is called once at `sre start`; the resulting instance is serialized to `.private/data.json` and reloaded on every later operation.
- **`Flavor`** *(optional)* — a `@dataclass` that lets a single `srelab.py` produce different variants of the lab at start time (e.g. random vs. fixed addresses, easy vs. hard). The GUI renders `flavor_form` as a form; the chosen `Flavor` is then passed to `Data.generate(flavor)`.
- **`NetScheme`** — declares the network topology (`_machine_specs`, `_network_specs`, `_topology`) and the imperative configuration of each *state*. The `initial` state runs at `sre start`; additional `@sre_state` methods can be applied later with `sre state <lab> <name>`.
- **`Grade`** — registers what to evaluate and how to score it. It does **not** run anything itself: SRE drives a multi-step `run_tests()` loop that calls `Grade.grade()` repeatedly, executing the commands registered by each call between passes. See *The grading lifecycle*.

The flow at `sre start` is: `Data.generate(flavor)` → `NetScheme(data).initial()` (which lays out interfaces and writes files). The flow at `sre eval` is: reload `data.json` into a fresh `Data`, rebuild `NetScheme`, then run `Grade.run_tests()` against the live containers.

3.4 Data class

Data is a dataclass that holds all lab-specific parameters (IP addresses, random secrets, etc.). It inherits from Data0.

```
from dataclasses import dataclass
from ipaddress import IPv4Interface
from SRE.lib_sre import Data0
from ips import random_ipv4networks, random_ipv4s

@dataclass(slots=True)
class Data(Data0):
    secret: str = ''
    vlan_id: int = 0

    @classmethod
    def generate(cls, flavor=None):
        data = cls(secret="changeme", vlan_id=42)
        # data.nets and data.ips are injected automatically by Data0.__post_init__
        data.nets.lan, data.nets.mgmt = random_ipv4networks([24, 24],␣
↪from_private_network=True)
        data.ips.router = random_ipv4s(data.nets.lan, 1)[0]    # an IPv4Interface, prefix␣
↪included
        data.ips.printer = IPv4Interface('192.0.2.7/24')        # explicit values work␣
↪too
        return data
```

Key points:

- data.ips holds IPv4Interface values (address + prefix length); data.nets holds IPv4Network values; data.ips6 holds IPv6Interface values; data.nets6 holds IPv6Network values; data.macs holds netaddr.EUI (MAC address) values.
- All five containers are injected automatically into every Data0 subclass instance by __post_init__ — no declaration needed (and do not declare fields with these names).
- The containers enforce their types: assigning a plain string to data.ips raises TypeError; always wrap with the correct type before assigning. The families are kept apart: data.ips refuses an IPv6Interface and data.ips6 an IPv4Interface.
- data.ips requires IPv4Interface (not bare IPv4Address) — always assign with a prefix, e.g. IPv4Interface('10.0.0.1/24'); likewise data.ips6 requires IPv6Interface, e.g. IPv6Interface('2001:db8::1/64').
- Data.generate(flavor) is called once at sre start; the result is saved to data.json.
- Serialization: the containers are stored as strings; standalone fields of type IPv4Address / IPv6Address, IPv4Interface / IPv6Interface, IPv4Network / IPv6Network, EUI and nested Data0 subclasses are encoded with markers and restored with their exact type. Any other field must be JSON-serialisable (sre check verifies the round trip).

3.4.1 IP helper functions (from /opt/sre/lib/ips.py)

```
from ipaddress import IPv6Network
from ips import (random_ipv4networks, random_ipv4s, random_ipv6networks, random_ipv6s,
                 random_ips_from_topology)
```

(continues on next page)

(continued from previous page)

```
# n disjoint IPv4 networks with the given prefix lengths (RFC 1918 ranges with␣
↪from_private_network=True):
nets = random_ipv4networks([24, 28, 24], from_private_network=True)
# n distinct random IPv4Interface values (prefix included) in a network;
# the network and broadcast addresses are never returned:
data.ips.host, data.ips.gw = random_ipv4s(nets[0], 2, exclude_nets=[nets[1]])

# IPv6 twins - search space: global unicast 2000::/3 by default, unique local fd00::/8␣
↪with
# from_private_network=True, or anything with from_network=IPv6Network('2001:db8::/32'):
data.nets6.lan, data.nets6.wan = random_ipv6networks([64, 64], from_private_network=True)
# n distinct IPv6Interface values; offset 0 (the subnet-router anycast address) is never␣
↪returned:
data.ips6.host, data.ips6.gw = random_ipv6s(data.nets6.lan, 2)

# Fill data.ips (and data.ips6 with ipv6=True) for every machine of a topology from
# data.nets / data.nets6 - data.ips.m for single-homed machines, data.ips.m_net␣
↪otherwise:
random_ips_from_topology(data, NetScheme._topology, ipv6=True)   # ipv4=False,␣
↪ipv6=True: IPv6-only lab
```

`random_ipv4s_with_range` / `random_ipv6s_with_range` draw ordered address ranges (for DHCP pools), `random_mac_address` random unicast MAC addresses.

3.4.2 IPv6 labs

Kathara starts every container with IPv6 **disabled** (`net.ipv6.conf.all.disable_ipv6=1`). A lab turns it on with the module-level option:

```
ipv6 = True   # IPv6 enabled in every machine of the lab
```

`Machine(ipv6=False)` (an `'ipv6': False` entry in `_machine_specs`) opts one machine out, `Machine(ipv6=True)` opts one in when the module option is absent. With IPv6 enabled Kathara sets `disable_ipv6=0`, `net.ipv6.conf.all.forwarding=1` and `accept_ra=0` in the container (every interface gets its link-local address), so, as for IPv4 (`set_ip_forward`), the `initial` state turns forwarding off on the hosts with `set_ipv6_forward(self, m, False)`. `sre export` writes the effective flag as `m[ipv6]="true"` in `lab.conf`.

A dual-stack lab keeps both families in the same `net_config` entry, IPv4 first:

```
from ipaddress import IPv4Network, IPv6Network
from net_config import get_net_config_from_topology, set_net_config_entry,␣
↪set_ip_forward, set_ipv6_forward

# by hand:
self.net_config = {
    'pc': [([d.ips.pc, d.ips6.pc], [(IPv4Network('0.0.0.0/0'), d.ips.gw), (IPv6Network(
↪'::/0'), d.ips6.gw)])],
}
# or from the topology (data.ips6 / data.nets6 follow the data.ips / data.nets naming; a␣
↪machine
# without an entry in data.ips6 - a SLAAC or IPv4-only host - simply gets no IPv6 part):
```

(continues on next page)

(continued from previous page)

```
self.net_config = get_net_config_from_topology(self, gateway='gw', ipv6=True)   #␣
↪default_route6=... for a ::/0 on gw;
# default_route (172.17.0.1, the Docker bridge, for a bridged=True gateway) /␣
↪default_route=None: no 0.0.0.0/0 on gw

# in initial(): set_net_config_entry() emits `ip addr add` / `ip route add` for both␣
↪families
set_net_config_entry(self, 'pc', self.net_config['pc'])
set_ipv6_forward(self, 'pc', False)
```

Readers take `ipv6=True` to *also* return the IPv6 side (link-local and temporary addresses are left out): `get_ip_addresses`, `get_routes` (adds `ip -6 route`, `default` → `('::', 0)`), `get_net_config_entry`, `get_persistent_net_config_entry` (parses `iface ethN inet6 static` stanzas); `set_persistent_net_config_entry` writes one `inet` and one `inet6` stanza per interface; `eval_net_config` compares both families (`net_config_entry_family(entry, 6)` grades one family apart); `eval_ping(self, 'pc', 'srv', ipv6=True)` pings the IPv6 address of a machine named in `net_config` (an IPv6 literal needs no flag); `create_hosts_file(..., ipv6=True)` adds the IPv6 lines; `setup_simple_tcp_server(.. ., ipv6=True)` listens on `::`.

Router advertisements / SLAAC: on the router `set_radvd(self, 'gw', {2: [d.nets6.lan]})` writes `/etc/radvd.conf` and starts radvd; on the hosts `set_slaac_client(self, 'pc')` sets `accept_ra=2` and `autoconf=1` (a forwarding host ignores RAs otherwise). Grade with `get_ip_addresses(self, 'pc', ipv6=True)` (the SLAAC address is in `d.nets6.lan`) and `get_routes(self, 'pc', ipv6=True)[('::', 0)]` (next hop `fe80::…`). The Docker bridge has no IPv6 unless the daemon is configured for it: no IPv6 reaches the outside through `bridged=True` machines, and `set_nat_gateway` stays IPv4. See `lab/sre/_DRAFT_dummy/ipv6_example.py` for a complete dual-stack lab.

3.4.3 `compute_pre_generate` and `compute_post_generate`

Two optional lifecycle hooks let a `Data` subclass derive auxiliary values without storing them in `data.json`. Override either as needed; the defaults are no-ops.

Hook	Signature	Receive	Purpose
`compute_pr`	`@classmethod def compute_pre_generate(cls, flavor=None)`	class	Set **class-level** attributes derived from `flavor` (machine lists, topology sizes, role tables, etc.) before any `Data` instance exists.
`compute_po`	`def compute_post_generate(se]`	instanc	Set **instance-level** attributes derived from the dataclass fields after generation or reload.

When they run:

1. At `sre start`: `Data.compute_pre_generate(flavor)` → `Data.generate(flavor)` → `data.compute_post_generate()`.
2. At `sre eval`, `sre state`, `sre connect`, etc.: after `data.json` is reloaded into a fresh `Data` instance via `from_json` / `from_dict` / `unpack`, both hooks run again — pre-generate first (with the persisted `flavor`), then post-generate.
3. `sre check` exercises the same sequence as a sanity check.

Because the hooks run on every reload, they should be **deterministic and side-effect-free**: no random number generation, no IP allocation, no file I/O. Anything that must be randomized once and then persisted belongs in `generate()`.

Typical use case — derive the machine list and per-machine spec from a `Flavor`:

```
@dataclass(slots=True)
class Data(Data0):
    secret: str = ''

    @classmethod
    def compute_pre_generate(cls, flavor=None):
        if flavor is None:
            flavor = Flavor()
        match flavor.network_size:
            case "small":
                r_max, m_max = 2, 2
            case "medium":
                r_max, m_max = 3, 4
            case _:
                r_max, m_max = 4, 7
        cls.routers = [f"r{i}" for i in range(1, r_max + 1)]
        cls.non_routers = [f"m{i}" for i in range(0, m_max + 1)]
        cls.machine_specs = {'gw': {'bridged': True}}

    @classmethod
    def generate(cls, flavor=None):
        data = cls(secret=random_password(16))
        # cls.routers / cls.non_routers are already populated by compute_pre_generate
        data.nets.lan = random_ipv4networks([24], from_private_network=True)[0]
        return data
```

`NetScheme.build()` and `Grade.grade()` can then read `self.data.routers` / `self.data.non_routers` directly, even on reload, without those fields needing to live in `data.json`.

3.5 `Flavor` class

A `Flavor` is an optional dataclass that parameterizes a lab at start time. If defined, the GUI presents a form to the student before starting.

A typical use case — see `lab/sre/static_routing.py` — is to switch a lab between two modes: when students work on it autonomously or during an exam, randomized IP addresses are preferable so each student has their own topology; in a supervised classroom, the instructor may want every student to share the same values, so a single explanation at the board applies to everyone. A `Flavor` lets the lab expose both modes from the same `srelab.py` and pick one at start time.

```
from dataclasses import dataclass
from typing import Tuple
from SRE.lib_sre import Flavor0

@dataclass(slots=True)
class Flavor(Flavor0):
    nb: int = 0

    flavor_form = """
    Number of clients: @@{nb:[0-9]+}@@
    Mode: @@{mode:>easy|hard}@@
```

(continues on next page)

(continued from previous page)

```
    """

    def allowed_by_user(self) -> Tuple[bool, str]:
        """Return (True, '') if the student is allowed this flavor."""
        if self.nb <= 5:
            return True, ""
        return False, "Maximum 5 clients allowed."

# Named presets (accessible as Flavor.easy, Flavor.hard, etc.)
Flavor.easy = Flavor(nb=1)
Flavor.hard = Flavor(nb=5)
```

The form takes the same fields as `question_form()` (`@@{field:regex}@@` single-line text, `@@{field:+N}@@` multi-line text of N lines, `@@{field:>a|b}@@` dropdown, `@@{field:?}@@` checkbox), plus `@@{name::Label}@@`, a button that submits the form.

Module-level control:

```
flavor_form_at_startup = True   # show the flavor form when the student opens the lab
```

`Flavor` API:

Method	Description
`to_dict()` / `from_dict(d)`	Serialize/deserialize field values
`from_form_dict(d`	Build a `Flavor` from form field strings, coercing to declared types (`int`, `bool`, `float`, `str`). Extra keys are set as plain attributes.
`allowed_by_user(`	Returns `(bool, message)`. Override to restrict what students can choose.

In `Data.generate(flavor)`, check `if flavor is not None` before using flavor fields.

3.6 `NetScheme` class

`NetScheme` declares the lab's network topology and exposes the imperative API used by every state method. Topology is declared at class level via three dicts; per-state actions live in methods decorated with `@sre_state` (covered in the next section).

```
from SRE.lib_sre import NetScheme0, sre_state, make_tr

tr = make_tr('en')

class NetScheme(NetScheme0):
    # Declare machines: keys are machine names, values are Machine kwargs
    _machine_specs = {
        'router':  {'color': 'green'},
        'client':  {},
        'hidden':  {'hidden': True, 'allow_connection': False},
    }

    # Declare network options: display hints, type of switch (hub by default)
    _network_specs = {
```

(continues on next page)

(continued from previous page)

```
        'lan':  {'color': 'yellow'},
        'mgmt': {'color': 'gray'},
    }

    # Declare topology: net_name → list of machines (or dict with explicit interface␣
↪numbers)
    _topology = {
        'lan':  ['router', 'client'],          # auto interface numbering
        'mgmt': {'router': 0, 'hidden': 1},    # explicit interface numbers
    }

    def __init__(self, data, running_lab_name):
        super().__init__(data=data, running_lab_name=running_lab_name)

        # Markdown text for the Informations tab (supports make_tr for i18n); see
        # "Numbered sections, references and table of contents" for the numbering helpers
        self.informations = tr(
            "## Lab description\nConfigure routing.",
            fr="## Description\nConfigurez le routage.",
        )
```

3.6.1 Declarations and accessors

Attribute / Method	Description
`_machine_specs`	Class-level dict: `{name: {Machine kwargs}}`
`_network_specs`	Class-level dict: `{net_name: {Network kwargs}}` — display hints and type of switch, see *`Network` parameters*
`_topology`	Class-level dict: `{net_name: [machine,...]}` or `{net_name: {machine: iface_index}}`
`self.data`	The `Data` instance
`self.informations`	Markdown text (or `TranslatedText`) for the Informations tab
`self.get_machines()`	Iterator over all `Machine` objects
`self.get_machine_names()`	Iterator over machine name strings
`self.get_networks()`	Iterator over all `Network` objects

3.6.2 `Machine` constructor parameters

Parameter	Default	Description
`image`	`params.default_do`	Docker image
`bridge`	`False`	Enable host-network bridging (required for `ports`)
`x11_ho`	`False`	Forward X11 to the host: inject `SRE_HOST_IP` (and the host's X magic cookie as `SRE_XAUTH_COOKIE`) so GUI apps launched in the container display on the host's X server. Requires `bridged=True` (refused at startup otherwise)
`hidden`	`False`	Hidden from the Machines tab
`allow_`	`True`	Show *Connect* button in the GUI
`shell`	`None`	Shell launched by `sre connect` (interactive terminal session)
`kathar`	`None`	Shell Kathara uses to execute startup `exec_commands` inside the container
`privile`	`None`	Run the container in privileged mode. Refused at startup unless `params.allow_privileged_machines = True`
`ipv6`	`None`	Enable IPv6 in the container. `None`: the module-level `ipv6` option decides (Kathara's own setting, off, when that one is absent too); `True` / `False`: per-machine override. See *IPv6 labs*
`color`	`None`	Node color in the SVG diagram
`exec_c`	`[]`	Commands run at container start
`sysctl`	`{}`	Kernel parameters, as an iterable of `"net.<key>=<value>"` strings (a dict is iterated over its keys, so `{"net.ipv6.conf.all.accept_ra=2": True}` works; a `{name: value}` dict is rejected by Kathara). Merged after Kathara's defaults, so they win
`envs`	`{}`	Environment variables
`ports`	`[]`	Port mappings, e.g. `["80XX:80/tcp"]` (X = wildcard digit)
`ulimit`	`{}`	ulimit settings
`volume`	`{}`	Volume mounts

Port wildcards: `"80XX:80/tcp"` allocates the first free port in 8000–8099.

3.6.3 `Network` parameters

Each `_network_specs` entry gives the keyword arguments of one network; a network that only appears in `_topology` takes the defaults.

Parameter	Default	Description
`color`	`None`	Node color in the schema
`shape`	`None`	Node shape in the schema
`mode`	`'hub'`	Type of switch the machines are plugged into: `'hub'` (every frame reaches every machine), `'switch'` (the switch learns the MAC addresses: a machine only receives its own traffic and the broadcasts) or `'managed'` (a switch with VLANs and a management console). See *Switches and VLANs*
`allow_`	`True`	Managed switch only: students may open its console (*Connect* button of the **Networks** tab, `sre connect`)
`vlans`	`{}`	Managed switch only: VLANs of the ports, by machine

3.6.4 Switches and VLANs

By default the machines of a network share a hub, as in every lab written before the switch types existed. `'mode': 'switch'` and `'mode': 'managed'` change that per network — the whole feature (the Networks tab, the console and its commands, states, grading, limits) is described in *Hubs, switches and VLANs*:

```
_network_specs = {
    'lan': {'mode': 'managed', 'allow_connection': False,
            'vlans': {'pc1': 10,                              # access port: untagged␣
↪frames in VLAN 10
                      'pc2': 20,
                      'r1': [10, 20],                         # trunk port: VLANs 10 and␣
↪20, tagged
                      'r2': {'vlan': 1, 'trunk': [30]}}},     # native VLAN 1 + VLAN 30␣
↪tagged
    'dmz': {'mode': 'switch'},
}
_topology = {'lan': ['pc1', 'pc2', 'r1', 'r2'], 'dmz': ['r1', 'srv'], 'old': ['srv', 'pc2
↪']}   # 'old' is a hub
```

- VLAN IDs go from 1 to 4094. A machine that `vlans` does not name stays in the default VLAN of the switch (VLAN 0, untagged) with every other undeclared port; a trunk port without `vlan` keeps that default VLAN for its untagged frames.
- On a trunk port the machine handles the tags itself, e.g. `ip link add link eth0 name eth0.10 type vlan id 10`.
- A switch that is not a hub drops the 802.1Q frames of a VLAN the port is not a member of: a trunk between two machines needs a hub or a managed switch with trunk ports, not a plain `'switch'`.
- An unknown `mode`, `vlans` on a network that is not managed, a machine that is not on the network or an invalid VLAN ID raise `ValueError` when the `NetScheme` is built, so `sre check` reports them.
- A managed switch has a management console: `sre connect <running_lab> <network>`, the *Connect* button of the **Networks** tab, a terminal in the **Terminals** tab. In user mode only the commands of `params.switch_user_commands` reach the switch, and `'allow_connection': False` closes the console to students.
- A state runs console commands with `self.switch_cmd(network, command, ...)`, where `@machine` stands for the port of a machine (see *Switch operations*); `grade()` reads the switch with `self.test_switch(network, command)` and `get_switch_ports()` (see *Switch helpers* in the Grading Library Reference).
- A save file and `sre export` keep the type of each switch and the declared VLANs, not what was changed on a console afterwards. A mode other than `hub` needs the VDE network plugin of the Kathara fork, installed by `make network-plugin` (see *Installation*).

See `lab/sre/_DRAFT_dummy/switch_example.py` for a complete lab: the three types of network, access and trunk ports with a router on a stick, an open and a closed console, two states built on `switch_cmd()` and a grade reading the switch.

3.6.5 Instructor-only texts — `instructor()`

`instructor(text, color=None, background=None)` marks a text as meant for the instructor: notes, hints, the solution. It takes a plain string or a `tr()` text and combines with `+` like any other text:

```
from SRE.lib_sre import NetScheme0, Grade0, instructor, make_tr
```

(continues on next page)

(continued from previous page)

```
class NetScheme(NetScheme0):
    def __init__(self, data, running_lab_name):
        super().__init__(data=data, running_lab_name=running_lab_name)
        self.informations = instructor(tr("""
            ### Solution
            - add the default route on `client`;
            - the `final` state does it.
            """, fr="""
            ### Corrigé
            - ajouter la route par défaut sur `client` ;
            - l'état `final` le fait.
            """)) + tr("Configure the routing.", fr="Configurez le routage.")

class Grade(Grade0):
    def grade(self):
        super().grade()
        self.question_text(
            title=tr("Gateway") + instructor(" (10.0.0.1)"),
            description=tr("Which gateway does the client use?")
                        + instructor(tr("\n\nAny notation is accepted."), color="#1a4fa0
↪", background="#e3edff"))
```

- In a normal project the function ignores its argument: it returns an empty text, and nothing of it is written to `info.json` (which students can read).
- In a project in **instructor mode** (`sre start --instructor-mode <lab>`, `sre restore --instructor-mode <file>`, or `sre set-instructor-mode <running_lab>` on a running project — privileged only) the text is kept, and the GUI displays it in red on a light red background while its *Instructor mode* button is on. `color` / `background` (`'#rrggbb'` or a colour name) change the colours of one text; `params.instructor_text_color` / `params.instructor_background_color` are the defaults.
- A **debug project** (`sre start --debug-project`) keeps the text as well: the same button displays or hides it, so the hints can be checked while the lab is being written.
- A question hash does not depend on the instructor texts: the answers stay attached when the mode is switched on a running project.

Rules:

- call `instructor()` from `NetScheme.__init__` (after `super().__init__()`) or from `Grade.grade()`. The mode of the project is not known while the module is imported, so a module-level call (`title`, `@sre_state(description=...)`, `Flavor.flavor_form`) always gives an empty text;
- use it for `self.informations` and for the titles and descriptions of the questions. The other texts (grade elements, state descriptions, files pushed to the machines) are plain text and would show the raw marks in instructor mode;
- a fragment is either inline text or one or several whole markdown blocks (paragraphs, lists, code blocks, tables). It is dedented on its own, so an indented triple-quoted block works whatever the indentation of the public text; it cannot be nested inside a list item of the public text;
- no `@@{field:regex}@@` form field inside a fragment (a student would not get the field);
- use `+` or `TranslatedText.format()` to combine texts, as with `tr()`: `%` and f-strings do not work on translated texts.

A lab file without the instructor texts — `sbin/strip-instructor`

The running system keeps the instructor texts away from students, but the lab file itself still holds them. `sbin/strip-instructor` writes the file without its `instructor()` calls, for instance before handing a lab over:

```
sbin/strip-instructor lab/s4/mylab.py /tmp/mylab_public.py   # output file
sbin/strip-instructor lab/s4/mylab.py /srv/public_labs/      # existing directory → /
↪srv/public_labs/mylab.py
sbin/strip-instructor -i /srv/public_labs/mylab.py           # replace the file itself
```

- an `instructor(...)` operand of a `+` disappears with its operator (`instructor(tr("note")) + tr("public")` → `tr("public")`); a call standing alone becomes `''`;
- the `_TRANSLATIONS` entries of the `tr()` strings that only those calls used are deleted, in every language;
- `instructor` is removed from its `from ... import` when nothing uses it any more;
- formatting and comments are kept; the result is checked (it must parse and hold no `instructor()` call), otherwise nothing is written.

The stripped lab behaves like the original outside the instructor mode. Limits:

- a text kept in a variable is not followed: with `note = tr("...")` then `instructor(note)`, the definition of `note` stays in the file. The script warns about such variables; write `note = instructor(tr("...."))` to have the text removed;
- comments are not examined, and only the lab file is processed (not the state directories of a directory lab);
- a question keeps its hash as long as the removed text has no language that the text it was added to lacks (the usual case). Otherwise the archives of the stripped lab do not match the original lab in `sre re-eval`.

In instructor mode every state can be applied from the GUI, as in a debug project: the **Apply Configuration** tab also lists the states that are not `user_allowed` (in red), so the instructor can apply a `final` state in front of the class without making it available to students. The project also gets a **Log** tab listing what every applied state executed (`self.cmd()`, `self.file()`…, with outputs and exit codes), again only while the button is on.

3.6.6 Numbered sections, references and table of contents — `info_subtitle()`, `info_toc()`

The headers of a course, its table of contents and the mentions of a section (`see section 7`) are computed, so that inserting a section renumbers nothing by hand. Four functions imported from `SRE.lib_sre`, like `tr()` and `instructor()`, emit plain-text *markers*; the numbers, the links and the table of contents are resolved when `info.json` is written, in every language, so there is nothing else to call:

```
from SRE.lib_sre import NetScheme0, Grade0, make_tr, no_tr, info_subtitle, info_tag,␣
↪info_ref, info_toc

tr = make_tr('fr')

class NetScheme(NetScheme0):
    def __init__(self, data, running_lab_name):
        super().__init__(data=data, running_lab_name=running_lab_name)
        self.informations = (
            no_tr("## ") + title + no_tr("\n")
```

(continues on next page)

(continued from previous page)

```
            + tr("**Sommaire**\n", en="**Contents**\n") + info_toc()
            + info_subtitle(tr("À quoi sert DHCP", en="What DHCP is for"), tag="why")
            + tr("""
Un serveur prête une adresse pour la durée d'un **bail** (section @@ref:lease@@).

### @@sec@@ Les acteurs

Trois rôles : @@tag:roles@@ client, serveur, relais.
""", en="""
A server lends an address for the duration of a **lease** (section @@ref:lease@@).

### @@sec@@ The actors

Three roles: @@tag:roles@@ client, server, relay.
""")
            + info_subtitle(tr("La vie d'un bail", en="The life of a lease"), tag="lease
↪")
            + tr("""...""")
        )

class Grade(Grade0):
    def grade(self):
        super().grade()
        self.question_text(
            title=tr("Le bail", en="The lease"),
            description=tr("Lisez la section @@ref:lease@@ des Informations (les rôles :␣
↪@@ref:roles:title@@).",
                           en="Read section @@ref:lease@@ of the Informations tab (the␣
↪roles: @@ref:roles:title@@)."))
```

The student sees a **Sommaire** listing *1. À quoi sert DHCP*, *1.1. Les acteurs* and *2. La vie d'un bail* as links, the headers `## 1. À quoi sert DHCP`, `### 1.1. Les acteurs`, `## 2. La vie d'un bail`, the sentence *(section 2)* where the number is a link that scrolls the tab to the section, and in the question *Lisez la section 2 des Informations (les rôles : 1. À quoi sert DHCP).*

function	marker it returns	resolved form in the course
`info_subtitle(titl level=1, tag="")`	`\n\n## @@sec@@ Title\n\n` (`###` for `level=2`…), `## @@sec:tag@@ Title` with a tag	`## <a id="tag"></a>2. Title` (an untagged section gets the id `sec.2`)
`info_tag(tag)`	`@@tag:tag@@`	`<a id="tag"></a>`; a reference to it gives the number of the section it stands in and scrolls to the spot
`info_ref(tag)` / `info_ref(tag, title=True)`	`@@ref:tag@@` / `@@ref:tag:title@@`	`[2.1](#tag)` / `[2.1. Title](#tag)`; in a question text: `2.1` / `2.1. Title` (its browser has no anchor)
`info_toc(depth=Non`	`\n\n@@toc@@\n\n`, `@@toc:1@@` with a depth	a nested bullet list of `[2.1. Title](#tag)` links, every level or the first *depth* levels

The functions return plain strings (a `tr()` title keeps its languages) that combine with `+` like any text. The markers may just as well be typed inside a `tr()` block — the natural form for a sub-section or a reference in the middle of a long text, and the only one inside a block holding literal braces, which `TranslatedText.`

`format()` would refuse: `### @@sec@@ Sub-title` is numbered by its number of `#` (`##`: level 1, `###`: level 2…), `(section @@ref:lease@@)` becomes a link.

Rules:

- a header without a marker is not numbered, and a `#` header (the lab title) never is;
- a tag is made of letters, digits, `_` and `-`, in English like the identifiers of the lab, and is unique; a level cannot be skipped (no `####` right after a `##`);
- a question text may hold `@@ref@@` markers only; they give bare numbers;
- the headers read `1.`, `2.1.`, `2.1.3.`, a reference drops the final dot; a marker inside a fenced code block is left as it is;
- do not number a header inside `instructor()`: the numbers must be the same in and out of the instructor mode, so a `@@sec@@`, `@@tag@@` or `@@toc@@` standing inside an instructor fragment is dropped (a `@@ref@@` there is resolved);
- the languages of a `tr()` course must declare the same sections and tags in the same order;
- a question hash is computed from the marked title, so renumbering the course never detaches the stored answers.

`sre check` resolves the course as `sre start` does, prints its outline (number, title, tag of every section) and reports a bad marker, a level jump, a tag defined twice or unknown — in the course or in a question — and languages whose sections differ, as a `FAIL`. The `sre export` PDF keeps the numbers and the table of contents without the links.

3.7 State methods

A *state* is a method of `NetScheme` decorated with `@sre_state`. The body of the method does not execute commands directly — instead, it **registers** operations (run a shell command, write a file, copy a file from the host, …) which SRE then applies in order against the running containers.

The `initial` state is always applied at `sre start` and is where the lab configures interfaces, launches services, and writes per-machine files derived from `Data`. Additional states are applied on demand with `sre state <running_lab> <state_name>`. Their main use case is the instructor's workflow: a state that brings the project into a fully-configured, all-questions-answered shape, so the grading code can be exercised end-to-end without going through the student workflow. Students see and can apply a state only if the module-level attribute `allow_user_states = True` **and** that state was decorated with `@sre_state(user_allowed=True)`.

```
@sre_state(user_allowed=False)
def initial(self):
    """Always applied at sre start."""
    d = self.data
    self.cmd('router', f'ip addr add {d.ips.router} dev eth0')
    self.cmd('router', 'sysctl -w net.ipv4.ip_forward=1')
    self.file('client', '/etc/resolv.conf', f'nameserver {d.ips.router.ip}\n')

@sre_state(user_allowed=True, description=tr("Final config", fr="Configuration finale"))
def final(self):
    """Optional additional state; students can apply it via 'sre state'."""
    self.cmd('router', 'iptables -A FORWARD -j ACCEPT')
```

3.7.1 Lifecycle states: `save` / `restore`

`NetScheme0` defines three *lifecycle* states that SRE applies itself: `initial` (at `sre start`), `save` (by `sre save`, just before the container filesystems are captured) and `restore` (by `sre restore`, on the new instance right after the containers have been redeployed from the saved filesystems). `save` and `restore` are empty by default and only matter for labs that set `allow_save_restore = True`.

A save captures **files only**: a restored container starts from its saved filesystem but nothing that `initial()` launched with `cmd()` is running any more. Relaunch those daemons in `restore()`:

```
@sre_state()
def restore(self):
    self.cmd('server', 'service ssh start')
    self.cmd('server', 'service isc-dhcp-server start', allow_error=True)
```

Lifecycle states are never user-allowed, never listed in the GUI, and `sre check` runs `save()` / `restore()` when the lab allows save/restore. `<srelab_dir>/save/` and `<srelab_dir>/restore/` are therefore reserved state-file directories (see *State file transfer*).

3.7.2 `@sre_state` decorator

Parameter	Description
`user_al`	If `False`, students cannot apply this state themselves (only root/instructor can).
`descrip`	Human-readable label shown in the GUI (supports `TranslatedText` from `make_tr`).
`multi_p`	If `True`, the state method is re-run before each step so that `self.cmd()` / `self.host_cmd()` return real results (see *Reading command results* below). Default `False`: the method is called once, as before.

3.7.3 Per-machine operations

These all register an op against a single machine and accept a `step=` parameter (see *Multi-step state setup* below).

Method	Description
`self.cmd(machine, command, step=1, timeout=120, default_value='', default_code=0, allow_error=False)`	Run a shell command inside `machine` (under `sh`, via `exetests.py`). Returns `(output, exit_code)` — a placeholder until the command has run, see *Reading command results*. Non-zero exit codes are reported on stderr unless `allow_error=True`.
`self.file(machine, path, content, permissions=0o644, owner="root:root", mtime=None, step=1)`	Create or overwrite a file inside `machine`. `content` may be `str` or `bytes`.
`self.append_to_file(machine, path, content, permissions=None, owner=None, mtime=None, step=1)`	Append to a file (creates it if missing).
`self.idempotent_append_to_file(mac path, content, ..., step=1)`	Same as `append_to_file` but only appends if the file does not already end with `content` — safe to call repeatedly.

3.7.4 File transfer between host and container

Method	Description
`self.cp_from_host(src, machine, dest, owner="root:root", permissions=None, mtime=None, step=1)`	Copy a file from the host into `machine`. Relative `src` is resolved against `params.files_dir(running_lab_name)`.
`self.cp_to_host(machine, path, dest, step=1)`	Copy a file from `machine` back to the host, inside `params.files_dir(running_lab_name)`. `dest` is restricted to that directory.

3.7.5 Host-side operations

Method	Description
`self.host_cmd(command, step=1, timeout=120, default_value='', default_code=0, allow_error=False)`	Run a shell command on the **host** (not inside any container). Returns `(stdout, exit_code)` with the same contract as `cmd()`. Refused if `params.execute_commands_on_host is False`.
`self.host_callback(callable, step=1)`	Invoke a Python callable on the host at this step, with no arguments. Useful when the next steps need values computed in Python.

3.7.6 Switch operations

Method	Description
`self.switch_cmd(networ command, step=1, default_value='', default_code=0, allow_error=False`	Run one command on the management console of the managed switch `network` (a network with `'mode': 'managed'`; `ValueError` otherwise). A word `@machine` in `command` is replaced by the number of the port the machine is plugged into, e.g. `port/setvlan @pc1 10`. Returns `(output, code)` with the same contract as `host_cmd()`: `code` is `0`, the error number given by the switch (`17`: the VLAN already exists) or `-2` when the command could not be run. See *Switches and VLANs*.

3.7.7 Multi-step state setup — the `step` parameter

Every state-method operation accepts `step=N` (default `1`). When SRE applies a state it groups all registered ops by step and runs them in ascending order: every step-1 op finishes before any step-2 op starts. This matters when later ops depend on earlier ones being in place — for example writing a config file at step 1 and only restarting the service at step 2:

```
@sre_state(user_allowed=False)
def initial(self):
    self.file('dns', '/etc/unbound/unbound.conf', conf, step=1)
    self.cmd('dns',  'systemctl restart unbound',    step=2)
```

Inside a single step the per-machine op order is preserved, and ops on different machines run in parallel. `host_cmd` / `host_callback` / `switch_cmd` for step `N` run, in registration order, *before* the container ops of step `N` (so a key generated on the host at step 1 can be copied into a machine at step 1).

Container commands registered with `self.cmd()` run under `sh` through `/usr/local/sbin/exetests.py`, the runner `Grade.test()` also uses: consecutive `cmd()` ops of one machine at one step are sent as a single batch, shell syntax such as `|`, `>>` or `&&` works, and each command is killed after `timeout` seconds (default `params.default_state_cmd_timeout`, 120 s; `0` disables the timeout; the exit code is then `-1`). A non-zero exit code is reported on stderr (hidden in `--user` mode) unless the call passes `allow_error=True`.

3.7.8 Reading command results — `multi_pass=True`

`self.cmd()`, `self.host_cmd()` and `self.switch_cmd()` return `(output, exit_code)` with the same register-then-resolve contract as `Grade.test()`: the value is the placeholder `(default_value, default_code)` until the command has actually run. By default a state method is called **once**, before anything runs, so it only ever sees placeholders.

Decorate the state with `@sre_state(multi_pass=True)` to have SRE call the method again before each step: the call that registers the ops of step `N+1` sees the real results of steps `1..N`. The method runs `max_step + 1` times, like `grade()` during an evaluation: a final call after the last step sees the results of every command and is the natural place to copy them into `self.data`. Each call only contributes the ops of the step about to be applied; re-registering the ops of an earlier step is expected and harmless.

```
@sre_state(multi_pass=True)
def initial(self):
    hostname, code = self.cmd('client', 'cat /etc/hostname', step=1)   # ('', 0) on the␣
↪first call
    self.file('server', '/etc/motd', f'welcome {hostname.strip()}\n', step=2)
    self.data.client_hostname = hostname.strip()   # real on the final call; data.json␣
↪is saved after initial()
```

Rules for a `multi_pass` state:

- **Consume a result at a later step than the command.** An op that uses a result (`file()`, another `cmd()`) must be registered by a call that runs after the command, i.e. at step `N+1` or later; a plain `self.data` assignment can simply rely on the final call. An op registered during the final call for a new, higher step extends the loop — another pass follows, exactly as a late `self.test(step=N)` extends `grade()`.
- **The method must be idempotent.** It runs several times: `print()` shows up once per pass, and anything random or filled in later must give the same answer on every pass. Wrap such values in `self.once(key, factory)`, which calls `factory()` the first time `key` is seen and returns the cached value afterwards (`pcap_gen.generate_pcap_tcp_example` does this for its ports and result dict).
- `self.step` is the number of steps already applied when the method runs (0 on the first call) and `self.max_step` the highest step registered so far.

`sre check` and `sre export` always call the method once and only see placeholders.

3.7.9 Network config helpers (from `/opt/sre/lib/net_config.py`)

```
from net_config import set_net_config_entry, set_sysctl, NetConfigEntry, SysctlConfig

# In initial():
set_net_config_entry(net_scheme=self, machine_name='router', nc_entry=[
    ([data.ips.router_lan], [(IPv4Network('0.0.0.0/0'), data.ips.gw)]),
])
set_sysctl(net_scheme=self, machine_name='router', sysctl_config={'ipv4.ip_forward': 1})
```

`NetConfig` is a list of interface entries; each entry is `([addresses], [(dest_network, gateway), ...])` or `'dhcp'` or `None`. An entry may hold both families (`[d.ips.pc, d.ips6.pc]`, routes to `IPv4Network('0.0.0.0/0')` and `IPv6Network('::/0')`): `set_net_config_entry` dispatches on the address family, and `get_net_config_from_topology(..., ipv6=True)` builds such entries — see *IPv6 labs*.

`set_ip_forward(net_scheme, machine_name, ip_forward, step=1)` and `set_ipv6_forward(net_scheme, machine_name, ipv6_forward, step=1)` set `net.ipv4.ip_forward` / `net.ipv6.conf.all.forwarding` (Kathara starts every machine with forwarding on in both families).

Persistent equivalents — these write to `/etc/network/interfaces` (one `inet static` and one `inet6 static` stanza per interface as needed) and `/etc/sysctl.d/99-sre.conf` so the config survives a container restart:

- `set_persistent_net_config_entry(net_scheme, machine_name, nc_entry)`
- `set_persistent_sysctl(net_scheme, machine_name, sysctl_config)`

3.7.10 State helpers (from `/opt/sre/lib/state_helpers.py`)

Convenience functions called inside `@sre_state` methods:

```
from state_helpers import (set_unbound_server, set_basic_unbound_server,
                           change_password, create_user,
                           hosts_file_content, create_hosts_file,
                           setup_simple_tcp_server, set_radvd, set_slaac_client)
```

Function	Description
`set_unbound_server(net_scheme machine)`	Drop a permissive Unbound DNS config into `/etc/unbound/unbound.conf` and start the service. (Thin alias for `set_basic_unbound_server`.)
`set_basic_unbound_server(net_s machine)`	Same as above; explicit name when you want to make the "basic / permissive" intent obvious.
`change_password(net_scheme, machine, username, password)`	Set a user's password via `chpasswd`. The password is written to a temporary file (never passed on the command line).
`create_user(net_scheme, machine, username, password, uid=None, gid=None)`	Create a user with `useradd` (if not already present) and set its password. `uid`/`gid` are passed as integers. The password is written to a temporary file; the username is passed via an environment variable to prevent shell injection.
`hosts_file_content(net_scheme domain_extension, included=None, ips=None, separator="\t\t", ipv6=False, ips6=None)`	Return `/etc/hosts` lines for a set of machines. Each machine gets one line per network it is connected to (two lines if multi-homed, with `machine_net` naming). Addresses are read from `net_scheme.data.ips.*` by convention (`machine` for single-homed, `machine_net` for multi-homed), or from the `ips` dict if provided. `included` defaults to all visible machines. `ipv6=True` adds, after each IPv4 line, the IPv6 line of the machines that have an address in `net_scheme.data.ips6.*` (or in the `ips6` dict).
`create_hosts_file(net_scheme, domain_extension, machine_list=None, included=None, ips=None, separator="\t\t", ipv6=False, ips6=None)`	Write `/etc/hosts` to each machine in `machine_list` (defaults to all visible machines). Each file begins with standard loopback entries (`127.0.0.1 localhost`, `127.0.1.1 <machine>`, plus `::1` / `ff02::1` / `ff02::2` with `ipv6=True`) followed by the lines from `hosts_file_content()`.
`setup_simple_tcp_server(net_s machine, port, answer, ip=None, ipv6=False)`	Start an idempotent daemon that answers `answer` to every TCP connection on `port`, bound to `ip` (IPv4 or IPv6, prefix stripped), to `0.0.0.0`, or to `::` (dual stack) with `ipv6=True`.
`set_radvd(net_scheme, machine, prefixes, step=1, *, rdnss=None, dnssl=None, min_rtr_adv_interval=3, max_rtr_adv_interval=10, adv_autonomous=True, adv_on_link=True, adv_router_addr=False, enable_forwarding=True)`	Write `/etc/radvd.conf` (`prefixes = {interface: [IPv6Network, ...]}`, interfaces as `'eth1'` or `1`, /64 prefixes for SLAAC) and start radvd, after enabling IPv6 forwarding. Returns the configuration text.
`set_slaac_client(net_scheme, machine, interfaces=None, step=1)`	Make the machine configure itself from router advertisements: `accept_ra=2` and `autoconf=1` on `interfaces` (default: every interface of the topology). Needed because Kathara starts the machines with forwarding on, which makes the kernel ignore RAs.

3.8 `Grade` class

`Grade` declares **what** the evaluator checks and **how** results are scored. It does not execute anything itself: every `self.test(...)` call merely registers a command. SRE then drives a multi-step loop (`run_tests()`) that calls `Grade.grade()` repeatedly, executing registered commands between passes and feeding their real results back into the next call.

```python
from SRE.lib_sre import Grade0

class Grade(Grade0):
    def grade(self):
        """Register tests, questions, and grade elements.
        Called multiple times per evaluation - must be side-effect-free."""
        super().grade()

        # Register a test: run a shell command in a container
        result, code = self.test('router', 'ping -c1 -W1 8.8.8.8', timeout=5)

        # Register a question (student's text answer)
        answer = self.question_text(
            title='Default gateway',
            description='What is the default gateway for the client?',
            default_answer='',
        )

        # Register a grade element and set its score
        self.add_grade_element('Connectivity', max_grade=4)
        if code == 0:
            self.set_grade('Connectivity', 4)
        elif '64 bytes' in result:
            self.set_grade('Connectivity', 2)

        self.add_grade_element('Gateway answer', max_grade=2)
        if str(self.data.ips.router.ip) in answer:
            self.set_grade('Gateway answer', 2)
```

3.8.1 The grading lifecycle

`Grade.grade()` is **not** called once. It is called repeatedly by `run_tests()`, with the registered commands actually executed between calls. The loop is:

1. `self.step` starts at 0, `self.max_step` at 1. `load_answers()` reads the student's `answers.json`.
2. `reset_before_grade()` clears questions, grade rubrics, and section counters.
3. `grade()` is called.
   - `self.test(machine, cmd, step=N)` registers `cmd` under `(machine, N)` and returns `(default_value, default_code)` (i.e. placeholders) the *first* time it is seen.
   - If a call uses `step=K` larger than `self.max_step`, `max_step` is bumped to `K`, extending the loop.
   - `self.add_grade_element` / `self.set_grade` / `self.question_*` calls all register entries; they don't read any container state directly.
4. `self.step` is incremented to `N`. All commands registered at step `N` are bundled per machine into one `EXETESTS@@@cmd1@@@cmd2@@@…` env var and run inside each container in parallel (16-worker `ThreadPoolExecutor`, via `/usr/local/sbin/exetests.py`). The `test_switch()` commands for step `N` are then sent to the consoles of the managed switches, and the host-side `test_host()` commands run in parallel on the host.
5. Results are stored back into `self._tests[(machine, N)]`. The loop returns to step 2. On this pass, the registration calls for step `N` find the entry already populated and return the **real** `(stdout, exit_code)`; the code paths gated on those results now execute.

6. When `self.step > self.max_step`, the loop exits. The archive (zstd-compressed msgpack with all tests, answers, errors, and grade list) is written to `params.archives_dir` and to every directory in the module-level `archive_dirs`.

This has two consequences for `grade()`:

- **It must be side-effect-free.** It is run several times — at least twice (one registration pass, one result pass), more if you use multi-step tests. Never call `subprocess`, write to disk, or mutate global state inside it. All work goes through `self.test` / `self.question_*`.
- **`self.test()` returns placeholders the first time it is reached.** Code that branches on the return value (e.g. `if code == 0:`) executes both with the placeholder (no-op) and with the real result. Make sure the placeholder branch is harmless — it will execute, but its `add_grade_element` calls are wiped by `reset_before_grade()` before the next pass, so only the final pass's calls end up in the archive.

A worked two-step example: configure step 1 to perform a setup action, then read its effect at step 2.

```
def grade(self):
    super().grade()
    self.test('router', 'systemctl restart bird', step=1)
    out, code = self.test('router', 'birdc show route', step=2)
    self.add_grade_element('OSPF routes', max_grade=5)
    if 'OSPF' in out:
        self.set_grade('OSPF routes', 5)
```

On the first call, both `self.test()` invocations register; both return placeholders. SRE executes the step-1 command (`systemctl restart bird`). On the second call, the step-1 test returns its real result; the step-2 test registers and returns its placeholder. SRE executes the step-2 command. On the third call, both tests return real results; `out` contains the routing table, the rubric is populated, and the loop exits.

3.8.2 Grade parts

A *grade part* is a named group that bundles related rubric items together. Parts are **purely presentational**: they affect how grades are displayed in the GUI's *Evaluations* tab and in the PDFs produced by `sre outline` — each part is rendered as a labeled block with a **subtotal row** summing its elements. The overall total, the archive contents, and the marks returned by `mark_exo_eval()` / `mark_self_eval()` are unchanged.

Register a part with `self.add_grade_part(title, description='')` and pass the returned object as `grade_part=` to every element that belongs to it:

```
def grade(self):
    super().grade()

    part1 = self.add_grade_part("part1", tr("Client DNS dig"))
    self.add_grade_element(title="dig_host1_a", max_grade=1, grade=int(host1_ok),
                           grade_part=part1, description=tr("dig - A de host1"))
    self.add_grade_element(title="dig_host2_a", max_grade=1, grade=int(host2_ok),
                           grade_part=part1, description=tr("dig - A de host2"))

    part2 = self.add_grade_part("part2", tr("Serveur DNS cache Unbound"))
    self.add_grade_element(title="unbound_running", max_grade=2, grade=int(unbound_ok),
                           grade_part=part2, description=tr("unbound actif"))
    # ...
```

Elements registered without `grade_part=` are shown ungrouped (above or between the parts, in registration order). Parts are rendered in the order they were registered with `add_grade_part()`. See `lab/sre/dns1.py` for a real-world example with three parts.

3.8.3 Self-eval vs instructor-eval scope

Every `add_grade_element` call takes a `scope=` bitmask that decides which audiences see the element. Three values are defined in `params`:

Value	Visible in
`SELF_EVAL_SCOPE` (1)	Student self-eval only — `sre eval --auto-eval` (GUI **Start evaluation** button)
`EXO_EVAL_SCOPE` (2)	Instructor view only — `sre eval` (non-auto), `sre outline`, `sre sheet`, periodic exam-mode evals
`BOTH_EVAL_SCOPE` (3, default)	Both audiences

The total mark reflects only the elements in the active scope: `mark_self_eval()` sums elements with `scope & SELF_EVAL_SCOPE`, `mark_exo_eval()` sums elements with `scope & EXO_EVAL_SCOPE`. The same `grade()` body can therefore produce a different total for a student self-eval than for the instructor's archive.

A common pattern is to register a **detailed rubric** for the instructor and a **coarse summary** for the student — enough feedback to know something is broken in a given part, without disclosing which individual check failed:

```
from SRE import params

def grade(self):
    super().grade()

    part1 = self.add_grade_part("part1", tr("DNS resolution"))
    host1_ok = ...   # bool computed from self.test(...) results
    host2_ok = ...
    ptr_ok   = ...

    # Detailed rubric - only the instructor (and sre outline / sre sheet) sees these
    self.add_grade_element('dig host1 A',   max_grade=1, grade=int(host1_ok),
                           grade_part=part1, scope=params.EXO_EVAL_SCOPE)
    self.add_grade_element('dig host2 A',   max_grade=1, grade=int(host2_ok),
                           grade_part=part1, scope=params.EXO_EVAL_SCOPE)
    self.add_grade_element('dig host1 PTR', max_grade=1, grade=int(ptr_ok),
                           grade_part=part1, scope=params.EXO_EVAL_SCOPE)

    # Summary - what the student sees during self-eval
    all_ok = host1_ok and host2_ok and ptr_ok
    self.add_grade_element('part1_global', max_grade=1, grade=int(all_ok),
                           scope=params.SELF_EVAL_SCOPE,
                           description='Part 1 (combined)')
```

During the student's self-eval, only the `SELF_EVAL_SCOPE` element appears: one row per part with a pass/fail score (no need for grade parts there). During the instructor's evaluation, only the three `EXO_EVAL_SCOPE` rows appear. `scope` and `grade_part` compose freely — elements in the same part can have different scopes and will be subtotalled accordingly for each audience.

3.8.4 Cheat answers

`cheat_answers` on `question_text` / `question_form` maps a state name (e.g. `'final'`) to an answer value. When the student has applied that state, the cheat answer is used instead of the student's actual input. Useful when the instructor's `final` state should produce a fully-passing evaluation.

3.8.5 Letter grades vs numeric marks

By default, marks are numeric, scaled to `params.default_maximum_mark`, and rounded to one decimal. Set `self._use_numerical_marks = False` (per instance, inside `grade()` or earlier) to switch to letter grades on the **total**: `A+` 18/20, `A` 16, `B` 14, `C` 12, `D` 10, else `F`.

Per-element letter conversion (`OK` / `MEH` / `FAIL`) is also available via `GradeElement.to_grade_letter()` — full marks → OK, partial → MEH, zero → FAIL.

3.8.6 Bonus elements

`self.add_grade_element(..., bonus=True)` registers a *bonus* element: its grade counts in the totals, but its `max_grade` does not (neither in the grand total nor in the subtotal of its `GradePart`), so the mark is computed against the non-bonus maximum only. As soon as a scope holds a bonus element, its mark is capped at `_maximum_mark`: bonus points make up for lost points but never push the mark above the maximum (a lab without bonus elements keeps its usual, uncapped mark). The reports still show the `max_grade` of a bonus element, with the word "Bonus": `2 (Bonus)` in the Max column of the GUI Evaluations tab, of the `sre outline` PDFs, of the `sre sheet` Questions sheet and of the `sre watch` screens; the `--bonus-in-prefix` flag of `sre outline` / `sre sheet` / `sre watch` writes `Bonus: <label>` before the label instead and leaves the Max column numeric.

```
part = self.add_grade_part('ebgp', tr("Part 2: eBGP sessions"))
self.add_grade_element('ebgp_md5', max_grade=1, grade=int(md5_ok), grade_part=part,␣
↪bonus=True,
                       description=tr("bonus: the r3-ispb session is MD5-authenticated␣
↪on both sides"))
```

3.8.7 Overriding `mark_exo_eval()` to adjust the overall mark

The overall mark stored in the archive (and surfaced by `sre outline`, `sre sheet`, and `sre cat`) is produced by `Grade.mark_exo_eval()`. The default implementation is:

```
def mark_exo_eval(self):
    return self._compute_mark(self._total_grade_exo_eval, self._total_max_exo_eval)
```

i.e. `ceil(10 · maximum_mark · total_grade / total_max) / 10` in numerical mode, or the A+/A/B/C/D/F letter in letter mode. `_total_grade_exo_eval` / `_total_max_exo_eval` are the sums over every `add_grade_element` that contributes to the instructor's view (i.e. excluding self-eval-only rubrics). `mark_self_eval()` is the counterpart used during student self-evaluations.

Override `mark_exo_eval()` (and/or `mark_self_eval()`) on the `Grade` subclass when you want a non-default scale — typically because the maximum of the rubric does not match the intended denominator. Common reasons:

- **Fixed denominator.** The exam was designed against an absolute target (e.g. 39 points). The rubric may sum to more, but you want the mark expressed against that fixed value. Replace `_total_max_exo_eval` with the constant, and cap at `_maximum_mark` so bonuses don't push above the cap:

```
import math

class Grade(Grade0):
    def mark_exo_eval(self):
        if not self._grade_list or self._total_max_exo_eval == 0:
            return None
        return min(self._maximum_mark,
                   math.ceil(10 * self._maximum_mark * self._total_grade_exo_eval /␣
↪39) / 10)
```

- **Bonus / penalty handling.** Bonus questions are better registered with `bonus=True` (see *Bonus elements*): no override needed. Penalty rubrics registered with `max_grade=0` (so they only subtract) don't enlarge the denominator either — but if you want them to *not* subtract below zero, clamp `total_grade` before scaling.

`mark_exo_eval()` is called once at the end of `run_tests()`, after every pass of `grade()` and after `compute_total()`. By that time `self._grade_list`, `self._total_grade_exo_eval` / `_total_max_exo_eval`, `self._maximum_mark`, and `self._use_numerical_marks` are all final, so the override is free to read them. Return `None` to signal "no mark" (shown as blank), a `float` for a numeric mark, or a string for a letter grade.

3.8.8 Grade methods

Method	Description
`self.test(machine, command, step=1, timeout=20, allow_error=False, default_value='', default_code=0)`	Register a command in a container; returns `(stdout, exit_code)` (placeholder on first pass, real result on subsequent passes).
`self.test_host(command, step=1, timeout=20, allow_error=False, default_value='', default_code=0)`	Same as `self.test` but the command runs on the host. Refused if `params.execute_commands_on_host is False`.
`self.test_switch(network, command, step=1, default_value='', default_code=0, allow_error=False)`	Same as `self.test` for one command of the management console of the managed switch `network` (`vlan/print`, `port/print @pc1`: `@machine` is the port number of the machine). The exit code is `0`, the error number given by the switch or `-2`. The result is archived with the tests of the machines, under the name of the network. See *Switches and VLANs*.
`self.question_text(title, section='', description='', hash=None, order=None, default_answer='', cheat_answers=None)`	Register a free-text question; returns the student's answer (or `default_answer`).
`self.question_form(title, section='', description='', hash=None, order=None, cheat_answers=None)`	Register a form question with inline `@@{field:regex}@@` (single-line text), `@@{field:+N}@@` (multi-line free text of N lines, `params.default_form_field_lines` without N), '@@{field:>opt1
`self.question_dummy(title, section='', description='', hash=None, order=None)`	Display-only block (no input shown to the student).
`self.add_grade_part(title, description='')`	Register a named group of grade elements and return the resulting `GradePart`. Pass it to `add_grade_element(..., grade_part=...)` to associate elements with this group; parts render in registration order with a subtotal row per part in the GUI Evaluations view and in `sre outline` PDFs.
`self.add_grade_element(tit max_grade, description='', grade=0, scope=params.BOTH_EVAL_SCOPE, grade_part=None, bonus=False)`	Add a graded rubric item. `scope` is a bitmask: `SELF_EVAL_SCOPE` (1) restricts the element to student self-eval, `EXO_EVAL_SCOPE` (2) restricts it to instructor/auto eval / `sre outline` / `sre sheet`, `BOTH_EVAL_SCOPE` (3, default) shows it everywhere. `grade_part` is a `GradePart` returned by `add_grade_part()`. `bonus=True` makes a bonus element: its grade counts, its `max_grade` is shown (with "Bonus") but not counted, and the mark is capped at the maximum (see *Bonus elements*).
`self.set_grade(title, grade)`	Set the score of a previously registered element.
`self.`	Increment the section counter at `level` and return its formatted label (e.g.

3.9 Grading Library Reference

3.9.1 DHCP helpers (from /opt/sre/lib/dhcp.py)

```
from dhcp import DhcpParameters, DhcpSubnet, set_dhcp_server, get_dhcp_server,␣
↪check_running_dhcp_server
```

Data classes:

Class	Purpose
DhcpSubne	One subnet block: subnet, range_start, range_end, optional routers, dns_servers, domain_name, default_lease_time, max_lease_time, fixed_addresses
DhcpParam	Full server config: interfaces_v4, interfaces_v6, subnets, authoritative, default_lease_time, max_lease_time, ddns_update_style

Functions used in NetScheme (state setup):

set_dhcp_server(net_scheme, machine, dhcp_params, step=1) — writes /etc/default/isc-dhcp-server and /etc/dhcp/dhcpd.conf from a DhcpParameters instance, then enables and restarts isc-dhcp-server.

Functions used in Grade (evaluation):

get_dhcp_server(grade, machine, step=1) → (DhcpParameters | None, int) — reads and parses the DHCP server configuration from the running container. Returns the parsed parameters and the number of parse errors. Returns (None, 1) if /etc/default/isc-dhcp-server is absent.

check_running_dhcp_server(grade, machine) → (bool, list[str]) — checks whether isc-dhcp-server is currently active. Returns (running, interfaces) where interfaces is the list of interface names dhcpd is bound to (e.g. ["eth0"]), or ["*"] if it listens on all interfaces. Interfaces are read from the live process command line, not from the config file.

3.9.2 Mail helpers (from /opt/sre/lib/smtp.py)

```
from smtp import (install_smtp_probe, smtp_probe, smtp_query, imap_query, get_postconf,␣
↪get_master_services,
                  get_mailq, deferred_to, get_maildir_messages, find_messages,␣
↪received_hops, parse_dsn,
                  parse_received_spf, parse_authentication_results,␣
↪parse_dkim_signature, get_doveconf,
                  render_unbound_records, spf_record_ok, dkim_txt_record,␣
↪generate_dkim_keypair)
```

Helpers of the SMTP labs (Postfix, Dovecot, OpenDKIM, SPF). The evaluation is behavioural: a hidden machine runs an **active probe** (lib/smtp_probe.py, standard library only) that dialogues with the student's servers, and what must arrive is read in the Maildirs.

Functions used in NetScheme (state setup):

install_smtp_probe(net_scheme, machine, step=1) — copies the probe script into *machine*.

render_unbound_records([(name, type, value), ...]) — local-data: lines for an unbound configuration (a TXT value is quoted and split into 255-character strings, (address, 'PTR', name) becomes local-data-ptr:); dkim_txt_record(public_key_b64) and generate_dkim_keypair(bits) (the private key as opendkim-genkey writes it) let Data.generate() carry the solution key.

Functions used in `Grade` (evaluation):

`smtp_probe(grade, machine, spec, step=1, timeout=45) → {query_id: ProbeResult}` — runs the probe with `spec = {'timeout': 5, 'queries': [...]}` built from `smtp_query(id, host, port=25, helo=, mail_from=, rcpt_to=[...], starttls=False, auth=(user, password), token=, subject=, body=)` and `imap_query(id, host, user, password, token=, poll=)`. A `ProbeResult` records every reply as `[code, text]` (`banner`, `ehlo`, `starttls`, `auth`, `mail`, `data`, `rcpt[address]`), the EHLO extensions before (`extensions`) and after STARTTLS (`extensions_tls`), the `queue_id`, and for IMAP `login`, `count` and `messages`; use `code(reply)`, `accepted(address)`, `sent`, `logged_in`. The spec must depend on lab data only (same command on every grade pass); the messages sent carry `X-SRE-Probe: token` and `X-SRE-Probe-Query: id`.

`get_postconf(grade, machine, params=POSTCONF_PARAMS, step=1) → {name: value}` (`postconf -x`), `mynetworks_covers(value, network)`, `relayhost_target(value) → (host, port)`, `get_master_services(grade, machine) → {'submission/inet': {..., 'options': {...}}}` (`postconf -M`), `get_mailq(grade, machine) → [MailqEntry]` (`postqueue -j`) and `deferred_to(entries, domain)`.

`get_maildir_messages(grade, machine, paths, step=, token=, expected=, max_wait=) → [MaildirMessage]` — every file of `new/` and `cur/` of the Maildirs *paths* in one base64 dump, after waiting at most *max_wait* seconds for *expected* files containing *token* (the deliveries of what the probe has just sent). Then `find_messages(messages, token, query=)`, `body_contains(msg, sentence)`, `received_hops(msg)` (most recent first: `from`, `from_ip`, `by`, `with`, `id`, `for`), `is_bounce(msg)` / `parse_dsn(msg)`, `parse_received_spf`, `parse_authentication_results`, `parse_dkim_signature`; `maildir_cleanup_command(paths, token)` removes the probe's messages at a later step, `queue_cleanup_command(sender)` its queued ones.

`get_doveconf(grade, machine)` (`doveconf -n` as nested dicts), `doveconf_listener(conf, path)`, `doveconf_value(conf, key)`; `parse_txt_strings(dig_output)`, `parse_spf`, `spf_record_ok(record, mx_ip=, mx_name=)`, `parse_dkim_txt`; `tls.get_tls_server_certificate(..., starttls='smtp')` reads the certificate of a STARTTLS server.

Facts verified on the mail image: OpenDKIM resolves with its own libunbound starting from the root, so a closed lab needs `Nameservers <dns>` in `opendkim.conf`; policyd-spf's `TestOnly = 1` still rejects on a `-all` fail (`Mail_From_reject = False` and `HELO_reject = False` give a header-only check); `nobody` is a valid local recipient of Postfix.

3.9.3 TLS helpers (from `/opt/sre/lib/tls.py`)

```
from tls import eval_rsa_private_key, set_rsa_private_key, eval_self_signed_certificate,␣
↪eval_certificate
```

Functions used in `NetScheme` (state setup):

`set_rsa_private_key(net_scheme, machine_name, key_file, password, bits=4096, cipher='AES-256-CBC')` — generates an encrypted RSA private key inside the container using `openssl genrsa`. File paths and the password are shell-quoted automatically.

Functions used in `Grade` (evaluation):

`eval_rsa_private_key(grade, machine_name, key_file, password=None, bits=4096, cipher='AES-256-CBC', step=1) → bool` — verifies that `key_file` is an RSA private key with the expected size and PEM cipher. Returns `True` if all checks pass.

`eval_self_signed_certificate(grade, machine_name, key_file, cert_file, password, cn=None, bits=4096, cipher='AES-256-CBC', step=1) → dict` — checks that `cert_file` is a valid self-signed certificate whose public key matches `key_file`. Returns a dict with keys `subject`, `issuer`, `common_name`, `not_before`, `not_after`, `serial`, `fingerprint`.

`eval_certificate(grade, machine_name, key_file, cert_file, ca_file, step=1) → dict` — checks that `cert_file` is a valid certificate signed by `ca_file` and whose public key matches `key_file`. Returns the same dict as `eval_self_signed_certificate`.

3.9.4 DNS helpers (from `/opt/sre/lib/dns.py`)

Used by `lab/sre/_DRAFT_misc/dns2.py` (private root, resolver, delegation, TSIG transfers, dynamic updates, DNSSEC, DoT/DoH, views, RPZ). Pure functions first, then `(grade, machine, …)` wrappers; fixtures in `tests/mock_data/dns/`.

- **Queries.** `dig_query(grade, m, server, "www.example.tp A", step=1, dnssec=True, cd=True, norecurse=True, tcp=True, key=(name, secret))` returns a `DigResult`: `status`, `flags` (`has_flag('ad')`), `answer` / `authority` / `additional` lists of `RR(name, ttl, rclass, rtype, rdata)`, `rdata('A')`, `soa_serial()`, `ttl()`, `error` (`'timeout'`, `'transfer failed'`, `'connection refused'`), `xfr_records` for AXFR listings; `referral(result)` gives the NS names and the glue of a referral. `kdig_query(grade, m, server, name, tls=True|https=True, ca_file=, hostname=)` → `KdigResult(session 'TLS'|'HTTPS', http_status, …)`. `nsupdate_run(grade, m, server, zone, ["update add host.example.tp. 300 A 10.0.0.9"], key=)` → `(ok, rcode)`; the same `nsupdate_cmd()` text serves in states (one `printf | nsupdate -y` line: exetests needs single-line commands).
- **DNSSEC.** `ecdsa_p256_keypair()` gives `(private_b64, public_b64)` to keep in `Data`; `render_bind_key_files(owner, priv, pub)` writes them as `dnssec-keygen` would (named adopts them with `dnssec-policy`); `ds_rdata(owner, dnskey_rdata)`, `keytag_of()`, `ds_covers_keys(ds_rdatas, dnskey_rdatas, owner)` check a published DS against the keys a server serves; `trust_anchor_line()` (unbound) and `render_bind_trust_anchors()` (`delv -a`) write the anchor.
- **Configurations.** `get_named_conf(grade, m)` parses `named-checkconf -p` (keys, options, zones, views); `get_unbound_conf(grade, m)` the concatenated `unbound.conf.d/*.conf` (`unbound_values(conf, 'server', 'access-control')`, `unbound_clauses(conf, 'rpz')`); `get_stubby_conf`, `get_firefox_doh` (policies.json or `network.trr.*` prefs), `get_zone_status(…, view=)` / `get_dnssec_status` (`rndc`), `get_named_journal` / `get_unbound_log` (events of `journalctl`: transfers, notifies, updates, RPZ hits), `get_unbound_list(…, 'forwards'|'stubs')`. `unbound_default_local_zone(net)` names the built-in local zone a resolver must declare `nodefault` before it can resolve a private reverse zone.
- **Renderers** for states and instructor texts: `render_zone_file`, `render_tsig_key`, `render_root_hints`, `render_rpz_zone`, `render_stubby_yml`, `reverse_zone_name` / `reverse_label`, `spki_pin`, `random_tsig_secret`; `tls.generate_ca_pem(cn)` for a lab-provided CA.

3.9.5 TCP port helpers (from `/opt/sre/lib/grade_helpers.py`)

```
from grade_helpers import eval_tcp_server, test_dig
```

`eval_tcp_server(grade, machine_name, server_name, step=1) → list[int] | None` — returns the TCP ports in LISTEN state of the processes matching `server_name` (`pgrep -f`, `ss -tlnp`), whatever the local address form (`0.0.0.0:80`, `*:80`, `[::]:80`, `[fd00::1]:8080`, `127.0.0.53%lo:53`), or `None` when no such process runs.

`test_dig(grade, machine_name, server_ip, *, proto='udp', port=53, request, timeout=2, step=1) → (stdout, exit_code)` — runs `dig +short` against `server_ip` (a string, an `IPv4Address` / `IPv6Address` or an `IPv4Interface` / `IPv6Interface`, prefix stripped).

3.9.6 OSPF helpers (from /opt/sre/lib/frr.py)

```
from frr import get_ospf_interfaces
```

`get_ospf_interfaces(grade, machine_name, step=1) → dict` — runs `vtysh -c "show ip ospf interface"` inside the container and parses the output. Returns a dict keyed by interface name, each value containing: `area`, `cost`, `state` (DR/BDR/DROther/…), `dr`, `bdr`, `neighbor_count`, `adj_neighbor_count`.

3.9.7 BGP helpers (from /opt/sre/lib/frr.py)

```
from frr import (get_bgp_summary, get_bgp_neighbor, get_bgp_routes, get_bgp_prefix,
                 get_bgp_advertised_routes, bgp_best_path, bgp_paths, is_bgp_protocol)
```

These helpers parse the `json` output of `vtysh` (FRR 9). Like the OSPF ones, each `get_*` registers one command through `grade.test()` (two-pass contract) and returns its empty shape (`{}`) on the registration pass, when BGP is not running or when the output cannot be decoded. Pass `allow_error=True` while BGP may not be configured yet.

Function	Returns	Description
`get_bgp_summary(grade machine, step=1, allow_error=False)`	`dict`	`show bgp ipv4 unicast summary`: `router_id`, `as`, `peers: {ip: {remote_as, state, established, policy_blocked, pfx_rcd, pfx_snt, description, uptime}}`. `policy_blocked` is `True` for an eBGP peer without inbound/outbound policy (RFC 8212, `(Policy)` in the text output)
`get_bgp_neighbor(gra machine, peer, step=1, allow_error=False)`	`dict`	`show bgp neighbors <peer>`: `remote_as`, `local_as`, `state`, `established`, `link` (`external`/`internal`), negotiated `hold_time`/`keepalive` (seconds), `local_host`/`foreign_host`, `update_source`, `route_reflector_client`, `next_hop_self`, `default_originate`, `route_map_in`/`route_map_out`, `prefix_list_in`/`prefix_list_out`, `accepted_prefixes`, `sent_prefixes`; `{}` for an unknown neighbour
`get_bgp_routes(grade machine, step=1, allow_error=False)`	`dict[ list[`	`show bgp ipv4 unicast`: `{prefix: [path, ...]}`; each path has `valid`, `best`, `reason` (FRR `selectionReason`), `path_from` (`external`/`internal`), `nexthop`, `peer`, `aspath` (`'65100 65000'`), `as_list` (`[65100, 65000]`), `locprf`, `med`, `weight`, `origin`, `local`
`bgp_best_path(routes, prefix)`	`dict`	The best path of `prefix` in a `get_bgp_routes()` result (`{}` when none)
`bgp_paths(routes, prefix, nexthop=None, peer=None, first_as=None, ends_with=None, path_from=None, valid=None)`	`list[`	Paths of `prefix` matching every given criterion (`ends_with=[65001, 65300]` compares the end of `as_list`)
`get_bgp_prefix(grade machine, prefix, step=1, allow_error=False)`	`dict`	`show bgp ipv4 unicast <prefix>`: `{'prefix', 'paths': [...]}` with the fields above plus `accessible` (next-hop reachability), `peer_router_id`, `originator_id`, `cluster_list` (route reflection) and `communities` (`['65001:300']`)
`get_bgp_advertised_r machine, peer, step=1, allow_error=False)`	`dict`	`show bgp ipv4 unicast neighbors <peer> advertised-routes`: `{prefix: {nexthop, aspath, as_list, locprf, med, weight, origin}}`, i.e. what is really sent to that peer after the outbound policy
`is_bgp_protocol(prot`	`bool`	`True` when a kernel route `protocol` (see `get_kernel_routes`) denotes a route installed by bgpd; `kernel_route_gateways(routes, prefix, protocols=BGP_ROUTE_PROTOCOLS)` lists the gateways of a BGP kernel route

`parse_frr_config` / `get_frr_running_config` also return the BGP configuration: `router_bgp` (`as`, `router_id`, `ebgp_requires_policy`, `network_import_check`, `bestpath`, `neighbors: {peer: {remote_as, description, update_source, timers, shutdown, password, next_hop_self, route_reflector_client, default_originate, route_map_in/out, prefix_list_in/out, ...}}`, `networks`, `aggregate_addresses`, `redistribute`), `prefix_lists`, `route_maps` (`{name: [{seq, action, match: [...], set: [...]}]}`), `as_path_lists`, `community_lists`; interface blocks expose their `ip address` lines in `addresses`.

Verified FRR 9 defaults worth knowing when writing a BGP lab: `bgp ebgp-requires-policy` is on (an eBGP session without route-map/prefix-list exchanges no prefix), `bgp network import-check` is on (a `network` statement needs a matching route, e.g. `ip route X/22 blackhole`), and `interface lo` / `ip address A/32` in `frr.conf` configures the loopback address through zebra.

3.9.8 Standard machine wiring (from `/opt/sre/lib/std.py`)

`machine_config(net_scheme, machine_name, config)` — concise helper for wiring a machine inside `NetScheme.build()` (or `initial()`). `config` is a `(interfaces, sysctls)` tuple:

- `interfaces`: list of `(network_name, address, routes)` entries, where `routes` is a list of `(dest_network, gateway)` pairs.
- `sysctls`: dict of kernel parameter names to values.

Compared to calling `set_net_config` and `set_sysctl` directly, `machine_config` resolves network names through the `NetScheme` topology automatically.

3.9.9 Miscellaneous generators (from `/opt/sre/lib/utils.py`)

```
from utils import random_password, random_sentence
```

Function	Description
`random_password(len`	Returns a random alphanumeric password of the given length
`random_sentence(len`	Returns a random sequence of lowercase words, space-separated, approximately `length` characters long

3.9.10 Network inspection helpers (from `/opt/sre/lib/net_config.py`)

These functions are used inside `Grade.grade()` to read live container state.

```
from net_config import (get_ip_addresses, get_routes, get_sysctl_conf,
                        get_net_config_entry, get_persistent_net_config_entry,
                        eval_net_config, net_config_entry_family,
                        get_ip_forward, get_ipv6_forward, get_sys_parameter_bool,
                        get_sys_parameter)
```

Function	Returns	Description
`get_ip_addresses(g machine, step=1, ipv6=False, link_local=False)`	`dict[str, list[tuple[ int]]]`	Run `ip a`; return `{iface: [(addr, prefixlen), ...]}` (sorted by prefix desc, addr asc). `ipv6=True` adds the IPv6 addresses after the IPv4 ones (SLAAC addresses included, temporary ones excluded; link-local ones only with `link_local=True`)
`get_routes(grade, machine, step=1, ipv6=False)`	`dict[tuple[ int], tuple[str, str,int]]`	Run `ip route` (and `ip -6 route` with `ipv6=True`); return `{(net, mask): (via, dev, metric)}` — `default` maps to `('0.0.0.0', 0)` (`('::', 0)` in the IPv6 table)
`get_sysctl_conf(gr machine, step=1)`	`dict[str, str]`	Read `/etc/sysctl.conf` and `/etc/sysctl.d/*.conf`; return `{key: value}`
`get_ip_forward(gra machine, step=1)`	`bool`	Read `/proc/sys/net/ipv4/ip_forward`; return `True` if `1`
`get_ipv6_forward(g machine, step=1)`	`bool`	Read `/proc/sys/net/ipv6/conf/all/forwarding`; return `True` if `1`
`get_sys_parameter( machine, param, step=1)`	`str | None`	Read an arbitrary `/proc/sys/...` file (e.g. `'net.ipv4.ip_forward'`); return string value or `None`
`get_sys_parameter_ machine, param, step=1)`	`bool | None`	Same as `get_sys_parameter` but cast to `bool` (`'1'` → `True`, `'0'` → `False`, else `None`)
`get_net_config_ent machine, step=1, ipv6=False)`	`NetConfigEn`	Run `ip a` + `ip route`; reconstruct a `NetConfigEntry` from live state. `ipv6=True` adds the IPv6 addresses and routes to the same entries (a SLAAC address counts as dynamic: an interface with only SLAAC addresses is `'dhcp'`; the RA default route has a `fe80::` next hop)
`get_persistent_net machine, step=1, ipv6=False)`	`tuple[NetCo int]`	Parse `/etc/network/interfaces`; return `(entry, n_errors)`. `ipv6=True` also parses the `inet6 static` stanzas (`address`, `netmask <prefixlen>`, `gateway`, `post-up ip [-6] route add` / `ip addr add`; 'inet6 auto
`net_config_entry_f version)`	`NetConfigEn`	Keep only the addresses and routes of IP `version` (4 or 6) in each tuple, to grade the two families of a dual-stack configuration apart

`eval_net_config(grade, expected, machine_name=None, current=None, step=1)` — compare a live or provided `NetConfigEntry` against an expected one (`0.0.0.0/0` and `::/0` are both default routes). Returns an attribute-accessible dict with the following keys:

Key	Description
`ips`	Number of IP addresses that match
`ips_expected`	Number of IP addresses in expected
`default_route`	`1` if the default route matches, `0` otherwise
`default_route_expected`	`1` if expected has a default route
`other_routes`	Number of matching non-default static routes
`other_routes_expected`	Number of non-default routes in expected
`wrong_routes`	Non-default routes present in current but not in expected
`dhcp_interfaces`	Number of positions where both expected and current are `'dhcp'`
`dhcp_interfaces_expected`	Count of `'dhcp'` entries in expected
`none_interfaces_expected`	Count of `None` entries in expected

If `current` is `None`, `get_net_config_entry(grade, machine_name, step)` is called automatically.

3.9.11 Switch helpers (from /opt/sre/lib/switch.py)

For the managed switches (see *Switches and VLANs*), on top of `Grade.test_switch()`:

```
from switch import get_switch_ports, get_switch_vlans

ports = get_switch_ports(self, 'lan')
# {'pc1': {'port': 2, 'interface': 'eth0', 'vlan': 10, 'tagged_vlans': []},
#  'r1':  {'port': 1, 'interface': 'eth0', 'vlan': 0,  'tagged_vlans': [10, 20]}}
same_vlan = bool(ports) and ports['pc1']['vlan'] == ports['pc2']['vlan']

vlans = get_switch_vlans(self, 'lan')      # {10: {'untagged': [2], 'tagged': [1]}, ...}␣
↪(port numbers)
```

`vlan` is the VLAN of the untagged frames of the port (0: the default VLAN of the switch), `tagged_vlans` the VLANs of a trunk port. Both functions return `{}` on the registration pass and when the console failed; they take `step=` and `allow_error=` like `test_switch()`. `parse_switch_ports(port_allprint, vlan_allprint)` and `parse_switch_vlans(vlan_allprint)` are the underlying pure parsers.

3.9.12 Ping helper (from /opt/sre/lib/ping.py)

```
from ping import eval_ping
```

`eval_ping(grade, src, dest, step=1, net_config=None, count=1, deadline=1, allow_error=False, ipv6=False)` → `bool` — run `ping -c count -w deadline dest` from `src` machine; return `True` if `"bytes from"` appears in the output. Raise `count` / `deadline` for multi-hop paths; `allow_error=True` keeps an expected failure out of the archive's errors.

`src` and `dest` can each be:

- An `IPv4Address` / `IPv6Address` (or Interface) object, or a valid IPv4 / IPv6 literal — used directly (the literal decides the family); `src` is resolved by reverse-lookup in `net_config`
- `"machine_name"` — machine name looked up in `net_config`; `dest` → first IPv4 address of the first interface, or first IPv6 address with `ipv6=True`
- `"machine_name:N"` or `"machine_name:ethN"` — resolves to interface index N of that machine (same family rule)

If `net_config` is not provided, `grade.net_scheme.net_config` is used. Raises `ValueError` on resolution failure.

3.9.13 SSH helpers (from /opt/sre/lib/ssh.py)

Used in `@sre_state` methods (state setup):

```
from ssh import (add_ssh_monitor_agent, create_ssh_key_on_host,
                 remove_ssh_password_authentication_on_sshd,
                 set_forward_ssh_agent_in_ssh_config, copy_ssh_pub_key_on_machine)
```

Function	Description
`add_ssh_monitor_agent(net_sc machine, step=1)`	Deploy and start an SSH monitor daemon on `machine`. It tails `/var/log/auth.log` and writes one line per forwarded-agent key to `/var/log/.ssh_monitor.log` when a user logs in via SSH. Used with `eval_ssh_connection_with_ssh_agent()`.
`create_ssh_key_on_host(net_s filename, bits=4096, key_type='rsa', password=None, step=1)`	Generate an SSH key pair on the **host** (`ssh-keygen`). Returns the `filename`.
`remove_ssh_password_authenti machine, restart_ssh=False, step=1)`	Set `PasswordAuthentication no` in `/etc/ssh/sshd_config`. Optionally restart sshd.
`set_forward_ssh_agent_in_ssh machine_name, step=1)`	Write `/etc/ssh/ssh_config.d/forward_agent.conf` with `ForwardAgent yes` for all hosts.
`copy_ssh_pub_key_on_machine( machine, pub_key, username, step=1)`	Copy a public key file (host path) into `~username/.ssh/authorized_keys` on `machine`.

Used in `Grade.grade()` (evaluation):

```
from ssh import (check_ssh_key, eval_ssh_public_key_in_authorized_keys,
                 eval_ssh_agent_exists, eval_ssh_agent_with_loaded_key,
                 eval_ssh_possible_with_password_authentification,
                 eval_ssh_connection_with_password, eval_ssh_connection_with_key,
                 eval_ssh_connection_with_ssh_agent, eval_synchronized_file)
```

Function	Returns	Description
`check_ssh_key(grade, machine, private_key, key_type='rsa', bits=4096, password=None, step=1)`	`bool`	Verify that a key pair on `machine` has the expected type, bit length, and passphrase. Public key expected at `private_key + '.pub'`.
`eval_ssh_public_key_in_authorize machine, username, public_key_file=None, public_key=None, step=1)`	`bool`	Check that a public key appears in `~username/.ssh/authorized_keys` with correct ownership and restricted permissions. Provide either `public_key_file` (host path) or `public_key` (content string).
`eval_ssh_agent_exists(grade, machine_name, username, step=1)`	`bool`	Return `True` if an `ssh-agent` process is running as `username` on `machine_name`.
`eval_ssh_agent_with_loaded_key(g machine_name, username, key_on_host, password=None, step=1)`	`bool`	Check that a specific private key (on the host) is loaded in a running agent belonging to `username` on `machine_name`.
`eval_ssh_possible_with_password_ src_machine, dest_machine, username='nobody', step=1)`	`bool`	Probe `dest_machine`'s sshd from `src_machine`; return `True` if password authentication is offered.
`eval_ssh_connection_with_passwor machine_name, username, step=1)`	`bool`	Read `/var/log/auth.log` on `machine_name`; return `True` if a successful password login for `username` is present.
`eval_ssh_connection_with_key(gra machine_name, username, step=1)`	`bool`	Read `/var/log/auth.log`; return `True` if a successful public-key login for `username` is present.
`eval_ssh_connection_with_ssh_age machine_name, username, key_on_host, password=None, step=1)`	`bool`	Read `/var/log/.ssh_monitor.log` (written by `add_ssh_monitor_agent`); return `True` if `username` logged in using an agent carrying `key_on_host`.
`eval_synchronized_file(grade, machine_list, filename, step=1)`	`tuple[ str]`	Return `(True, content)` if `filename` has identical content and mtime on every machine in `machine_list`; `(False, '')` otherwise.

3.9.14 Packet capture helpers (from `/opt/sre/lib/pcap_gen.py`)

Used in `@sre_state` methods (state setup):

```
from pcap_gen import generate_pcap_tcp_example, setup_tcp_client_server
```

`generate_pcap_tcp_example(net_scheme, src_machine, dst_machine, dst_ip, dst_interface, output_file, dst_port_min=2000, dst_port_max=2999, payload_size=10, step=1) → dict`

Generate a synthetic TCP traffic capture for pcap analysis exercises. Uses 3 consecutive steps starting at `step`. The pcap file is written to `output_file` inside `dst_machine` (must be under `/shared/` for host-side analysis). Returns a dict with the captured port numbers and one TCP frame chosen from each direction:

Key	Description
`server_port`, `client_port`	TCP port numbers used
`packet_src_to_dst`	1-based frame number (src → dst direction)
`packet_src_to_dst_tcp_window`	TCP window field of that frame
`packet_src_to_dst_absolute_seq_number`	Absolute sequence number
`packet_src_to_dst_absolute_ack_number`	Absolute acknowledgment number
`packet_dst_to_src`	Same fields for the dst → src direction

The dict is populated at `step+2` via a host callback after the capture finishes.

`setup_tcp_client_server(net_scheme, src_machine, dst_machine, src_ip, dst_ip, secret, dst_port_min=3000, dst_port_max=3999, interval=3, step=1) → dict`

Deploy a persistent background TCP client/server pair for generating live traffic throughout the lab. Uses 2 steps. The server runs on `dst_machine`, the client on `src_machine`; both survive for the lifetime of the lab. Returns `{'server_port': int, 'client_port': int}`.

Used in `Grade.grade()` (evaluation):

```
from pcap_gen import check_zero_window_probe, get_frame_info
```

`check_zero_window_probe(grade, file, max_length, packet_number) → bool`

Return `True` if the packet at 1-based `packet_number` in the pcap file (relative to the project shared directory) is a valid TCP Zero Window Probe. Checks: file within `max_length` KiB; target packet has ≤ 1 byte payload; a prior packet in the reverse direction advertised window=0; target SEQ equals `SND.UNA` or `SND.UNA-1`.

`get_frame_info(grade, filename, max_length, frame_number) → dict | None`

Return all parsed fields for a single frame from a pcap file (relative to the project shared directory). Returns `None` if the file is missing, too large, not a valid pcap, or `frame_number` does not exist. The returned dict always contains `frame_number`, `frame_link_type`, `captured_length`, `original_length`, `timestamp_sec`, `timestamp_usec`; plus protocol-specific fields for Ethernet, Linux cooked capture, ARP, IPv4, IPv6, ICMP, TCP, and UDP headers.

3.10 Module-level Attributes

Declare these at the top level of `srelab.py` to customize behavior. They are read with `getattr(module, name, default)`, so any attribute may simply be omitted to keep the default.

3.10.1 Lab identity and presentation

Attribute	Type	Default	Description
`title`	`str | Transl`	filename without `.py`	Lab title shown in the GUI tab header and in exported reports. Pass a `TranslatedText` (via `tr()` / `make_tr()`) for multilingual titles. The same title is also surfaced in the GUI's *Open project* picker once an admin has run `sre make-titles` to materialise per-directory `titles.json` sidecars (see `sre make-titles` in the CLI reference)
`defau`	`str`	`'en'`	Language code used to resolve `TranslatedText` values when the GUI's locale is unavailable and when `make_tr()` is called without an explicit `default_language=`

3.10.2 Student self-evaluation

Attribute	Type	Default	Description
allow_self_	bool	Fals	If True, students may trigger their own evaluation from the GUI
delay_betwe	int	0	Cooldown in seconds between two student self-evaluations (only enforced for sre --user eval --auto-eval, i.e. when the student presses **Start evaluation** in the GUI; periodic background evals do not count). The remaining delay is stored under params.self_grade_timestamp_dir (/var/lib/sre/last_self_grades/{lab_name})
no_mark_on_	bool	Fals	During a student-triggered evaluation, hide numeric grades and show only the OK / MEH / FAIL letters
hide_potent	bool	Fals	During a student-triggered evaluation, drop grade-list entries whose grade and max_grade are both 0 (i.e. penalty rubrics that have not yet fired). Has no effect on instructor-triggered evals

3.10.3 Automatic evaluation

Attribute	Type	Default	Description
eval_interval_	int	0	Auto-eval interval in seconds when no exam is configured. 0 disables periodic evaluation
eval_before_ex	bool	False	If True, run a final evaluation when the student closes the project
use_numerical_	bool	params.use_numerical_marks_by_ (True)	If False, the lab reports OK / MEH / FAIL letters instead of numeric scores
display_marks_	bool	params.display_marks_in_auto_e (False)	If False, the numeric mark is hidden from students during automatic (periodic) evaluations even when use_numerical_marks=True
maximum_mark	int | float	params.default_maximum_mark (20)	Upper bound used when normalizing the lab's total grade

3.10.4 Archives and recordings

Attribute	Type	Default	Description
archive_di	list	[]	Extra directories to write evaluation archives (.zst) to, in addition to params.archive_dirs
files_to_s	list	[]	Container paths whose contents are copied into each evaluation archive (under the files key). Useful for capturing config files, logs, or pcap traces produced by the student
record_ses	bool | None	None	Override whether terminal sessions opened from the GUI are recorded (asciinema). None defers to the global / exam setting
save_recor	list	[]	Extra directories to write session-record archives (.tar.gz) to during an exam, in addition to params.save_records_dir (/var/lib/sre/archives). Read by sre save-records and by the automatic save loop during eval-exam
save_recor	int	params.default_save_rec (60)	Minimum delay in seconds between two automatic session-record saves for the same project during an exam. Set to 0 to disable automatic saving for this lab

3.10.5 State machine and flavors

Attribute	Type	Default	Description
allow_user_	bool	Fals	If True, students can apply states decorated with @sre_state(user_allowed=True) from the GUI. States decorated with user_allowed=False remain instructor-only
flavor_form	bool	Fals	If True and a Flavor subclass is defined, the GUI shows the flavor form before calling sre start, letting the student parameterize the lab

3.10.6 Networking

Attribu	Type	Default	Description
ipv6	bool	params.default_ipv6 (None: Kathara's own setting, off)	If True, IPv6 is enabled in every machine (disable_ipv6=0, forwarding and accept_ra=0 set by Kathara); Machine(ipv6=...) overrides it per machine. The effective value is written to the lab.conf of sre export. See *IPv6 labs*

3.10.7 Filesystem and export

Attribute	Type	Default	Description
`shared_`	`bool`	`Fals`	Create `/home/sre/{running_lab_name}/` (mode `0o777`) and bind-mount it into each container for file exchange between host and containers
`export_`	`bool`	`True`	If `False`, disable **File → Export** in the GUI and refuse `sre export` for this lab
`allow_`	`bool`	`Fals`	If `True`, students may save a running project to a file (**File → Save Project**, `sre save`) and restore it later as a new instance (**File → Restore Project**, `sre restore`). The save file holds `data.json` and `.private/files` — in clear unless `save_key` is set — so enable it only for labs whose `Data` holds nothing the student must not read, or set `save_key`
`save_k`	`str`	*(unset)*	Non-empty secret string. When set, the payload of every save file is encrypted (AES-256-GCM, key derived from `save_key`); only a lab with the same key can restore it, and students cannot restore cleartext files. Keep the lab file unreadable by students

3.10.8 Schema rendering

These attributes tweak the graphviz topology diagram drawn for the GUI's *Schema* tab and embedded in the PDF produced by `sre export`.

Attribute	Type	Default	Description
`show_nat_netwo`	`bool`	`params.default_show_nat_netw` (`True`)	If `False`, hide the host-network vertex even when machines are marked `bridged=True`
`host_network_r`	`str`	`params.default_host_network_` (`"Internet"`)	Label shown on the host-network vertex
`host_network_c`	`str`	`params.default_host_network_` (`"deepskyblue"`)	Fill color of the host-network vertex (graphviz color name or `#rrggbb`)
`host_network_e`	`bool`	`params.default_host_network_` (`False`)	If `True`, draw one separate host-network vertex per `bridged=True` machine instead of a single shared one
`host_network_e`	`floa`	`params.default_host_network_` (`1.0`)	Relative length (graphviz `len`) applied to edges between `bridged=True` machines and the host vertex. `< 1` shortens them, `> 1` lengthens them
`schema_splines`	`str`	`params.graphviz_default_spli` (`"curved"`)	Graphviz `splines` attribute (e.g. `"line"`, `"ortho"`, `"curved"`, `"polyline"`)
`schema_overlap`	`str`	`params.graphviz_default_over` (`"prism"`)	Graphviz `overlap` attribute controlling how the layout resolves node overlaps

3.11 Complete Minimal Example

```
# /opt/sre/lab/example/srelab.py

from dataclasses import dataclass
from ipaddress import IPv4Interface, IPv4Network
```

(continues on next page)

(continued from previous page)

```
from typing import Dict

from SRE.lib_sre import Data0, NetScheme0, Grade0, sre_state, make_tr
from ips import random_ipv4networks, random_ipv4s
from net_config import set_net_config_entry, NetConfigEntry,␣
↪get_net_config_from_topology, set_ip_forward
from state_helpers import set_nat_gateway

tr = make_tr('en')
title = tr("Example: Router", fr="Exemple : Routeur")
allow_self_grade = True
delay_between_self_grade = 60
eval_interval_without_exam_mode = 120


@dataclass(slots=True)
class Data(Data0):
    @classmethod
    def generate(cls, flavor=None):
        d = cls()
        d.nets.lan1, d.nets.lan2 = random_ipv4networks([24, 24],␣
↪from_private_network=True)
        d.ips.router_lan1, d.ips.client1 = random_ipv4s(d.nets.lan1, 2)    #␣
↪IPv4Interface values
        d.ips.router_lan2, d.ips.client2 = random_ipv4s(d.nets.lan2, 2)
        return d


class NetScheme(NetScheme0):
    _machine_specs = {
        'router': {'color': 'green'},
        'client1': {},
        'client2': {},
    }
    _network_specs = {
        'lan1': {'color': 'yellow'},
        'lan2': {'color': 'cyan'},
    }
    _topology = {
        'lan1': ['router', 'client1'],
        'lan2': ['router', 'client2'],
    }

    def __init__(self, data, running_lab_name):
        super().__init__(data=data, running_lab_name=running_lab_name)
        self.informations = tr(
            "## Router Lab\nConfigure routing between lan1 and lan2.",
            fr="## TP Routage\nConfigurez le routage entre lan1 et lan2.",
        )
        d = data
        self.net_config = get_net_config_from_topology(net_scheme=self, gateway="router")
        # self.net_config: Dict[str, NetConfigEntry] = {
```

(continues on next page)

(continued from previous page)

```
        #     'router': [([d.ips.router_lan1], [(d.nets.lan2, d.ips.router_lan2)]),
        #                ([d.ips.router_lan2], [])],
        #     'client1': [([d.ips.client1], [(IPv4Network('0.0.0.0/0'),␣
↪d.ips.router_lan1)])],
        #     'client2': [([d.ips.client2], [(IPv4Network('0.0.0.0/0'),␣
↪d.ips.router_lan2)])],
        # }

    @sre_state(user_allowed=False)
    def initial(self):
        for machine, config in self.net_config.items():
            set_net_config_entry(net_scheme=self, machine_name=machine, nc_entry=config)
        self.cmd('router', 'sysctl -w net.ipv4.ip_forward=1')
        for m in self.get_visible_machine_names():
            set_ip_forward(net_scheme=self, machine_name=m, ip_forward=(m == "router"))
        set_nat_gateway(net_scheme=self, machine="router")



class Grade(Grade0):
    def grade(self):
        super().grade()
        d = self.data

        # Test cross-network reachability
        result, code = self.test('client1', f'ping -c1 -W2 {d.ips.router_lan2.ip}',␣
↪timeout=5)
        self.add_grade_element('Cross-network ping', max_grade=10)
        if code == 0:
            self.set_grade('Cross-network ping', 10)

        # Ask a question
        answer = self.question_text(
            title=tr('Routing protocol', fr='Protocole de routage'),
            description=tr('Which mechanism routes packets between the two LANs?',
                           fr='Quel mécanisme route les paquets entre les deux LANs ?'),
        )
        self.add_grade_element('Routing answer', max_grade=5)
        if 'ip_forward' in answer or 'forward' in answer.lower():
            self.set_grade('Routing answer', 5)
```

CHAPTER

FOUR

HUBS, SWITCHES AND VLANS

Every network of a lab connects its machines through one of three kinds of equipment. The lab chooses the kind per network; a network the lab says nothing about is a hub, as all networks were before this choice existed.

Kind	`mode` in the lab	Name in the GUI	What it does
Hub	`'hub'` (default)	*Hub*	Every frame is sent to every machine of the network: any machine can capture the traffic of the others (`tcpdump`, Wireshark).
Switch	`'switch`	*Switch*	The switch learns the MAC addresses and sends a frame to its destination only: a machine sees its own traffic and the broadcasts. No configuration, no console.
Manageable switch	`'manage`	*Manageable switch*	A switch with VLANs (access and trunk ports) and a management console, which the students may open unless the lab closes it.

This page describes the feature as a whole. The reference tables of the lab API are in the *Lab Authoring Guide*, the commands in the *CLI Reference*, and the plugin a host needs for switches in *Installation*.

4.1 In the GUI

4.1.1 The Networks tab

A project always shows a **Networks** tab, after **Machines**, whatever the types of its networks. The tab lists every network the student can see — hubs included — with three columns:

Column	Content
Name	Name of the network, as on the schema.
Type	*Hub*, *Switch* or *Manageable switch*. On a red background for a manageable switch whose console the lab closed to the students.
Connection	A green *Connect* button for a manageable switch the students may use. Empty otherwise: a hub or a plain switch has no console, and a closed manageable switch is not offered.

Connect opens the management console of the switch in an external terminal, like the *Connect* button of the **Machines** tab does for a machine. Each click opens a new terminal.

The **Terminals** tab also holds one embedded terminal per manageable switch the students may use, next to those of the machines.

A network that only connects hidden machines is not listed.

In a debug project (`sre start --debug-project`) every network is listed, and the manageable switches the lab closed to the students can be used all the same: their type stays red, but they get an orange *Connect* button in the Networks tab and a terminal in the Terminals tab, whose title is orange like the title of a machine the students cannot connect to.

4.1.2 The console of a manageable switch

The console is the management console of the underlying `vde_switch`. The prompt is the name of the network; `exit`, `quit` or Ctrl-D leave it.

```
lan$ vlan/print
VLAN 0010
 -- Port 0001 tagged=1 active=1 status=Forwarding
 -- Port 0002 tagged=0 active=1 status=Forwarding
VLAN 0020
 -- Port 0001 tagged=1 active=1 status=Forwarding
 -- Port 0003 tagged=0 active=1 status=Forwarding
lan$ port/print
Port 0001 untagged_vlan=0000 ACTIVE - NOT Unnamed Allocatable
 Current User: root Access Control: (User: NONE - Group: NONE)
  -- endpoint ID 0003 module unix prog   : kathara r1:eth0 user=0 pid=64086
Port 0002 untagged_vlan=0010 ACTIVE - NOT Unnamed Allocatable
 Current User: root Access Control: (User: NONE - Group: NONE)
  -- endpoint ID 0008 module unix prog   : kathara pc1:eth0 user=0 pid=64086
...
lan$ port/setvlan 3 10
lan$ exit
```

Command	Effect
`help`	List of the commands of the switch.
`port/print`, `port/allprint`	Ports in use / every port: the VLAN of its untagged frames (`untagged_vlan`) and what is plugged into it (`kathara <machine>:eth<N>`).
`vlan/print`, `vlan/allprint`	VLANs with their ports; `tagged=1` marks a trunk member, `tagged=0` an access port.
`vlan/create <vlan>`, `vlan/remove <vlan>`	Create / remove a VLAN.
`port/setvlan <port> <vlan>`	Put a port in a VLAN for its untagged frames (access port).
`vlan/addport <vlan> <port>`, `vlan/delport <vlan> <port>`	Add / remove a port as tagged member of a VLAN (trunk port).
`hash/print`, `hash/find <mac>`	MAC address table.
`fstp/setfstp <0|1>`, `fstp/print`	Spanning tree on / off, and its state.

Things to know:

- **Port numbers are not predictable.** They are given when the project starts: read them with `port/print`, where each port names the machine interface plugged into it.

- **VLAN 0 is the default VLAN** of the switch: a port the lab did not put in a VLAN is there, untagged, with every other such port.
- **A command the switch refuses** prints its error after `error:` (for instance `error: File exists` when creating a VLAN that exists) and the session goes on.
- **Students get a subset of the commands.** Through the GUI (user mode) only the commands of `params.switch_user_commands` reach the switch: those of the table above, `showinfo`, `port/showinfo`, `hash/showinfo`, `fstp/showinfo`, `fstp/setedge` and `fstp/bonus`. Any other one — creating or removing ports, loading plugins, shutting the switch down… — is answered `error: command '…' is not allowed` without being sent. A privileged user (`sre connect` from a root shell) is not restricted.
- **The console ends with the project.** When the project is closed (**File → Close Project**, `sre stop`, `sre wipe`) an open console prints *The project was closed* and ends within a second, so its terminal closes like the terminal of a machine.
- **What is typed on the console is not saved** with **File → Save Project**: a restored project gets the switches as the lab declares them (see *Limits*).

4.2 In a lab

4.2.1 Declaring the kind of a network

The kind of a network and, for a manageable switch, its VLANs and the access to its console are given in `_network_specs`. `_topology` keeps its syntax, and a network without entry — or without `mode` — is a hub.

```
class NetScheme(NetScheme0):
    _machine_specs = {'pc1': {}, 'pc2': {}, 'r1': {}, 'r2': {}, 'srv': {}}
    _network_specs = {
        'lan': {'mode': 'managed',                          # manageable switch
                'allow_connection': False,                  # students may not open its␣
↪console (default: True)
                'vlans': {'pc1': 10,                        # access port: untagged␣
↪frames in VLAN 10
                          'pc2': 20,
                          'r1': [10, 20],                   # trunk port: VLANs 10 and␣
↪20, tagged
                          'r2': {'vlan': 1, 'trunk': [30]}}},  # native VLAN 1 + VLAN 30␣
↪tagged
        'dmz': {'mode': 'switch'},                          # switch without VLAN nor␣
↪console
    }
    _topology = {
        'lan': ['pc1', 'pc2', 'r1', 'r2'],
        'dmz': ['r1', 'srv'],
        'old': ['srv', 'pc2'],                              # no entry in _
↪network_specs: a hub
    }
```

Key	Default	Meaning
`mode`	`'hub'`	`'hub'`, `'switch'` or `'managed'`.
`allow_cor`	`True`	Manageable switch only: the students may open its console. `False` shows the type of the switch in red in the Networks tab, without *Connect* button, and makes `sre connect` refuse them.
`vlans`	`{}`	Manageable switch only: the VLANs of the ports, by machine name.

The value given for a machine in `vlans` describes its port:

Value	Port
`10`	Access port: the untagged frames of the machine are in VLAN 10.
`[10, 20]`	Trunk port: the machine exchanges the frames of VLANs 10 and 20 with 802.1Q tags. Its untagged frames stay in the default VLAN.
`{'vlan': 1, 'trunk': [10, 20]}`	Both: native (untagged) VLAN 1 and tagged VLANs 10 and 20.

Rules:

- VLAN IDs go from 1 to 4094, and a VLAN cannot be both the untagged and a tagged VLAN of one port.
- A machine `vlans` does not name stays in the default VLAN of the switch (VLAN 0), with every other undeclared port.
- On a trunk port the machine handles the tags itself, typically with one sub-interface per VLAN — the classic *router on a stick*:

```
self.cmd('r1', 'ip link add link eth0 name eth0.10 type vlan id 10')
self.cmd('r1', f'ip addr add {d.ips.r1_vlan10} dev eth0.10')
self.cmd('r1', 'ip link set eth0.10 up')
```

- A switch that is not a hub drops the tagged frames of a VLAN the port is not a member of: a trunk between two machines needs a hub, or a manageable switch with trunk ports — not a plain `'switch'`.
- An unknown `mode`, `vlans` on a network that is not manageable, a machine that is not on the network or an invalid VLAN ID raise `ValueError` when the `NetScheme` is built: `sre check` reports them before any start.

4.2.2 Running switch commands from a state

`self.switch_cmd(network, command, step=1, default_value='', default_code=0, allow_error=False)` runs one command of the console of a manageable switch when the state is applied. Since port numbers are not known in advance, a word `@machine` in the command stands for the number of the port that machine is plugged into:

```
@sre_state(user_allowed=True, description="pc2 joins VLAN 10")
def move(self):
    self.switch_cmd('lan', 'vlan/create 10', allow_error=True)   # code 17: the VLAN␣
↪already exists
    self.switch_cmd('lan', 'port/setvlan @pc2 10')
```

It follows the contract of `host_cmd()`: it returns `(output, code)`, a placeholder until the command has run (the real result is visible in a `multi_pass` state). `code` is `0`, the error number given by the switch, or `-2` when the command could not be run at all (console unreachable, unknown `@machine`). The switch commands of a step run with its host operations, in registration order, before the operations on the containers. See *Switch operations*.

4.2.3 Grading the configuration of a switch

`self.test_switch(network, command, step=1, ...)` registers a console command in `grade()` like `self.test()` does for a machine, and returns `(output, code)` on the following pass. The helpers of `lib/switch.py` turn the port and VLAN tables into something directly usable:

```
from switch import get_switch_ports, get_switch_vlans

class Grade(Grade0):
    def grade(self):
        super().grade()
        ports = get_switch_ports(self, 'lan')
        # {'pc1': {'port': 2, 'interface': 'eth0', 'vlan': 10, 'tagged_vlans': []},
        #  'r1':  {'port': 1, 'interface': 'eth0', 'vlan': 0,  'tagged_vlans': [10, 20]}}
        in_vlan_10 = ports.get('pc2', {}).get('vlan') == 10
        self.add_grade_element(title='pc2 in VLAN 10', grade=int(in_vlan_10),␣
↪max_grade=1)

        vlans = get_switch_vlans(self, 'lan')   # {10: {'untagged': [2], 'tagged': [1]},␣
↪...} (port numbers)
```

Both helpers return `{}` on the registration pass and when the console failed. The results are archived with the tests of the machines, under the name of the network, so `sre cat`, `sre re-eval` and `sre check-eval` handle them like any other test. Behavioural checks (a ping between two machines of a VLAN, a capture on a third machine) remain the most robust way to grade what the students were asked to obtain; reading the switch tells *how* they obtained it. See *Switch helpers*.

4.2.4 A complete example

`lab/sre/_DRAFT_dummy/switch_example.py` uses everything above in one small lab: a hub, a switch and two manageable switches (one open, one closed), access ports and a trunk port with a router on a stick, two states built on `switch_cmd()`, questions about what a third machine sees on the hub and on the switch, and a grade reading the VLANs from the switch. It is in a `_DRAFT_` directory: start it as a privileged user with `sre start sre/_DRAFT_dummy/switch_example.py`.

4.3 On the command line

Command	With a switch
`sre connect <running_lab> <network>`	Opens the console of a manageable switch. In user mode: the allowed commands only, and refused when the lab closed the console (`allow_connection`) or when the network only connects hidden machines — unless the project is a debug project.
`sre exec <running_lab> <network> <command…>`	Privileged: runs one console command and exits with 0 or the error number of the switch, e.g. `sre exec <running_lab> lan port/setvlan @pc2 10`.
`sre check <lab> [<state>]`	Prints the kind of each network that is not a hub with the declared VLANs, and the switch commands of the states.
`sre export <running_lab>`	Writes the kinds and the VLANs in `lab.conf`: `pc1[0]="lan/vlan=10"`, `r1[0]="lan/trunk=10,20"`, `CD_MODE[lan]="managed"`. This syntax is the one of the Kathara fork SRE installs.

A hub or a plain switch has no console: `sre connect` and `sre exec` answer `device <network> is a hub: it has no console`.

In a debug project, and for the states of a project in instructor mode, the **Log** tab shows the switch commands a state or an evaluation ran, as `stepN - on switch <network> : <command>` followed by the output and the exit code.

4.4 Limits

- **Save and restore.** A save file keeps the kind of each network and the declared VLANs, not what was changed on a console afterwards: a restored project starts again from the lab's declaration. A lab for which this matters re-applies its own configuration in its `restore` state with `switch_cmd()`.
- **Schema.** The schema draws every network the same way, whatever its kind; the Networks tab tells them apart.
- **Trunks through a plain switch.** See the rules above: use a hub or a manageable switch.
- **Rates.** Links are user-space switches (about 45–50 Mbit/s in TCP), whatever the kind.

4.5 What a host needs

Hubs work everywhere. Switches and manageable switches need the Kathara fork SRE installs (`make venv`) and its VDE network plugin, which `make network-plugin` installs in place of the stock plugin — once per host, see *Installation*. On a host that only has the stock plugin, starting a lab with a switch fails cleanly: nothing is left running and the message names the networks concerned and the command to run.

```
sre: Collision domain `sw` cannot be deployed in `switch` mode: the Kathara Network␣
↪Plugin `kathara/katharanp_vde:amd64` does not support it. Use an up-to-date VDE␣
↪version of the plugin.
This lab has networks that are not hubs (lan, dmz, sw): they need the Kathara network␣
↪plugin with the switch types.
Install it with `make network-plugin` in /opt/sre (see the installation guide).
```

4.6 Tests

- Unit tests (no Docker): `tests/test_switch_modes.py` (declaration, deployment calls, `switch_cmd()`, `test_switch()`), `tests/test_switch_console.py` (console, allowed commands, `sre connect` / `sre exec`), `tests/test_switch_lib.py` (helpers of `lib/switch.py`), `tests/test_gui_switches_view.py` (Networks tab).
- Live tests (`make docker-tests`, real containers): `TestHub` checks that a third machine captures the traffic of a hub — it runs with any plugin — and `TestSwitchModes` that it does not on a switch nor inside a VLAN of a manageable switch, plus VLAN isolation, the console, a state, grading, export and save / restore. `TestSwitchModes` is skipped on a host without the plugin with the switch types.

CHAPTER

FIVE

TRANSLATING A LAB

5.1 Internationalization with `tr()` / `make_tr()`

```
from SRE.lib_sre import make_tr

default_language = 'en'
tr = make_tr(default_language)

title = tr("SSH Lab", fr="TP SSH")
# In NetScheme.__init__:
self.informations = tr("## Description\nConfigure SSH.",
                       fr="## Description\nConfigurez SSH.")
```

`tr(default, **lang_overrides)` returns a `TranslatedText` dict subclass. Call `.resolve('fr')` to get the string for a given language.

`make_tr(default_lang, translations=None)` returns a `tr()` function bound to *default_lang* as the key for the first positional argument. When `translations=` is omitted, the caller module's globals are inspected for a `_TRANSLATIONS` dict at each call; pass it explicitly to avoid relying on module-level state.

5.2 Marking strings that must NOT be translated: `no_tr()`

```
from SRE.lib_sre import no_tr

label = no_tr("MTU")              # short internal label, identical in every language
```

`no_tr()` is an identity passthrough at runtime — it returns the string unchanged. Its sole purpose is to flag a literal so the translation toolchain leaves it alone: it is never wrapped in `tr()` and never registered in `_TRANSLATIONS`. Use it for short internal labels (grade-element titles, protocol names, command names) that are not natural-language prose.

By default, `prepare-sre-translations` already wraps any whitespace-free single-token bare string in `no_tr()` rather than `tr()` (see `--translate-isolated-words` below to override).

5.3 Centralizing translations with `_TRANSLATIONS`

For labs with many strings, keeping translations inline in every `tr()` call becomes unwieldy. The `_TRANSLATIONS` dict centralises them all in one place:

```
from SRE.lib_sre import make_tr

default_language = 'en'
tr = make_tr(default_language)

_TRANSLATIONS = {
    'fr': {
        "SSH Lab":                  "TP SSH",
        "Configure the SSH server": "Configurez le serveur SSH.",
        "SSH port":                 None,   # not yet translated
    },
}

title       = tr("SSH Lab")
description = tr("Configure the SSH server")
```

`None` marks a string that still needs translation; `tr()` falls back to the default-language text in that case.

You can pass the dict explicitly to `make_tr()`, which avoids any reliance on module-level state:

```
tr = make_tr('en', translations=_TRANSLATIONS)
```

5.3.1 Where to put _TRANSLATIONS

Two valid placements; the choice affects which `tr()` calls can resolve translations at import time.

At the top of the file (default for `prepare-sre-translations`): `_TRANSLATIONS` is defined right after `tr = make_tr(...)`, before any `tr()` call. Every `tr()` call — including module-level and class-body ones evaluated at import time — can look up its translation. Inline `fr=`/`de=`/… kwargs are not needed.

At the end of the file (`prepare-sre-translations --translations-at-the-end`): `_TRANSLATIONS` lives at the very bottom, out of the way of the lab code. Because Python evaluates the module top-to-bottom, any `tr()` call in the module body or in class bodies runs *before* `_TRANSLATIONS` exists. To keep those import-time calls correct, `prepare-sre-translations` keeps (and re-adds on each run) inline lang kwargs on every such call. `tr()` calls inside method bodies — which only run at lab-execution time — do not need the kwargs and remain bare.

5.3.2 Dynamic strings with format placeholders

For strings that embed runtime values, use `{placeholder}` syntax in `_TRANSLATIONS` values and call `.format()` on the result:

```
_TRANSLATIONS = {
    'fr': {
        "The machine {machine} is configured": "La machine {machine} est configurée",
    },
}

msg = tr("The machine {machine} is configured").format(machine=m)
```

The static key `"The machine {machine} is configured"` is what `tr()` looks up in `_TRANSLATIONS`; `.format()` applies `str.format` to every language value at once and returns a new `TranslatedText`.

This is the equivalent of the inline f-string form:

```
msg = tr(f"The machine {m} is configured",
         fr=f"La machine {m} est configurée")
```

The inline f-string form still works and is fine for one-off strings. Use `_TRANSLATIONS` + `.format()` when the same template appears in many places, or when you want the translation toolchain (`check-sre-translations`, `add-sre-translations`) to see the string as a static literal.

5.4 Translation toolchain

Three command-line tools manage the complete translation lifecycle.

5.4.1 1. `prepare-sre-translations` — migrate and scaffold

`sbin/prepare-sre-translations` reads a lab file and:

- wraps bare translatable strings (module-level `title`, `description`, `informations`, `lab_name`; `self.informations` / `self.description`; `title=`/`description=`/`informations=` kwargs; first positional arg of `question_text`/`question_form`/`question_dummy`/`add_grade_element`/`add_grade_part`) in `tr()` — or in `no_tr()` if the string is a single whitespace-free token
- recurses into `BinOp` (`"x" + var`) and `IfExp` (`"x" if c else "y"`) expressions in those positions, wrapping each leaf
- lowers bare f-strings to `tr("template").format(var=var, ...)`
- creates or updates `_TRANSLATIONS` with one entry per string per language (value `None` for strings not yet translated)
- inserts `tr = make_tr(...)` and the matching `from SRE.lib_sre import make_tr` (and `no_tr` if needed) if absent

```
prepare-sre-translations [--move-tr-strings] [--default-language xx]
                         [--change-default-language xx]
                         [--translations-at-the-end]
                         [--translate-isolated-words] file
```

Option	Effect
(no options)	Wrap bare strings; create/update `_TRANSLATIONS` after `tr = make_tr(...)`; infer default language from file or assume `en`
`--default-xx`	Declare (or confirm) the default language; error if the file already uses a different one
`--change-d xx`	Pivot the file's source language to `xx`: rewrite every `tr()` literal to its `xx` translation, re-key `_TRANSLATIONS` so the inner keys are the new source texts, and preserve the previous default as a regular language. Errors out if any `tr()` string lacks an `xx` translation, or any translatable string is still bare.
`--move-tr-`	Strip inline `fr=`/`de=`/… kwargs from existing `tr()` calls and move them into `_TRANSLATIONS`
`--translat`	Place `_TRANSLATIONS` at the very end of the file; keep/add inline lang kwargs on every import-time `tr()` call (module/class body) so they don't depend on the late definition
`--translat`	Also wrap whitespace-free single-token strings in `tr()` (default: wrap them in `no_tr()`)

`--default-language` and `--change-default-language` are mutually exclusive.

Strings already wrapped in `no_tr(...)` are left untouched: never wrapped in `tr()` and never added to `_TRANSLATIONS`.

Typical first run on an existing lab:

```
# Wrap bare strings, declare the language, migrate inline kwargs:
sbin/prepare-sre-translations --default-language en --move-tr-strings lab/my_lab.py
```

The file is modified in-place. Run it repeatedly — it is idempotent: already-wrapped strings and existing translations are left unchanged.

5.4.2 2. `check-sre-translations` — verify consistency

`sbin/check-sre-translations` performs a static (AST-level) analysis and reports three categories of problem:

Category	Meaning
`MISSING`	String appears in a `tr()` call but has no entry in `_TRANSLATIONS`
`UNTRANSLATED`	Entry exists but its value is `None` (translation not yet done)
`VANISHED`	Entry exists in `_TRANSLATIONS` but no `tr()` call uses that string any more

```
sbin/check-sre-translations lab/my_lab.py
# lab/my_lab.py: MISSING       [fr] 'SSH port'
# lab/my_lab.py: UNTRANSLATED  [fr] 'Configure the SSH server'
# lab/my_lab.py: VANISHED      [de] 'Old string'
# lab/my_lab.py: ok (12 strings, ['fr', 'de'] languages)
```

Exit code is 0 if no issues are found, 1 otherwise. Suitable for CI.

Multiple files are accepted; let the shell expand globs:

```
sbin/check-sre-translations lab/s4/*.py
```

5.4.3 3. `add-sre-translations` — machine-translate missing strings

`sbin/add-sre-translations` calls an online translation service to fill in every `None` value for one target language, then writes the result back into the file.

```
add-sre-translations --language XX [--service YY]
    [--api-key KEY] [--credentials FILE] [--region REGION]
    [--libre-url URL] [--auto] file
```

Services (`--service`):

Service	Default credentials
`deepl` *(default)*	`--api-key` or `$DEEPL_API_KEY`
`google`	`--api-key` or `$GOOGLE_API_KEY` (REST); or `--credentials service-account.json` / `$GOOGLE_APPLICATION_CREDENTIALS`
`libre`	`--api-key` or `$LIBRE_API_KEY` (optional); `--libre-url` for a self-hosted instance (default: `https://libretranslate.com`)
`azure`	`--api-key` or `$AZURE_TRANSLATOR_KEY`; `--region` (default: `global`)
`amazon`	Standard AWS credential chain (`~/.aws/` or env vars); `--region` or `$AWS_DEFAULT_REGION`

Interactive mode (default when stdin is a TTY):

For each string the tool shows the machine translation and prompts:

```
[fr] 'SSH port'
     → 'Port SSH'
  Accept [Enter] or type correction (q to quit):
```

Press Enter to accept, type a correction, or `q` to stop early. Progress is always saved, even on Ctrl-C or an API error. Pass `--auto` to accept all suggestions without prompting.

Typical session:

```
export DEEPL_API_KEY=your-key-here

# Fill in French translations interactively:
sbin/add-sre-translations --language fr lab/my_lab.py

# Fill in German translations automatically:
sbin/add-sre-translations --language de --auto lab/my_lab.py

# Verify everything is done:
sbin/check-sre-translations lab/my_lab.py
```

5.5 Complete translation workflow

For a new lab starting from scratch in English:

```
# 1. Scaffold: wrap strings, create _TRANSLATIONS skeleton at the end of the file
sbin/prepare-sre-translations --default-language en --translations-at-the-end␣
↪lab/my_lab.py

# 2. Verify what needs translating
sbin/check-sre-translations lab/my_lab.py

# 3. Fill in French with interactive review
sbin/add-sre-translations --language fr lab/my_lab.py

# 4. Final check - should report no issues
sbin/check-sre-translations lab/my_lab.py
```

For a lab that already uses inline `tr("text", fr="traduction")` kwargs:

```
# Migrate inline kwargs into _TRANSLATIONS in one step
sbin/prepare-sre-translations --move-tr-strings lab/my_lab.py

# Review and fill in anything still missing
sbin/check-sre-translations lab/my_lab.py
sbin/add-sre-translations --language fr lab/my_lab.py
```

5.6 Switching the default language: `--change-default-language`

`prepare-sre-translations --change-default-language xx` pivots the file's source language to `xx`. The resulting file is *semantically equivalent* to the original — same translations, just keyed off a different source — and `check-sre-translations` should report it clean immediately after the pivot.

It performs four edits:

- `default_language = '...'` is set to `xx`;
- the literal first argument of `make_tr('...')` is set to `xx` (a `Name` arg like `make_tr(default_language)` is left as-is and inherits the new value);
- every `tr("old_text", ...)` call's first positional arg is replaced with its `xx` translation; the `xx=...` kwarg, if present, is dropped (it is now the default), and if the call had any inline language kwargs, the previous default is added as `<prev>="old_text"` to preserve the inline-kwarg style;
- `_TRANSLATIONS` is re-keyed: the inner keys flip from the previous source texts to the new `xx` source texts, the `xx` top-level entry disappears, and the previous default appears as a regular language entry mapping new-source → previous-source text.

5.6.1 Prerequisites

The pivot only runs if the file is already fully prepared and every `tr()` literal has an `xx` translation reachable via inline kwargs or `_TRANSLATIONS`. The script reports an error and leaves the file untouched when it finds any of:

- a `tr()` literal without an `xx` translation (`add-sre-translations --language xx` is the suggested fix);
- a `tr()` call whose first argument is not a string literal (variables, f-strings, expressions cannot be statically rewritten — lower or inline these first by running `prepare-sre-translations` without `--change-default-language`);
- a bare translatable string still waiting to be wrapped (same fix: run `prepare-sre-translations` without `--change-default-language` first).

`--change-default-language` is incompatible with `--default-language`, and rejects a pivot to the language that is already the file's default.

5.6.2 Example: switching to English as a pivot before translating

Machine-translation services produce noticeably better results when English is the source language, especially for less-common target pairs. If your lab is currently in another language but you want to pivot to English so every subsequent `add-sre-translations --language fr/de/es/...` call uses English as `source_lang`:

```
# 1. Fill in English translations first - these become the new source.
sbin/add-sre-translations --language en lab/my_lab.py

# 2. Pivot the file: tr() literals + _TRANSLATIONS keys + declarations all move to en.
sbin/prepare-sre-translations --change-default-language en lab/my_lab.py

# 3. Every add-sre-translations call now uses English as source_lang.
sbin/add-sre-translations --language fr lab/my_lab.py
sbin/add-sre-translations --language de lab/my_lab.py
sbin/add-sre-translations --language es lab/my_lab.py
```

CHAPTER

SIX

INSTALLATION AND SETUP

6.1 Prerequisites

- A Linux host. `scripts/install.sh` knows how to `apt-get install` missing packages on Debian and Ubuntu; on anything else you must install the prerequisites yourself.
- **Root** access. The installer refuses to start unless `EUID == 0` (run with `sudo`).
- The **Docker daemon running**, with `/var/run/docker.sock` present. The installer reads the socket's GID and grants the `sre` user access to it; if the socket is missing it aborts with a hint to start the daemon.
- **Python 3.13**. On Ubuntu < 24.04 you may need the `deadsnakes` PPA; on Debian < 13 you need to upgrade or build it from source. The installer will offer to `apt-get install python3.13` on Debian-likes.
- **AppArmor** (on by default on Debian and Ubuntu) needs no action: the Kathara fork runs the non-privileged machines with the Docker option `apparmor=unconfined`, because Docker's `docker-default` profile denies the remount of `/proc/sys` and every write under `/proc/sys/net/` (the `initial` state of a lab could not reset `ip_forward`, and the students could not set it either; Docker runs the privileged machines unconfined by itself). With a stock Kathara, boot the host with `apparmor=0` on the kernel command line instead.

These tools must already be on `PATH` (always present on a POSIX system): `sed grep stat getent useradd groupadd usermod install`.

These tools are auto-installed via `apt-get` on Debian-likes if missing:

- `docker` (any variety) `asciinema make gcc graphviz python3.13`

On other distributions install them with your package manager before running the installer.

6.2 Deployment layout

SRE's runtime root defaults to `/opt/sre`, but the installer makes it configurable. Clone the repo (`git clone https://github.com/sysreseval/sysreseval /opt/sre`) or untar the release into `/opt/sre` — or into any other location you prefer.

`scripts/install.sh` detects where it is being run from and offers that path as the default — `/opt/sre` when run from `/opt/sre/scripts/`, otherwise the parent of the script directory. The chosen value is patched into `src/SRE/params.py` (`main_sre_dir`) and substituted for `/opt/sre` in the sudoers rule, the `.desktop` file, and the `sre-preload-images.service` unit at install time.

The installer is **interactive** — it asks roughly ten configuration questions, then creates the `sre` user, installs a sudoers file, and runs the build — and is meant to be run **once per build**, not once per student workstation. The trick to scaling from a single interactive run to a classroom of student machines is to share the resulting

runtime tree (and replay a small per-host setup on each workstation). The rest of this document uses `/opt/sre` as the canonical example path; substitute your `main_sre_dir` wherever it appears if you chose something else.

Two common topologies:

Single read-only NFS export (recommended for classrooms). Clone the repo on a build server inside the directory you intend to export, run `sudo ./scripts/install.sh` there once, then export the directory read-only and mount it as `/opt/sre` on every student workstation. Updates take effect everywhere as soon as the export is refreshed, and students cannot tamper with the binaries or with the project files. The shell wrappers in `sbin/` and `bin/` don't write to their own tree, and all runtime state lives elsewhere — `/var/lib/sre` (`sre_pub_dir`) and `/home/sre` (`sre_user_public_dir`) stay on the local workstation — so a read-only mount works.

Build once, replicate per workstation. Run the installer once on a reference machine cloned at `/opt/sre`, then `rsync` (or image-clone) the resulting `/opt/sre` tree to every workstation. Running `install.sh` interactively on each machine is impractical for more than a handful of hosts.

Either way, each student workstation still needs a few per-host bits that aren't part of the `/opt/sre` tree:

- the `sre` system user/group, with the UID/GID baked into `params.py` at build time
- `/etc/sudoers.d/sre`
- membership of `sre` in the local `docker` group
- a copy of `scripts/etc/sre_bash_completion` into `/etc/bash_completion.d/sre`
- a copy of `scripts/etc/sysreseval.desktop` into `/usr/share/applications/`
- raised inotify limits (`scripts/etc/sre-inotify.conf` into `/etc/sysctl.d/`, then `sysctl --system`)
- the other post-install steps (see *Post-install steps*)
- for the NFS topology, the mount itself

Script these from the relevant sections of `scripts/install.sh` if you have many machines.

The `/opt/sre` tree must contain `lab/`, `lib/`, `graphics/`, `translations/`, `locale/`, `bin/`, `sbin/`, `src/`, `venv/`. Lab authors also browse `src/` while writing `srelab.py` files. See *Runtime & Internals* for the expected layout.

If you'd rather build in a separate working checkout (CI runner, personal dev tree) and move the result over afterwards, `rsync` or bind-mount the built tree into `/opt/sre` once `make install` has finished, then continue with the *Post-install steps*.

6.3 Debian package

On Debian 13 (trixie) and 14 (forky) SRE can be installed from Debian packages instead of a checkout: `python3-kathara` (the SRE fork of Kathara as a Python library), `sysreseval` and `sysreseval-doc`, plus `python3-fs` on forky (rebuilt from the trixie source package, since forky dropped it). Every other dependency is a Debian package. The packages are served by the apt repository of the project on GitHub Pages (signed; `<dist>` is `trixie` or `forky`):

```
sudo curl -fsSL -o /usr/share/keyrings/sysreseval-archive-keyring.gpg␣
↪https://sysreseval.github.io/sysreseval/apt/sysreseval-archive-keyring.gpg
sudo curl -fsSL -o /etc/apt/sources.list.d/sysreseval.sources␣
↪https://sysreseval.github.io/sysreseval/apt/sysreseval-<dist>.sources
sudo apt update && sudo apt install sysreseval
```

The same `.deb` files are attached to each GitHub release: `apt install ./python3-kathara_*.deb ./sysreseval_*.deb` installs them with their dependencies without the repository.

The package follows the FHS instead of the `/opt/sre` tree:

Path	Content
`/usr/sbin/sre`, `/usr/bin/sysreseval`, `/usr/bin/sre-wrapper`	the CLI, the GUI, the wrapper (`sudo /usr/sbin/sre --user`)
`/usr/bin/strip-instructor`, `/usr/bin/*-sre-translations`	the lab-authoring tools
`/usr/sbin/sysreseval-network-plugin`	`make network-plugin` of a checkout (`scripts/network-plugin.sh`)
`/usr/lib/sysreseval/`	the Python code (`src/`), `lib/` readable by the `sre` user only
`/usr/share/sysreseval/lab/sre/`	the labs of the project, linked as `/var/lib/sre/lab/sre`
`/var/lib/sre/`	projects, archives, `exam.json`, `home/` (the directories shared with the machines), `lab/` (the site's own labs next to the `sre` link)
`/etc/sysreseval/params.py`	the site parameters (a conffile)
`/etc/sudoers.d/sysreseval`, `/usr/lib/sysctl.d/60-sysreseval.conf`, the `.desktop` file, the bash completion, `sysreseval-preload-images.service` (disabled)	the per-host bits of the list above

`params.py` is not edited on a packaged host: `/etc/sysreseval/params.py` (Python syntax) sets the site's values of any parameter (`admin_gids`, `terminal_cmd_prefix`, `allow_privileged_machines`, `archive_dirs`, `sre_user_public_dir`…). It is executed after the base parameters and before the derived ones, so a changed `sre_pub_dir` moves the projects, archives and labs with it. The `sre` system user is created with a dynamic uid unless it exists already (create it beforehand with a fixed uid when `/var/lib/sre` or the archives are shared over NFS); its uid, gid and the `docker` gid are read from the accounts at run time. `sre update` is disabled on a packaged host (apt updates it), `apt purge` keeps `/var/lib/sre` and the user. `sysreseval-network-plugin` still has to be run once per Docker host for the labs with switch types, and the X server must accept TCP connections (post-install steps 2 below).

Releasing. `scripts/release.sh 0.13.2` (or `make release VERSION=0.13.2`) asks to confirm the version and the uncommitted leftovers, asks for release notes (one item per line; `-m` / `--notes-file` give them, `--yes` skips the questions), sets `sre_version` in `params.py`, adds the `0.13.2-1` entry to `debian/changelog` with the notes as its bullets, runs the unit tests, commits `Release 0.13.2` (notes in the body), tags `0.13.2`, pushes, creates the GitHub release with the notes followed by the generated list of commits, then waits for the two workflows: `release.yml` builds the trixie and forky packages and attaches them to the release, `pages.yml` deploys GitHub Pages with the documentation of `docs/` and the apt repository rebuilt by `scripts/apt-repo.sh` from the packages of every release (nothing binary is kept in git). Once per repository, `scripts/release.sh --setup` generates the signing key of the repository (with `gpg`, in your keyring), stores it as the secret `APT_SIGNING_KEY` and switches GitHub Pages to the Actions deployment.

Building the packages by hand. `scripts/build-deb.sh --dist trixie --ref 0.12.0 --test-install` builds the packages of the release tagged `0.12.0` in a `debian:trixie` container (Docker on the build host, which may be a Mac or a CI runner), checks them with lintian and installs them in a second container where it exercises `sre` (`sre list` through the link, `sre check` of a shipped lab, a student running `sre-wrapper`, the purge). `--local` packages the working tree instead (`make deb DIST=forky`); `--arch arm64` builds for that architecture; the result is in `build/deb/<dist>/`. The packaging lives in `debian/` (sysreseval), `packaging/kathara/` (the overlay applied to the fork tarball at the commit of `pin.sh`) and `packaging/python-fs/` (the forky rebuild). `debian/rules` sets `install_layout =`

"fhs" and the version in the installed params.py; the unit tests of the repository run during the build unless DEB_BUILD_OPTIONS=nocheck (--no-check). The GitHub workflow .github/workflows/release.yml runs the script for trixie and forky on every tag and attaches the .deb files to the release.

6.4 Updating

sudo sre update refreshes an installed tree from the GitHub repository: the latest release by default, --main for the head of main, --check to only see whether a newer version exists. It downloads the sources, merges src/SRE/params.py with the site's settings (customised parameters are kept; the questions concern only the parameters both customised here and changed upstream), replaces the files, keeps the site's own labs (everything in lab/ but lab/sre/) and extra lib/ files, rebuilds bin/sre-wrapper and records the installed commit in <main_sre_dir>/.sre-update.json. See the *CLI reference* for the details.

Before it replaces anything it runs sre wipe: run it outside class hours, every running project is stopped. It must run on the host that owns the tree: the NFS server in the read-only export topology (without a Docker daemon there, the wipe is skipped and sre wipe has to be run on the client hosts), and every workstation of the "replicate per workstation" topology. The per-host bits listed above are not touched; when the templates of scripts/etc/ changed, the update says so and they have to be reinstalled by hand. The venv is rebuilt only with --venv (or make venv afterwards; the update says when the recipe changed). To go back to the previous version: sudo sre update --ref <previous commit>, printed at the end of each update. The sample labs of lab/sre/ are reinstalled in full at every update.

6.5 Running the installer

```
sudo ./scripts/install.sh
```

The script is interactive. It asks, in order:

Prompt	Default	Purpose
main_sre_d	/opt/sre if the script is at /opt/sre/scripts/, otherwise the script's parent directory	Runtime root for SRE binaries, libs, labs, graphics, translations, locale. Substituted for /opt/sre in the sudoers rule, .desktop file, and systemd unit at install time.
sre UID	1100	UID of the system user that owns running labs.
sre GID	1100	GID of the matching group.
sre_pub_d:	/var/lib/sre	Public state: running projects, archives, exam.json.
sre_user_l	/home/sre	Per-lab shared /home/sre/{running_lab} mounts.
allow_pri	y	Whether srelab.py files may declare privileged containers.
execute_c	shell	shell, split, or False — controls how lab host_cmd() calls are executed.
extra authorized src dirs	/home	Extra directories (beyond main_sre_dir + '/lab', which is always included) from which srelab.py files may be loaded. Type - for none to restrict loading to the lab directory only.
Admin UIDs / GIDs	*(none)*	Extra UIDs/GIDs that SRE treats as administrators.

The Docker socket GID is detected automatically (no prompt).

After you confirm the summary, the installer:

1. Saves a backup of params.py to params.py.bak.<YYYYmmdd-HHMMSS>.

2. Patches the answers into `src/SRE/params.py` (single-line `key = value` substitutions, each verified after the edit).
3. Runs `make remove-debug-mode` if `debug_mode = True` is currently set — `make install` refuses to build while debug mode is on.
4. Creates the `sre` system group and user (`nologin` shell, home `/home/sre`) if they don't already exist. If a user/group with the requested name exists with a different UID/GID, it is **kept as-is** rather than overwritten — the installer warns and continues.
5. Adds `sre` to the Docker group resolved from the socket GID.
6. Installs `/etc/sudoers.d/sre` (**required** — the `sysreseval` GUI shells out to `sudo /opt/sre/sbin/sre --user` via `sre-wrapper`, so labs cannot start without this rule), validated by `visudo -c` before being kept. The rule it installs is:

   ```
   Defaults!/opt/sre/sbin/sre env_keep += "USER_USERNAME SRE_XAUTH_COOKIE"
   ALL  ALL= NOPASSWD: /opt/sre/sbin/sre --user *
   ```

   The rule grants passwordless access to every user on the host, not a specific group — `sre-wrapper` is the only entry point and it always passes `--user` with the caller's real login, so widening the sudoers scope doesn't lower the security envelope. `sre` enforces this itself: a `--user` call is accepted only when sudo ran `sre` directly (`SUDO_COMMAND`) and the compiled `sre-wrapper` binary is an ancestor of the process (`/proc/<pid>/exe`), so the C binary built by `make sre-wrapper` is required and a script cannot stand in for it. Restrict it to a specific group (e.g. `%etudiant`) if your site policy requires it.
7. Optionally installs `scripts/etc/sysreseval.desktop` to `/usr/share/applications/` so the GUI appears in the desktop application menu.
8. Optionally installs `scripts/etc/sre_bash_completion` to `/etc/bash_completion.d/sre` so the `sre` CLI gets bash completion system-wide.
9. Optionally installs the `sre-preload-images.service` systemd unit (oneshot, requires `opt-sre.mount`; not enabled by default — enable with `systemctl enable --now sre-preload-images.service` once `/opt/sre` is mounted).
10. Optionally installs `scripts/etc/sre-inotify.conf` to `/etc/sysctl.d/60-sre-inotify.conf` and runs `sysctl --system`, raising the kernel inotify limits that privileged labs (systemd PID 1 in each container) would otherwise exhaust.
11. Runs `make venv` then `make install` (which is `check-debug-mode` + `sre-wrapper` + `wrappers`).
12. Optionally creates symlinks in `/usr/local/bin/` and `/usr/local/sbin/` pointing to each executable under `main_sre_dir/bin/` and `main_sre_dir/sbin/`, so `sre`, `sysreseval`, and `sre-wrapper` can be launched without their full path. Existing non-symlink files at the same paths are left untouched (with a warning).

6.6 Install manually

If you prefer to skip `scripts/install.sh` (e.g. on a non-Debian distribution, or to integrate with your own configuration management), perform the same steps by hand from a checkout of the repo:

1. **Create the `sre` system group and user.** Pick a UID/GID (the installer defaults to `1100`). The user needs a real home for shared `/home/sre/{running_lab}` mounts but no shell login:

   ```
   groupadd --system --gid 1100 sre
   useradd --system --uid 1100 --gid 1100 --home-dir /home/sre --shell /
   ↪usr/sbin/nologin sre
   install -d -o sre -g sre -m 0755 /home/sre
   ```

2. **Add `sre` to the Docker group** so it can talk to `/var/run/docker.sock`:

```
usermod -aG "$(stat -c %G /var/run/docker.sock)" sre
```

3. **Edit `src/SRE/params.py`** to match your site. At minimum review:

   - `main_sre_dir` — runtime root for SRE binaries, libs, labs, graphics, translations, locale (default `/opt/sre`). If you change this, also rewrite the literal `/opt/sre` in the sudoers rule, `.desktop` file, and `sre-preload-images.service` snippets below.
   - `sre_uid`, `sre_gid` — must match the user/group created above.
   - `docker_gid` — GID of the group owning `/var/run/docker.sock`.
   - `sre_pub_dir` — public state directory (default `/var/lib/sre`).
   - `sre_user_public_dir` — per-lab shared directory (default `/home/sre`).
   - `allow_privileged_machines` — whether `srelab.py` files may declare privileged containers.
   - `execute_commands_on_host` — `'shell'`, `'split'`, or `False`.
   - `authorized_src_dir` — list of directories from which `srelab.py` files may be loaded. Must include `main_sre_dir + '/lab'`; defaults to also including `/home`.
   - `admin_uids`, `admin_gids` — extra UIDs/GIDs treated as administrators.
   - `terminal_cmd_prefix`, `terminal_title_opt` — the external terminal emulator the GUI launches to open machine connections (default `mate-terminal`). Check which terminals are installed and uncomment the matching pair shown in `params.py`:

```
command -v mate-terminal gnome-terminal xfce4-terminal xterm terminator
```

   - `debug_mode` — **must be `False`** before building (`make wrappers` refuses otherwise). Use `make remove-debug-mode` if needed.

4. **Install the sudoers rule** — required for `sysreseval` to start labs (the GUI invokes `sudo /opt/sre/sbin/sre --user` through `sre-wrapper`). The rule allows every user on the host to invoke the wrapper without a password; restrict the first column to a specific group (e.g. `%etudiant`) if your site policy requires it. Then validate before keeping the file:

```
Defaults!/opt/sre/sbin/sre env_keep += "USER_USERNAME SRE_XAUTH_COOKIE"
ALL  ALL= NOPASSWD: /opt/sre/sbin/sre --user *
```

```
visudo -cf /etc/sudoers.d/sre && chmod 0440 /etc/sudoers.d/sre
```

5. **(Optional) Install the desktop entry** so the GUI appears in the desktop application menu:

```
install -m 0644 -o root -g root scripts/etc/sysreseval.desktop /
↪usr/share/applications/sysreseval.desktop
```

6. **(Optional) Install the image pre-pull systemd unit** from `scripts/etc/sre-preload-images.service` into `/etc/systemd/system/`. It requires `opt-sre.mount` and is not enabled by default — see *Pre-loading Docker images* below.

7. **(Recommended) Raise the kernel inotify limits.** Privileged labs run `/sbin/init` (systemd PID 1) inside each container, and systemd reserves inotify watches per cgroup. Debian defaults (`max_user_instances = 128`) are quickly exhausted once two privileged labs run side by side, after which systemd in any new container exits at startup with `Failed to create control group inotify object: Too many open files`. Drop in the file shipped under `scripts/etc/` and apply it:

```
install -m 0644 scripts/etc/sre-inotify.conf /etc/sysctl.d/60-sre-inotify.conf
sysctl --system
```

8. **(Optional) Install bash completion** for the `sre` CLI. The script under `scripts/etc/sre_bash_completion` completes subcommands, running lab names (read from `/var/lib/sre/projects/`), available labs (via `sre list`), and option arguments. Install it system-wide:

   ```
   cp scripts/etc/sre_bash_completion /etc/bash_completion.d/sre
   ```

   Or source it per-user from `~/.bashrc`:

   ```
   . /path/to/sre_bash_completion
   ```

9. **Build the venv and the binaries:**

   ```
   make venv      # creates venv/ with Python 3.13 deps - see Dependencies below
   make install   # check-debug-mode + sre-wrapper + wrappers
   ```

10. **(For labs with switches) Install the network plugin with the switch types:** `make network-plugin` — see *Network plugin for the switch types*. Labs whose networks are all hubs do not need it.

You can also run the CLI and GUI directly from source against the venv:

```
source venv/bin/activate
python3 -W ignore src/sre.py <command>     # CLI
python3 src/sysreseval.py                  # GUI
```

Most `sre` subcommands need root and a working Docker setup, but the pure-Python paths (e.g. `sre cat`, `sre sheet`, `sre outline`) work from a venv as a regular user against pre-existing archive files.

6.7 Post-install steps

Once the installer has built the binaries, finish the deployment:

6.7.1 1. Deploy and verify

1. Make sure the built tree is reachable at `/opt/sre` on every workstation that will run labs. If you ran the installer from `/opt/sre` directly (per *Deployment layout*) this is already done; otherwise copy/bind-mount the built tree into place, or expose it via the read-only NFS export and mount it as `/opt/sre` on each workstation.

2. Verify Docker access for the `sre` user:

   ```
   sudo -u sre docker ps
   ```

   The Docker images SRE needs are pulled from Docker Hub on first lab start; no local image build is required.

3. Make sure `/opt/sre/bin` and `/opt/sre/sbin` are reachable on `PATH` so that `sysreseval`, `sre-wrapper`, and `sre` can be launched without their full path. The installer offers to create the symlinks under `/usr/local/bin` and `/usr/local/sbin` for you; if you skipped that prompt (or installed by hand), either add the two directories to the system-wide `PATH` (e.g. via `/etc/profile.d/sre.sh`), or create the symlinks manually:

```
ln -s /opt/sre/bin/*  /usr/local/bin/
ln -s /opt/sre/sbin/* /usr/local/sbin/
```

(Optional) Tune `src/SRE/params.py` further — see *Runtime & Internals*.

6.7.2 2. Enable X11 access for lab virtual machines

Labs run graphical (X11) applications inside their containers and those apps display on the host's X server (if the parameter `x11_host` is set to `True` in a virtual machine configuration). For this to work the host's X server must accept TCP connections on **port 6000** (display `:0`). Modern X servers start with `-nolisten tcp`, so TCP is disabled by default and must be turned on explicitly. Access to individual labs is then authorized per project via an xauth cookie (`SRE_XAUTH_COOKIE` / `sre start --xauth-file`), so no `xhost` tweaks are needed — only the TCP listener.

How to enable TCP listening depends on your display manager. Two common cases:

- **GDM** — in `/etc/gdm3/custom.conf`:

  ```
  [security]
  DisallowTCP=false
  ```

- **LightDM** — in `/etc/lightdm/lightdm.conf` under `[Seat:*]`:

  ```
  xserver-allow-tcp=true
  ```

Restart the display manager (or log out and back in) for the change to take effect, then verify:

```
ss -ltn | grep 6000
```

You should see the X server listening on `0.0.0.0:6000` (and/or `[::]:6000`). `scripts/install.sh` runs this same check at the end and warns if no X server is listening on port 6000.

6.7.3 3. Restricting student access to Docker

For an exam to be meaningful, students must not be able to talk to Docker directly — otherwise a `docker exec -it ...` from a regular shell would let them connect to a forbidden container and bypass the lab's restrictions.

The recommended setup:

1. Remove students from the `docker` group entirely. Only `sre` (and any administrator account) should be a member.
2. Create a separate `docker-access` group and add the student accounts to it.
3. Toggle that group's access to `/var/run/docker.sock` with the two helpers shipped under `scripts/`:
   - `scripts/docker-allowed` grants `rw` on the socket to `docker-access` (run before non-exam sessions).
   - `scripts/docker-forbidden` revokes it (run before an exam).

Both helpers use `setfacl` on the Docker socket, so the change is immediate and survives until the next call.

6.7.4 4. Sharing evaluation archives for `sre watch`

`sre watch <directory>` runs an interactive terminal dashboard that monitors evaluation archives as they appear and alerts on student inactivity or errors. To use it during an exercise session or an exam, every workstation must drop its archives into a directory the instructor can read from one place. Set the `archive_dirs` attribute in `params.py` (or on some lab's `srelab.py`) so evaluations are written into the shared location in addition to the local one.

There are several ways to set up the shared hierarchy:

Single NFS share, server-side watch. Export one NFS directory, restrict read-write access to the `sre` user (e.g. `rw,no_root_squash` for the workstations, mounted only as `sre`), and run `sre watch` directly on the NFS server. This is the simplest setup and works well when the instructor's session is on the server itself.

Per-workstation NFS shares with nightly rotation. Export one subdirectory per student workstation, each writable only by that workstation. The script `scripts/misc/rotate-nfs-dirs.py` is meant to be invoked from `cron` (typically once a night). On each run it:

1. Renames `/home/sre-archives/current` to `/home/sre-archives/YYYY-MM-DD` (the previous day), so past-day archives are no longer reachable from any student computer.
2. Re-creates `/home/sre-archives/current/<machine>` for every workstation listed in the `MACHINES` constant.
3. Rewrites `/etc/exports.d/sre-archives.exports` so each subdirectory is exported only to the matching workstation IP.
4. `systemctl reload nfs-kernel-server`.

Edit the `MACHINES`, `BASE_DIR`, and `EXPORTS_FILE` constants at the top of the script to match your site before installing the cron job. `sre watch /home/sre-archives/current` then sees every workstation's live archives.

Read-only cross-mount for the instructor. In addition to either of the above, you can re-export the shared directory read-only to the instructor's workstation. The instructor can then run `sre watch` from their own machine without logging into the server.

6.7.5 5. Pre-loading Docker images

When an exam starts, every workstation pulls the same images at the same time — without precaution this can saturate the network and cause a meltdown. The fix is to pre-pull the images on each host ahead of time.

The unit `scripts/etc/sre-preload-images.service` (offered for installation by `scripts/install.sh`) runs once at boot and shells out to:

```
/opt/sre/sbin/sre preload-images --random-delay 120 /opt/sre/lab/
```

The `--random-delay` spreads the pulls over a 120-second window so workstations don't hit Docker Hub simultaneously. The unit requires `opt-sre.mount` (i.e. `/opt/sre` mounted as a systemd mount unit) and is **not enabled by default**:

```
systemctl enable --now sre-preload-images.service
```

6.8 Reference

6.8.1 Build targets

Target	Output
`make wrappers`	Marks `sbin/sre` and `bin/sysreseval` shell wrappers executable. Refuses to build while `params.debug_mode = True`.
`make sre-wrapper`	`bin/sre-wrapper` (small C helper that escalates via `sudo`).
`make install`	`check-debug-mode` + `sre-wrapper` + `wrappers`.
`make network-plugin`	Kathara VDE network plugin with the switch types, built from the fork and installed in place of the stock plugin (see *Network plugin for the switch types*).
`make translations`	Compile Qt `.ts` → `.qm` and gettext `.po` → `.mo`.
`make docs`	API docs (pdoc) + main docs (Sphinx) → `docs/html/main/`, `docs/html/api/`, `docs/documentation.pdf`.
`make set-debug-mode` / `make remove-debug-mode`	Flip `debug_mode` in `src/SRE/params.py`.

`debug_mode` must be `False` for `make wrappers` and `make install`, but **must be `True`** to run exam-mode integration tests — the GUI then emits the JSON event stream those tests assert on.

6.8.2 Tests

```
make tests                          # unit tests (excludes test_exam_mode.py)
make test FILE=test_net_config.py   # single file
make functional-tests               # functional tests (test_functional.py)
make exam-tests                     # exam-mode integration tests (requires␣
↪debug_mode=True)
make docker-tests                   # Docker integration tests with real containers␣
↪(root; start/stop, save/restore)
make all-tests                      # tests + functional-tests + exam-tests +␣
↪docker-tests
```

Unit tests don't need Docker or Kathara — they cover serialization, topology, grading logic, and pure-Python helpers. Functional and exam-mode tests stand up more of the system: see `tests/conftest.py` for the shared fixtures (`mock_sre_args`, `tmp_lab_dir`, `tmp_pub_dir`).

Pytest runs with `-p no:cacheprovider` so the production install (`/opt/sre/`, read-only) doesn't try to write a cache.

6.8.3 Building the documentation

```
make docs            # api_doc + main_doc_pdf + main_doc_html
make main_doc_html   # Sphinx HTML only → docs/html/main/
make main_doc_pdf    # Sphinx LaTeX → docs/documentation.pdf
make api_doc         # pdoc API → docs/html/api/
```

The Sphinx targets pip-install `sphinx myst-parser furo` into the existing venv on demand; `api_doc` does the same for `pdoc`.

6.8.4 Network plugin for the switch types

A lab may give each network a type of switch (`'mode': 'hub'`, `'switch'` or `'managed'` in its `_network_specs`, see *Hubs, switches and VLANs*). Hubs — the default, and the only type of every lab written before — work with the stock Kathara network plugin, the one Kathara downloads by itself on the first start. The two other types need the VDE plugin of the fork `emotchane/NetworkPlugin` (branch `main`), installed under the name Kathara uses (`kathara/katharanp_vde:<arch>`):

```
make network-plugin      # as root or a member of the docker group
```

The target installs the plugin **in place of the stock one** and enables it. It first tries the published build, `sysreseval/katharanp_vde:<arch>` on Docker Hub (`NETWORK_PLUGIN_IMAGE`), downloaded under the name Kathara uses; when that one is not available, or with `BUILD=1`, it clones the fork into `build/` and builds the plugin for the architecture of the host (a few minutes; needs `git`, `python3` and Docker with `buildx`). It does nothing when the installed plugin already knows the switch types (`FORCE=1` installs it again) and refuses to run while a Kathara network exists: stop the running projects first (`sre wipe`). The plugin belongs to Docker, not to the SRE tree: it is installed once per host and stays when SRE is reinstalled. `scripts/install.sh` offers to run the target.

To publish a build (maintainers), from a clone of the fork on a host of the wanted architecture, in a terminal, after `docker login` with an account allowed to push to the organisation:

```
cd NetworkPlugin/vde
make push_amd64 PLUGIN_NAME=sysreseval/katharanp_vde     # builds␣
↪sysreseval/katharanp_vde:amd64 and pushes it
```

A plugin that knows the switch types says so in its settings:

```
docker plugin inspect kathara/katharanp_vde:amd64 --format '{{.Settings.Env}}'    # ...␣
↪KATHARA_SWITCH_MODES=hub,switch,managed
```

Without it `sre start` of a lab that uses a switch fails — Kathara's message (*the Kathara Network Plugin … does not support it*) followed by the networks concerned and a reminder of `make network-plugin` — and nothing is left running. The management console of a managed switch is a unix socket the plugin gives to the group of `/var/run/docker.sock`: the `sre` user reaches it as it reaches Docker.

6.8.5 Dependencies

Installed by `make venv`:

Package	Purpose
`kathara`	Docker lab orchestration — installed from the fork `emotchane/Kathara`, branch `main`, which adds `save_lab()` / `restore_lab()` (used by `sre save` / `sre restore`) and the switch modes of the collision domains (hub / switch / managed switch with VLANs, used by the `mode` of a network); `git` must be available when running `make venv`
`cryptography`	AES-GCM encryption of save files for labs that define `save_key`
`pyside6`	Qt6 GUI
`msgpack`	Efficient binary serialization
`zstandard`	Archive compression
`graphviz`	Network topology diagrams
`odfpy`	LibreOffice ODS spreadsheet export
`fpdf2`	PDF report generation
`markdown`	Lab description rendering
`netaddr`	MAC address handling
`pytest`	Test runner

Documentation builds also fetch `sphinx`, `myst-parser`, `furo`, and `pdoc` on demand.

CHAPTER

SEVEN

GUI REFERENCE — SYSRESEVAL

7.1 Launching

```
source venv/bin/activate && python3 src/sysreseval.py   # dev
sysreseval                                              # production
```

One instance per user. A PID file at `/tmp/sysreseval-{uid}.pid` kills any prior instance on startup.

7.2 Opening a project

File → Open Project (or the `+` tab) shows a dialog built from `sre list --with-titles`. Leaves display each lab's translated `title` (from per-directory `titles.json` sidecars, generated by `sre make-titles`), falling back to the filename without `.py`. If the lab's `Flavor` sets `flavor_form_at_startup = True`, a form runs first. A progress dialog then tracks image pulls and container starts (JSON on stderr from `sre start`); the project tab appears when every machine is up.

During an exam, **Open Project** is disabled — only pre-authorised exam labs open automatically.

7.3 Saving and restoring a project

File → Save Project appears only when the current project's lab sets `allow_save_restore = True`. A file dialog asks where to write the `.sre` save file (default `~/<lab>.sre`); the GUI then runs `sre-wrapper save <project>` with its stdout redirected to that file, showing a progress dialog. The file contains the filesystem of every machine (as a diff against its base image) and the project files; it is encrypted when the lab defines `save_key`.

File → Restore Project (always available outside exam mode) asks for a `.sre` file and runs `sre-wrapper restore -` with the file on stdin, showing the same image-pull / machine-start progress as **Open Project**. The saved project comes back as a **new** tab (new instance); the original one, if still open, is untouched. Only files are restored — the lab's `restore` state relaunches its services.

7.4 Project tabs

Tab	Content
Sche	Topology drawn by graphviz from the machines of `info.json`, without the hidden machines. Scroll = zoom, click-drag = pan.
Full Sche	Debug projects only (`sre start --debug-project`). The same drawing and controls with every machine: the hidden machines, their links and the networks only they use are in red, like the titles of the hidden machines in the Terminals tab. The Schema tab keeps showing what a student sees.
Information	Markdown lab description from `self.informations` in `NetScheme.__init__()`.
Machin	Per-machine state (polled from Kathara every 1 s), NAT network, ports, plus a *Connect* button opening an external terminal.
Networl	Always shown. One row per network with its type — *Hub*, *Switch* or *Manageable switch* — and, for a manageable switch the lab leaves open to students, a *Connect* button opening its management console in an external terminal (`sre-wrapper connect <project> <network>`). The type of a manageable switch the lab closed is on a red background and has no button — except in a debug project, where it gets an orange *Connect* button and an orange-titled terminal in the Terminals tab. See *Hubs, switches and VLANs* for the three types and the commands of the console.
Question	Three types: `question_text` (free text), `question_form` (inline `@@{field:regex}@@` single-line text, `@@{field:+N}@@` multi-line text of N lines, `@@{field:>opt1|opt2}@@` dropdown or `@@{field:?}@@` checkbox), `question_dummy` (display only). Persisted to `answers/answers.json` on every edit.
Evaluation	Last grade table + *Start evaluation* button. Letter grades (OK/MEH/FAIL) appear when the lab sets `no_mark_on_self_grade`. A countdown replaces the button while `delay_between_self_grade` is active.
Terminals	Embedded terminals per machine, launched via `params.terminal_cmd_prefix`, plus one per manageable switch students may use (its console).
Log	Debug projects (`sre start --debug-project`), and instructor-mode projects while the *Instructor mode* button is on (states only). Live tail of the project's `operations.log`: one `=== <time> state <name>` / `=== <time> evaluation` header per run, then one 'stepN - on <machine

`answers/answers.json` also carries metadata (`hostname`, `login`, `fullname`, `email`, `language`, `answers_updated_at`, plus `exam_*` fields in exam mode); see *Archive format* for the full schema.

Start evaluation runs `sre eval --auto-eval` in a background thread. The periodic background eval (`eval_interval_without_exam_mode`) and `eval_before_exit` run plain `sre eval` (no cooldown, no log line, no stdout).

7.5 Instructor mode

A project in instructor mode (`sre start --instructor-mode`, `sre restore --instructor-mode` or `sre set-instructor-mode`, all privileged) adds a checkable **Instructor mode** button (mortarboard icon) at the left of the Wrap / Settings / Language buttons. The button is visible only while the current tab is such a project; it is off when the GUI starts and its state is not saved.

Button	Informations / Questions	Apply Configuration	Log tab
off	Exactly what a student sees: the `instructor()` texts are not displayed.	Only the states a student may apply (the tab is hidden when there is none).	Hidden.
on	The `instructor()` texts are displayed in colour (`params.instructor_text_color` on `params.instructor_background_color`, or the colours given by the lab).	Every state of the lab, those reserved to the instructor in red.	Shown, with what each applied state executed (no evaluation).

The button applies to every open project in instructor mode. Switching the mode of an open project with `sre set-instructor-mode` / `sre remove-instructor-mode` is picked up within a second.

A **debug project** (`sre start --debug-project`) shows the same button: its `info.json` holds the `instructor()` texts too. There the button only displays or hides those texts in the Informations and Questions views; every state and the Log tab are shown whatever its position, as in any debug project.

7.6 Exam mode

When `/var/lib/sre/exam.json` exists, the GUI switches modes: File → Open / Restore / Save / Close / Close All are disabled, and answers re-save with an updated `exam_time_remaining` on every exam-config change.

Phase	Trigger	Shows / actions
Waiting	`now < start_after`	SRE logo + countdown to `start_after`. Calls `sre pre-start-exam` once ~60 s before `start_after`.
Active	`start_after ≤ now < end` and exam projects up	Project tabs + remaining-time countdown. Calls `sre start-exam` once on entry, `sre eval-exam` every `eval_interval`; re-calls `pre-start-exam` if `labs` changes.
Ende	`now end_before`, or `now started_at + duration`	Calls `sre eval-exam` then `sre end-exam` once; displays the *That's All Folks* logo.

7.7 Settings

File → Settings (persisted in `~/.config/sysreseval`, INI):

Section	Setting	Default	Effect / applies
Interface	Font size	10 pt	Menus and dialog text; immediate
Terminal	Font size	12 pt	Terminals opened after the change
Terminal	Color scheme	Black on White	`black_on_white` / `white_on_black`; new terminals
Schema	Lines	Straight	Straight or curved edges; next lab open
Schema	Spacing	3 (0–9)	Node spacing; next lab open
Schema	Nodes	Shapes	Icons or geometric shapes; next lab open
Content	Font size	12 pt	Informations + Questions panes; immediate

CHAPTER

EIGHT

CLI REFERENCE — SRE

`sre --help` groups subcommands into five buckets: **dual** (admins + GUI via `sre-wrapper`), **admin-only**, **exam management**, **post-exam management**, and **internal** (GUI-only).

8.1 Global options

```
sre [--user] [--debug] <subcommand> ...
```

Option	Description
`--user`	Set by `sre-wrapper`; reads student login from `USER_USERNAME` (else `LOGNAME`). Mutually exclusive with `--debug`.
`--debug`	Verbose diagnostics to stderr.

Access is restricted to `root`, the `sre` user (`params.sre_uid = 1100`), members of `params.admin_uids`, or members of any `params.admin_gids` group. Non-privileged users may only run `cat`, `check-eval`, `re-eval`, `sheet`, and `outline`.

8.2 Dual commands

8.2.1 `sre start [-d data] [-p] [--flavor … | --flavor-json … | --set-flavor-name …] [--xauth-file <file>] [--instructor-mode] <lab> [data_version]`

Starts a new lab instance, named `{timestamp}@@@{lab_name}@@@{username}`. Imports `srelab.py`, runs `Data.generate(flavor)` (or loads `-d data.json`), builds `NetScheme`, pulls images, deploys via Kathara, applies the `initial` state, and writes `info.json`. See *Internals* for file-transfer and progress mechanics.

Argument / Option	Description
`lab`	Lab name (from `sre list`) or, with `-p`, a filesystem path.
`-p` / `--path`	Treat `lab` as a path.
`-d` / `--data`	Reuse an existing `data.json` instead of generating one.
`data_version`	Optional seed for `Data.generate()` (privileged only; incompatible with `-d`).
`--flavor key:val …`	Flavor field values (e.g. `--flavor nb:3 mode:hard`).
`--flavor-json <json>`	Flavor as JSON dict (used by the GUI).
`--set-flavor-name <name>`	Use a named preset `Flavor` (privileged only).
`--xauth-file <file>`	Read `SRE_XAUTH_COOKIE` from an X authority file instead of the env var (privileged only; for `x11_host=True` machines).
`--instructor-mode`	Start the project in *instructor mode* (privileged only).

8.2.2 `sre stop <running_lab>`

Undeploys containers and removes the project directory from `/var/lib/sre/projects/`. Kathara only undeploys the containers whose `user` label matches the caller, so `stop` first aligns `SUDO_UID` with the label used at deploy time (`sre` for non-privileged labs, the project owner for privileged ones; both are tried when the lab is not loaded, as in `pre-start-exam`).

8.2.3 `sre save [-o <file>] [--full-images] <running_lab>`

Saves a running project into a single save file (`.sre`): the filesystem of every container (captured by the patched Kathara `save_lab`, as a diff against the base image by default) plus the project files (`data.json`, `answers/`, `.private/files`, the user public dir with `shared/` and the volume directories, `.private/mnt`). The lab must set `allow_save_restore = True`; the lab's `save` state is applied first (see *Lifecycle states*). Only files are captured, not running processes.

Option	Description
`-o` / `--output <file>`	Write the save file here instead of stdout (privileged only).
`--full-images`	Store full committed device images instead of filesystem diffs — self-contained but large (privileged only).

In user mode (`--user`, i.e. from the GUI) the file is always written to stdout. When the lab defines `save_key`, the payload is encrypted. See *Save file format*.

8.2.4 `sre restore [--instructor-mode] <save_file | ->`

Restores a save file as a **new** running project (fresh timestamp, current user): loads the lab module named in the header (`allow_save_restore` required), recreates the project directories and copies the saved project files back, rebuilds the Kathara lab from the restored `data.json` (fresh host ports, X11 cookie, volumes, `/shared`), redeploys the containers from the saved filesystems (`Kathara.restore_lab(…, lab=…)`, same `pull`/`deploy` progress events as `start`), refreshes `exetests.py`, applies the lab's `restore` state and writes `info.json`. On failure everything created so far is rolled back.

`-` reads the save file from stdin — the only form accepted in user mode (the GUI opens the file as the student). Labs referenced by an absolute path (`sre start -p`) can only be restored by privileged users, unless the path lies inside the lab directory (`/opt/sre/lab`), in which case the save file is restored like any

other lab of that directory. An encrypted file needs a lab with the same `save_key`; a cleartext file for a lab that defines `save_key` is refused in user mode (accepted with a warning otherwise).

`--instructor-mode` (privileged only) puts the new project in instructor mode. A save file never carries the mode: without the option the restored project is a normal one, whatever the saved project was.

8.2.5 sre wipe

Stops every running Kathara lab and removes everything under `/var/lib/sre/projects/`.

8.2.6 sre connect [--shell <shell>] [--exec <argument…>] [--no-records] <running_lab> <device>

Opens a terminal session on a device: the shell of a machine, or the management console of a managed switch when `<device>` is a network declared with `'mode': 'managed'` (prompt `<network>$` , `exit` leaves). In user mode the console only accepts the commands of `params.switch_user_commands`, and it is refused when the lab sets `'allow_connection': False` on the switch. The commands of the console are listed in *Hubs, switches and VLANs*.

Option	Description
`--shell <shell>`	Override the machine default shell (privileged only; not for a switch).
`--exec <argument…>`	Run one command via the shell and return (consumes rest of argv). On a switch: one console command, privileged only.
`--no-records`	Do not record the terminal session (privileged only).

8.2.7 sre eval [-p path] [--auto-eval] <running_lab>

Evaluates a running lab. The atomic lock file `.private/eval_in_progress` prevents concurrent evals.

`--auto-eval` marks a *user-triggered self-evaluation* (sent by the GUI's *Start evaluation*). With `--user`, it enables the `delay_between_self_grade` cooldown (outside exam mode), appends a line to `.private/auto_eval.log`, and prints the result JSON. Without `--auto-eval`, a `--user` invocation runs silently — no cooldown, no log, no stdout. The `auto_eval_count` (lines in `.private/auto_eval.log` before this run) is assigned to `self.auto_eval_count` and embedded in the archive's `answers`. If the lab sets `no_mark_on_self_grade`, letter grades (OK/MEH/FAIL) replace numeric grades. Results are archived as `.zst` — see *Archive format*.

8.2.8 sre state <running_lab> <state_name>

Applies `NetScheme.<state_name>()` to a running lab. In user mode only the states declared `@sre_state(user_allowed=True)` of a lab setting `allow_user_states = True` are accepted, except on a debug project or a project in instructor mode, where every state is. File operations registered via `file()` / `append_to_file()` are injected via in-memory tar archives sent to `container.put_archive("/")`.

8.3 Admin-only commands

8.3.1 sre exec [--shell <shell>] <running_lab> <device> <command…>

Runs `shell -c <command>` in a container and forwards stdout/stderr/exit code. Default shell is `params.default_exec_shell`.

When <device> is a managed switch (a network with 'mode': 'managed'), <command…> is one command of its management console, e.g. sre exec <running_lab> lan vlan/print or sre exec <running_lab> lan port/setvlan @pc1 10 (@machine: the port of that machine). The exit status is 0, or the error number given by the switch (17 when a VLAN already exists) with the message on stderr.

8.3.2 sre set-instructor-mode <running_lab> and sre remove-instructor-mode <running_lab>

Switch the **instructor mode** of a running project (sre start --instructor-mode and sre restore --instructor-mode set it from the beginning). The mode is meant for the instructor's own project and has three effects:

- the texts the lab wraps in instructor() (see *Lab Authoring*) are kept in info.json; for any other project but a debug project (sre start --debug-project, which keeps them too) they are dropped before the file is written, so students cannot read them;
- every sre state appends what it executed to the project's operations.log (evaluations are not logged, unlike in a debug project);
- every state can be applied, as to a debug project: sre state in user mode (what the GUI runs) no longer requires allow_user_states nor @sre_state(user_allowed=True).

In the GUI all three show only while the *Instructor mode* button is on (see *GUI*). Nothing else changes: hidden machines and grades are those of a normal project.

Both commands write info.json again; the question hashes do not depend on the mode, so the answers already given stay attached. remove-instructor-mode also deletes operations.log, unless the project is a debug project (which also keeps its instructor texts).

8.3.3 sre eval-all [--display-grades / --no-display-grades]

Evaluates every running lab instance concurrently. --no-display-grades suppresses the per-lab grade summary.

8.3.4 sre check [-p] <lab> [<state>]

Validates a lab module without deploying: imports it, runs Data.generate(), checks that the Data instance survives the data.json round trip (to_json() / from_json(), pack() / unpack()), builds NetScheme, runs initial(), calls Grade.grade(). With state, also validates that state method. The networks that are not hubs are printed with their type and the VLANs of their ports, and the console commands a state sends to a managed switch are listed with its other operations (switch <network>: cmd …). The numbered sections of informations (info_subtitle() / @@sec@@ markers) are resolved as at sre start and printed as an outline (number, title, tag), and the @@ref@@ references of the question texts are checked: a marker outside a header, a level jump, a tag defined twice or unknown, or languages whose sections differ, is a FAIL.

8.3.5 sre watch [--timeout <sec>] [--interval <sec>] [-H <regexp>] [-S <time>] [-L] <dir…>

Live terminal dashboard scanning <dir…> for .zst archives. Default refresh: params.default_dashboard_refresh_interval_in_watch_command. --timeout is the inactivity-alert threshold (default 90 s). -H/--hostname-filter and -S/--starting-time preset the R and S keys.

One row per **running project instance** on a host: hostname + lab + the start timestamp of running_lab_name. A lab opened twice by the same student gives two rows with their own grades and alerts; a lab stopped then started again keeps its old row until it is dismissed (P), hidden (S) or collapsed. -L/--only-last-instances (the L key) collapses to one row per host and lab: the most recently *started*

instance, whatever the age of its archives. Dismissals and `S` apply after the collapse, so they never reveal an older instance. Lab counts and statistics are per instance.

Columns (from each archive):

Column	Source
HOSTNAME / LOGIN	`answers["hostname"]` / `answers["login"]`
LAB NAME	Middle segment of `running_lab_name`, its `@` decoded back to `/` (the lab path given to `sre start`)
STARTED	First segment of `running_lab_name` (instance start; time only when today)
GRADE	`total_grade / total_max`
ERR	Length of `errors`
LAST EVAL	`eval_date` (`YYYYmmddHHMMSS`)
TIME REMAINING	`answers["exam_time_remaining"]` minus elapsed since `answers["answers_updated_at"]`

Keys:

Key	Context	Action
`t`	Any	Toggle focus between Projects and Alerts
`↑` / `↓`	Projects / Alerts	Move selection
`Enter`	Projects	On a project row: its grade elements; on a `Lab:` row: per-element aggregation over the lab's instances
`P`	Projects	Dismiss selected project instance (hostname + lab + start)
`H`	Projects	Dismiss all projects from the selected hostname
`R`	Projects	Set a hostname regexp filter (empty = show all; initial value: `-H`)
`S`	Projects	Only show projects with an archive received after a time (empty = no limit; initial value: `-S`)
`L`	Projects	Toggle "only last instances": one row per host and lab, the most recently started (initial value: `-L`)
`U`	Either	Un-dismiss all
`d` / `Enter`	Alerts	Dismiss selected alert
`q` / `Ctrl-C`	Any	Quit
`?`	Any	Toggle help

Dismissals self-expire: inactivity alerts return on new archives, error alerts return when the count changes.

8.3.6 `sre preload-images [--random-delay <sec>] <file_or_dir…>`

Scans lab `.py` files for `image=` / `'image':` patterns and `docker pull`s each unique image. `--random-delay <sec>` waits 0–N seconds before pulling (smooths thundering-herd when invoked across many hosts).

8.3.7 `sre make-titles [-o <file> | -r] <directory>`

Generates `titles.json` sidecars so the GUI shows friendly labels (e.g. `static_routing.py` → "Static routing"). Labs without a `title` attribute are skipped.

Option	Description
`-o, --output-file <file>`	Write all titles to `<file>` (incompatible with `-r`).
`-r, --recursive`	One `titles.json` per directory containing labs (incompatible with `-o`).

Output keys match `sre list`: `"<name>.py"` for file labs, `"<dirname>"` for directory labs. Files are written atomically with sorted keys; stale entries are dropped at read time by `sre list --with-titles`.

```
{
  "static_routing.py": {"en": "Static routing", "fr": "Routage statique"},
  "tp_ssh":            {"en": "SSH lab",        "fr": "TP SSH"}
}
```

Typical usage: `sudo /opt/sre/sbin/sre make-titles -r /opt/sre/lab`.

8.3.8 sre update [--main | --ref <tag|branch|commit>] [--check | --force] [-y] [--venv]

Updates the installation (`params.main_sre_dir`, `/opt/sre`) from the GitHub repository `sysreseval/sysreseval` (`params.update_github_repository`). Privileged only, and root only: the tree is root-owned. By default the latest published release is installed. The installed version is the commit recorded by the previous update in `<main_sre_dir>/.sre-update.json` (or the checked-out commit of a `git clone`), so an unchanged repository means "already up to date".

Option	Description
`--main`	Follow the head of the `main` branch instead of the latest release.
`--ref <tag|branch|commi`	Install this tag, branch or commit, e.g. the previous commit printed by the last update, to go back.
`--check`	Only report the installed and the available versions; nothing is changed.
`--force`	Install again even when the requested commit is already installed.
`-y, --yes`	No question: every customised parameter of `params.py` is kept and no confirmation is asked (required when stdin is not a terminal).
`--venv`	Rebuild the Python virtual environment (`make venv`, a few minutes) as the last step.

What an update does, in order:

1. Checks: no exam configured (`sre del-exam` first), a writable tree (on a read-only NFS mount, run it on the server), one update at a time.
2. Downloads the source archive of the commit into a temporary directory of the tree and decides everything before touching anything: which files change, which are deleted, what `params.py` becomes, and that the new code compiles with the venv's Python.
3. Prints a summary and asks for a confirmation (unless `--yes`).
4. Runs `sre wipe`: every running project is stopped and removed, since the new code may not handle projects started by the old one. Without a Docker daemon on the host (`params.docker_socket_path` missing, e.g. an NFS server), the wipe is skipped with a warning: run `sre wipe` on the client hosts.
5. Replaces the files and reapplies the ownership and modes of `scripts/install.sh` (root, world-readable; the entries of `lab/` and `lib/` to the `sre` user, not world-readable), removes the stale `__pycache__` directories, then runs `make fonts sre-wrapper` and makes `sbin/sre` and `bin/sysreseval` executable again.

What is installed, kept and deleted:

- `lab/`: only `lab/sre/` (`params.update_lab_subdirs`) is refreshed, and reinstalled in full; every other entry of `lab/` is the site's own and is left alone.
- `lib/`: the files of the repository are refreshed; the site's extra files are kept.
- Files the repository dropped are deleted only when a previous update (or the git index of a cloned tree) installed them: the first update of a tree that was not cloned deletes nothing.
- Never touched: `venv/`, `build/`, `graphics/fonts/`, `bin/sre-wrapper`, `.git/`, the `params.py` backups.
- `src/SRE/params.py` is merged, not copied. The installed file is backed up as `params.py.bak.<timestamp>` first. A parameter the site never changed takes the new value; a parameter customised here that the new version does not change keeps its value; a parameter both customised here and changed upstream is asked about, the installed value being the default (kept with `--yes`). `sre_version` and `default_docker_image_version` always take the new value; new parameters are added, dropped ones removed. The pristine upstream values come from the state file (or `git show` on a cloned tree); without them, every difference is asked about. The merged file must compile and run before anything is replaced.

The state file `.sre-update.json` records the commit, the ref, the date, the previous commit, the manifest of the installed files and the pristine parameter values. Hints printed at the end: the command to go back (`sre update --ref <previous commit>`), `make venv` when the `venv` recipe of the Makefile changed, the `scripts/etc/` templates that changed (system files to reinstall by hand), and the reminder that replicated hosts need their own `sre update` and NFS clients their own `sre wipe`.

8.4 Exam management

Exam state lives in `/var/lib/sre/exam.json` — see *Exam Reference* for the field schema. All field names are exposed as `params.exam_*` constants.

8.4.1 `sre set-exam [options]`

Creates or updates `exam.json`. Only provided options are updated; existing fields are preserved.

Option	Description
`--labs <lab[:flavor]…>`	Authorised labs, optionally with a named flavor preset (e.g. `tp1:hard`). Required on first creation.
`--start-after <datetime>`	Exam opens after this moment. ISO (`2026-06-01T09:00`) or time-only (`09:00`, today).
`--end-before <datetime>`	Exam closes before this moment.
`--duration <minutes>`	Max duration; ends at `started_at + duration` or `end_before`, whichever first. Effective only once formally started.
`--eval-interval <seconds>`	Period between automatic evals (default 60).
`--record-sessions <bool>`	Record terminal sessions (`true`/`false`/`0`/`no`).

8.4.2 `sre del-exam`

Removes `exam.json` and wipes all running projects.

8.4.3 `sre save-records -d <dir> [--only-last-record]`

Archives the `records/` directories of running projects as `.tar.gz` files in `<dir>`. `--only-last-record` deletes prior archives for the same project so only the latest remains.

8.5 Post-exam management

Reads and transforms `.zst` evaluation archives — see *Archive format* for the schema.

8.5.1 `sre cat [options] <file…>`

Prints archive contents. Without a field option, all fields are shown.

Option	Shows
`--data`	`data_json` (serialized `Data`)
`--tests`	Raw test results per machine
`--errors`	Evaluation errors
`--answers`	Student answers
`--grades`	Grade list, total grade, total max
`--files`	Files saved in the archive
`--extract-files`	Extract saved files to the current directory
`--json`	Emit a single JSON dict per file on stdout

8.5.2 `sre check-eval [-s <srelab>] <file…>`

Diffs stored grades against a re-grade (same logic as `re-eval` but without writing files). Prints per-element changes plus a summary count. `-s` selects an updated `srelab.py` or directory; if omitted, uses each archive's `.private/srelab` symlink.

8.5.3 `sre re-eval -s <srelab> -p <prefix> [-d <outdir>] [-r] <file_or_dir…>`

Re-grades archives with an updated `srelab.py`. Useful for fixing a grading bug post-exam.

Option	Description
`-s` / `--srelab`	Updated `srelab.py` or directory.
`-p` / `--prefix`	Prepended to each output archive filename.
`-d` / `--output-dir`	Output directory (default: current).
`-r` / `--recursive`	Recurse into subdirectories.

8.5.4 `sre sheet -o <output.ods> [-r] [-S <time>] [-F <time>] [--separate-instances] <file_or_dir…>`

Exports archives to a LibreOffice ODS spreadsheet. Per distinct lab name, three sheets are produced:

Sheet	Content
{lab_nam	One row per archive, sorted by login, hostname, project_start, eval_date: login, fullname, email, hostname, project_start (start of the running project instance), eval_date, errors, total_grade, total_max, mark, maximum_mark, then one column per grade element
Question {lab_nam	Per-question: max_grade, maximum grade, average, number of projects, projects grade 0
Sessions {lab_nam	Per-student best score: login, hostname, max score, sum of maxima, then best score per question. With --separate-instances, one row per running project instance instead (a student who opened the lab twice gets two rows), with a project start column after hostname

fullname and email come from the archive's answers (empty if absent).

8.5.5 sre outline [-o <summary.ods>] [-d <pdf_dir>] [-r] [-S <time>] [-F <time>] [--lang <lang>] [--no-timeline] [--remaining-time] [--separate-instances] [--users-file <file>] <file_or_dir…>

Generates per-student PDF reports plus a summary ODS — best-graded archive per student (login + hostname + lab). At least one of -o / -d is required.

When a student opened the same lab several times, the best grade is taken across every instance and the evaluation history shows one table per instance, each headed by its start time. --separate-instances produces one PDF and one ODS row per running project instance instead: the PDF name gains __<start timestamp>, its header a "Project started" line, and the ODS a project_start column.

Option	Description
-o / --output-file	Output .ods summary (omit to skip).
-d / --pdf-directory	Output dir for PDFs (omit to skip).
-r / --recursive	Recurse into subdirectories.
--lang <lang>	Force PDF language (e.g. en, fr).
--no-timeline	Omit the evaluation-history table.
--remaining-time	Include the time-remaining column in the history.
--separate-instances	One report and one ODS row per running project instance instead of one per student.
--users-file <file>	User list LOGIN NAME EMAIL (whitespace/CSV; # comments; <3-field lines skipped).

Name / email resolution: --users-file (if provided) → fullname / email from archive answers. Columns appear only when at least one student has a non-empty value.

8.6 Internal commands (GUI-only)

8.6.1 sre list [--with-titles]

Prints a JSON array of lab names relative to /opt/sre/lab/ (s4/tp_ssh, s2/lab.py, …). Labs whose components match params.exam_only_affix (_EXAM_, _OLD_, _DRAFT_, _TESTS_) are hidden.

With `--with-titles`, output becomes `[{"name": <path>, "title": <dict|null>}, …]` — titles come from per-directory `titles.json` (see `sre make-titles`); stale entries are dropped, and labs missing a title get `"title": null` (GUI then falls back to the filename).

8.6.2 `sre export <running_lab> [--sep <N>] [--curved] [--shapes] [--reverse] [--random-seed <N>]`

Exports a running project as a Kathara zip archive, base64-encoded on stdout. Flags tune the embedded schema: `--sep` 0–9 (default 3), `--curved` for curved edges, `--shapes` for geometric nodes, `--reverse` to flip insertion order, `--random-seed` for node-order permutation. The `lab.conf` of the archive carries the type of the networks that are not hubs and the VLANs of the ports (`CD_MODE[lan]="managed"`, `pc1[0]="lan/vlan=10"`, `r1[0]="lan/trunk=10,20"`): this is the syntax of the Kathara fork SRE installs, not of the upstream Kathara.

8.6.3 `sre pre-start-exam`

Stops non-exam projects, then starts all exam labs so images are pre-pulled. Called by the GUI ~60 s before `start_after`.

8.6.4 `sre start-exam`

Stamps `started_at` in `exam.json`. Called by the GUI once `now  start_after`.

8.6.5 `sre eval-exam`

Evaluates every running exam project concurrently. Called by the GUI on a periodic timer (every `eval_interval`) and once at exam end.

8.6.6 `sre end-exam`

Stamps `ended_at` and runs a final concurrent eval on every running exam project. Called by the GUI when the exam ends.

CHAPTER

NINE

EXAM REFERENCE

9.1 `exam.json` — single source of truth

Lives at `/var/lib/sre/exam.json` (`params.sre_pub_dir + "/" + params.exam_json_name`). The same file must exist on every student PC — distribute via NFS, rsync, or any deployment mechanism. All field names are exposed as `params.exam_*` constants; never hard-code them.

Field	Constant	Type	Meaning
`labs`	`params.exam_labs`	`[[lab, flavor|null], ...]`	Authorised labs + optional flavor preset (required)
`start_afte`	`params.exam_start_after`	ISO datetime	Exam opens after this moment
`end_before`	`params.exam_end_before`	ISO datetime	Exam closes before this moment
`duration`	`params.exam_duration`	int (min)	Max duration; default `params.default_exam_duration = 90`
`eval_inter`	`params.exam_eval_interva`	int (s)	Auto-eval period; default `params.default_eval_interval_during_exams = 60`
`record_ses`	`params.exam_record_sessi`	bool	Record terminal sessions; default `true`
`pre_start_`	`params.exam_pre_start_da`	list[ISO]	Written by `pre-start-exam`; one entry per call
`started_at`	`params.exam_started_at`	ISO datetime	Written by `start-exam`
`ended_at`	`params.exam_ended_at`	ISO datetime	Written by `end-exam`

The first six are set by the instructor (via `sre set-exam`); the last three are runtime state markers.

`labs` format: each entry is a two-element list `[lab_cli_arg, flavor_or_null]`, where `lab_cli_arg` is what `sre start` would receive (e.g. `s4/tp_ssh`) and the second element is the name of a `Flavor` preset or `null`:

```
"labs": [["s4/tp_ssh", null], ["s4/tp_dhcp", "hard"]]
```

Legacy plain-string entries are still accepted on read; always unpack via `params.parse_lab_entry(entry)`.

9.2 GUI exam logic

Each student GUI reads `exam.json` every second (`_update_exam_state`) and runs the same logic — there is no coordinator. The phase is computed from `exam.json` content alone (`_compute_exam_phase`):

- **Waiting** — `start_after` set, `now < start_after`, `started_at` not set.
- **Ended** — `end_before` set and `now  end_before`, **or** `started_at` set and `now  started_at + duration` (duration defaults to `params.default_exam_duration = 90`). The duration check fires only when `started_at` is set — without it, the exam was never formally started.
- Otherwise: **Active**.

`real_starting_time` (drives the countdown) = `started_at` if set, else `start_after` if set, else GUI process start time.

Phas	Preconditions	Actions
Wait ing	—	Show SRE logo + countdown to `start_after`. Once `now  start_after - params.max_duration_between_exam_pre_start_and_start` (60 s) **and** `pre_start_date` is absent, fire `sre pre-start-exam` once (stops non-exam projects, starts exam labs, appends to `pre_start_date`).
Ac- tive	Exam containers up — one running project per lab in `labs`, no others (`_projects_ready`)	Fire `sre start-exam` once (stamps `started_at`; calls `pre-start-exam` internally if `pre_start_date` is missing). Show tabs + countdown. Every `eval_interval` s, fire `sre eval-exam` (skipped if previous still running). If `labs` changes, re-fire `sre pre-start-exam`.
Ende	—	Fire `sre end-exam` once (stamps `ended_at`, runs a final concurrent eval with `save-records --force`). Show "That's All Folks" until the instructor runs `sre del-exam`.

9.3 Commands

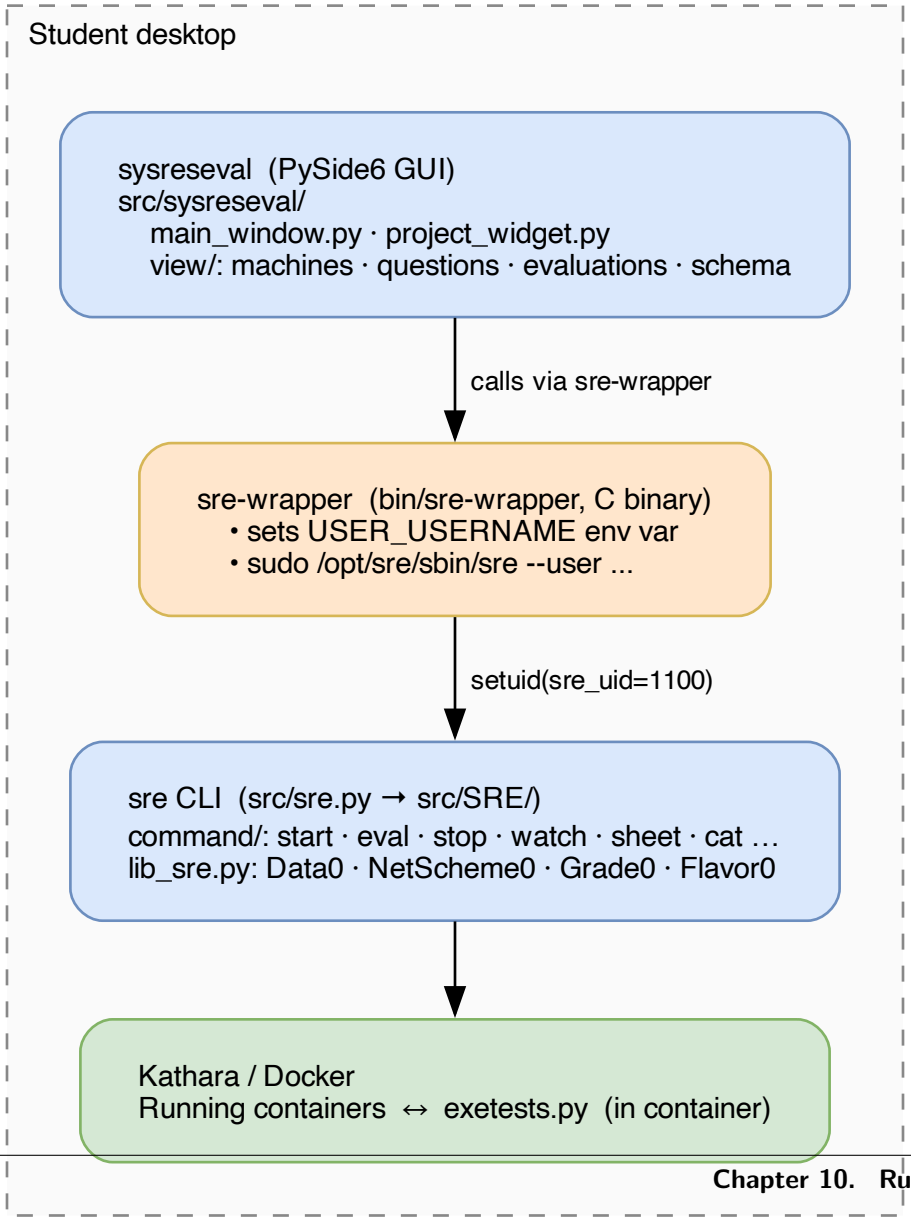
See *CLI Reference* for `set-exam`, `del-exam`, `pre-start-exam`, `start-exam`, `eval-exam`, `end-exam`, `save-records`, `watch`, `cat`, `check-eval`, `re-eval`, `sheet`, `outline`.

CHAPTER

TEN

RUNTIME & INTERNALS

10.1 Architecture



Every command that touches Docker runs as user `sre` (uid `1100`): `sre-wrapper` exports `USER_USERNAME` and calls `sudo sre --user`; the CLI then drops from root to `sre_uid`. How the drop happens depends on the project:

- **Non-privileged projects** (no machine has `privileged=True`): privileges are dropped **permanently** at start via `setuid(sre_uid)` / `setgid(docker_gid)` — the process can never regain root for the rest of its life.
- **Privileged projects** (at least one machine has `privileged=True`): privileges are dropped **temporarily** via `seteuid`/`setegid` only. Kathara requires the deploy/connect/exec paths to run as genuine root (to mount cgroups, attach a TTY to `/sbin/init`, etc.), so the CLI raises the effective uid back to 0 around those operations and lowers it again afterwards. Host-side subprocesses still drop to `sre_uid` in the child via `preexec_drop_to_sre()`, so user code on the host never runs as root.

10.2 Filesystem layout

Path	Description
`/opt/sre/`	Installation root
`/opt/sre/sbin/sre`	CLI binary
`/opt/sre/bin/sre-wrapper`	C binary for student use (handles sudo)
`/opt/sre/bin/sysreseval`	GUI launcher script
`/opt/sre/lab/`	Lab files
`/opt/sre/lib/`	Shared libraries for project files (`ips.py`, `std.py`, `exetests.py`, `net_config.py`, `dhcp.py`, `ping.py`, `ssh.py`, `tls.py`, `grade_helpers.py`, `frr.py`, `state_helpers.py`, `pcap_gen.py`, `smtp.py`, `smtp_probe.py`, `utils.py`)
`/opt/sre/graphics/`	SVG icons and logos
`/opt/sre/locale/`	Compiled gettext translations for the CLI
`/opt/sre/.sre-update.json`	State of `sre update`: installed commit and ref, previous commit, manifest of the installed files, pristine upstream values of `params.py`
`/var/lib/sre/`	Runtime state root (`sre_pub_dir`)
`/var/lib/sre/exam.json`	Exam configuration (absent = no exam)
`/var/lib/sre/projects/`	One directory per running lab instance
`/var/lib/sre/archives/`	Default evaluation archive directory
`/var/lib/sre/last_self_gra`	Self-grade cooldown timestamps
`/home/sre/`	Shared directories for projects with `shared_path=True` (mode `0o777`, removed on stop/wipe)
`/var/lib/sre/tmp/`	Temporary directories of `sre save` / `sre restore` (`params.save_tmp_dir`; not `/tmp`, which may be a small tmpfs)

10.2.1 Running lab directory

```
/var/lib/sre/projects/{timestamp}@@@{lab_name}@@@{username}/
+-- .private/               mode 0o700
|   +-- data.json           serialized Data instance, mode 0o600
|   +-- srelab              symlink to the lab's srelab.py
|   +-- eval_in_progress    lock file (PID) during active eval
```

(continues on next page)

(continued from previous page)

```
|   +-- auto_eval.log       one ISO timestamp per student self-evaluation
+-- info.json               public machine/switch/question metadata (InfoLab;␣
↪`switches`: the visible networks with their type)
+-- operations.log          debug projects only: operations of every state / evaluation␣
↪(GUI Log tab), mode 0o644
+-- answers/
|   +-- answers.json        student answers (updated by GUI)
|   +-- cheat.json          instructor-provided answers (if any)
+-- shared/                 mode 0o777 (only if shared_path=True)
```

10.3 Configuration

10.3.1 Constants (`src/SRE/params.py`)

Constant	Value	Description
`sre_uid`	`1100`	UID of the `sre` system user
`docker_gid`	`988`	GID of the `docker` group
`max_docker_concurrency`	`16`	Max parallel Docker API calls during eval
`default_eval_interval_during_exam`	`60`	Default auto-eval period (s)
`default_exam_duration`	`90`	Default exam duration (min), used if `duration` absent from `exam.json`
`default_inactivity_threshold_in_w`	`90`	`sre watch` inactivity alert threshold
`max_duration_between_exam_pre_sta`	`60`	Window before `start_after` during which `pre-start-exam` fires (s)
`email_in_gecos_last_field`	`True`	Extract last GECOS sub-field as student email
`terminal_cmd_prefix`	`["/usr/bin/mate-terminal", "--"]`	External terminal emulator
`terminal_font_size`	`12`	Default terminal font size (pt)
`terminal_color_scheme`	`"black_on_white"`	Default terminal colors
`exam_only_affix`	`["_EXAM_", "_OLD_", "_DRAFT_", "_TESTS_"]`	Name substrings that hide labs from `sre list`
`authorized_src_dir`	`['/opt/sre/lab', '/home/etudiant']`	Allowed lab source directories
`default_network_mode`	`"hub"`	Type of switch of a network that declares no `mode` (`network_modes`: `hub`, `switch`, `managed`)
`switch_user_commands`	`("help", "port/print", "port/setvlan", "vlan/create", …)`	Console commands a student may run on a managed switch; any other is refused in user mode
`hostname_keyword` / `login_keyword` / `fullname_keyword` / `email_keyword` / `language_keyword`	`"hostname"` / `"login"` / `"fullname"` / `"email"` / `"language"`	Keys used in `answers.json` and archive `answers` dict

10.3.2 Environment variables

GUI / wrapper → sre CLI

Variable	Used by	Description
`USER_USERNAME`	sre CLI (with `--user`)	Student login name (set by `sre-wrapper` from the invoking `USER`)
`SRE_PUB_DIR`	sre CLI, GUI, tests	Override `/var/lib/sre`
`SRE_WRAPPER`	params.py	Override the sre-wrapper path
`LANG`	sre CLI	Loads French help strings when set to a French locale

sre CLI → containers

Injected into machines at deploy / `exec` / state-transition time.

Variable	Description
`SRE_LAB_NAM`	Running lab name (`{timestamp}@@@{lab_name}@@@{username}`) — defined by `params.sre_name_env_variable`
`SRE_XAUTH_C`	X authority cookie forwarded into privileged containers so GUI apps can reach the host X server — defined by `params.sre_xauth_cookie_env_variable`
`SRE_HOST_IP`	Host IP reachable from containers (default `172.17.0.1`) — defined by `params.sre_host_ip_env_variable`

10.3.3 Locale

The CLI uses `gettext` (`locale/fr/LC_MESSAGES/sre.mo`); the GUI uses Qt `.qm` catalogs (`translations/sysreseval_fr.qm`). Both are regenerated by `make translations`.

10.4 Dynamic module loading

Each lab is loaded at runtime via `importlib.util.spec_from_file_location('srelab', ...)`. The module path must sit under `params.authorized_src_dir` (`/opt/sre/lab/` or `/home/etudiant/`), and `/opt/sre/lib/` is prepended to `sys.path` so labs can `from ips import …`, `from net_config import …`, etc.

10.5 Data serialization

`Data0` encodes to JSON/msgpack with a polymorphic envelope:

```
{
  "__type__": "srelab.Data",
  "data": {
    "secret": "changeme",
    "ips":   {"router": "10.0.0.1/24"},
    "nets":  {"lan1": "10.0.0.0/24"},
    "ips6":  {"router": "2001:db8::1/64"},
    "nets6": {"lan1": "2001:db8::/64"},
    "macs":  {}
  }
}
```

Value	Encoding
`IPv4Interface` inside `ips` container	`"10.0.0.1/24"`
`IPv4Network` inside `nets` container	`"10.0.0.0/24"`
`IPv6Interface` inside `ips6` container	`"2001:db8::1/64"`
`IPv6Network` inside `nets6` container	`"2001:db8::/64"`
`EUI` inside `macs` container	`"00-11-22-33-44-55"`
`IPv4Address` / `IPv6Address` (standalone field)	`{"__ip__": "10.0.0.1"}`
`IPv4Interface` / `IPv6Interface` (standalone field)	`{"__iface__": "10.0.0.1/24"}`
`IPv4Network` / `IPv6Network` (standalone field)	`{"__net__": "10.0.0.0/24"}`
`EUI` (standalone field)	`{"__mac__": "00-11-22-33-44-55"}`
Nested `Data0` subclass	`{"__type__": "module.ClassName", "data": {...}}`

The container keys and markers are the `params.data_json_*` constants. Files written before IPv6 support have no `ips6` / `nets6` keys: they load as empty containers.

`data.json` is saved mode `0o600` inside `.private/` (mode `0o700`).

10.6 Test execution

`Grade.run_tests()` per step:

1. Call `grade()` to *register* tests (no execution).
2. Group commands per machine and encode them into one env var `EXETESTS@@@{timeout}:{cmd}@@@{timeout}:{cmd}@@@…`.
3. Run `exetests.py` inside each container; outputs are separated by a per-run UUID.
4. Parse outputs into `self._tests[(machine, step)][(command, timeout)]`; repeat until `step > max_step`; call `grade()` a final time to compute grades.

Up to 16 machines tested concurrently (`ThreadPoolExecutor`). Exit codes: `0`–`255` actual, `-1` = timeout.

The console commands registered with `Grade.test_switch()` are stored in the same `self._tests`, under `(network, step)` and `(command, 0)`. They are not sent to `exetests.py`: after the containers of a step, `run_tests()` runs them one by one on the console of the managed switch (see *Switches*). Being ordinary entries of `tests`, they are archived and replayed by `sre re-eval` like the others.

10.7 Switches

The type of a network (`Network.mode`: hub, switch, managed switch — see *Hubs, switches and VLANs*) is the *mode* of its Kathara collision domain, a feature of the Kathara fork and of its VDE network plugin.

- **Deployment.** `NetScheme0.get_new_lab_from_scheme()` sets the mode of the Kathara link of each network that is not a hub and passes the declared VLANs of a port (`vlan=`, `tagged_vlans=`) when a machine is connected to a managed switch. Nothing is said for a hub, so a lab with hubs only makes the same Kathara calls as before and runs with the stock plugin. Kathara refuses a non-hub mode when the plugin does not advertise it (`KATHARA_SWITCH_MODES` in the plugin settings): `sre start` / `sre restore` then roll back and `start.quit_if_no_switch_modes()` reports the networks concerned.
- **Console.** A managed switch is a `vde_switch` started with a management socket, which Kathara reaches from the host (`Kathara.exec_link()`, one connection per command). `src/SRE/switch_console.py` is the only place where SRE uses it: `SwitchConsole.run(network, command)` re-

turns `(output, code)` — `0`, the error number of the switch, or `-2` — and never raises; a word `@machine` is replaced by the port found with `Kathara.get_link_ports()` (ports are labelled `<machine>:eth<N>` by the plugin). It serves `NetScheme.switch_cmd()` (applied by `sre state` with the host operations of the step), `Grade.test_switch()`, `sre exec` and `sre connect`.

- **Students.** `sre connect <project> <network>` in user mode is a prompt of SRE, not a raw access to the socket (which belongs to root and the Docker group): each line is checked against `params.switch_user_commands` before it is sent, and `connect.get_switch()` refuses a hub or a plain switch, a network of hidden machines only, and a switch closed by `allow_connection` (a debug project lifts the last two).
- **End of a session.** The shell of a machine dies with its container; nothing would end the prompt of a switch when its project is stopped. `switch_console.interactive()` therefore polls the project directory from a thread (every `params.switch_console_watch_interval` second) and, once it is gone, interrupts the prompt of the main thread with `SIGINT` — as Ctrl-C does, so that `readline` restores the terminal — and returns.
- **GUI.** `Grade0.save_lab_info()` writes `switches` in `info.json`: `{name, mode, allow_connection}` for every network a student sees. `view/switches_view.py` draws the Networks tab from it; the tab is always shown, hubs only or not.
- **Operations log.** Switch commands of states and evaluations are logged as `stepN - on switch <network> : <command>`.

10.8 Course text (`informations`)

The course of a lab and its question texts may hold the plain-text markers of `SRE.informations` (`## @@sec:tag@@ Title`, `@@tag:name@@`, `@@ref:tag@@` / `@@ref:tag:title@@`, `@@toc@@` / `@@toc:depth@@`), emitted by `info_subtitle()`, `info_tag()`, `info_ref()`, `info_toc()` (defined there, re-exported by `SRE.lib_sre` next to `tr()` and `instructor()`) or typed inside a `tr()` block. The engine is `src/SRE/informations.py` (imports `instructor_text` only):

- `Grade0.save_lab_info()` calls `resolve_informations()` on the `TranslatedText` of `informations` (language by language; the languages must declare the same sections and tags, in the same order) and `resolve_text()` on the question titles and descriptions and on the state descriptions with the resulting `Numbering`, before `strip_instructor()`. The GUI, `sre export` and the archives only ever see resolved text; `sre check` runs the same resolution and prints `outline()`.
- Resolved forms: a header becomes `## <a id="tag"></a>2.1. Title` (an untagged section gets the id `sec.2.1`, which no tag can equal), an anchor `<a id="tag"></a>`, a reference `[2.1](#tag)` in the course and the bare `2.1` in a question (the browser of a question has no anchor), the table of contents a nested markdown bullet list of such links, indented like the line of its marker so that the `dedent()` of `markdown_to_html()` still finds the common margin. Markers inside fenced code blocks are left untouched.
- The `@@sec@@`, `@@tag@@` and `@@toc@@` markers standing inside an instructor fragment are dropped (`@@ref@@` is resolved), so that a project in instructor mode is numbered like a student's one. A question hash is computed at registration from the marked title (`_tt_hash_str`), so a renumbering of the course never detaches stored answers.
- QTextBrowser knows the `id` anchors and scrolls to them on a `#tag` link without leaving the document. fpdf2 raises on an `<a>` without `href` and on a `#tag` link to a named destination it does not know: `pdf.markdown_to_html()` removes the anchors and unwraps those links (the numbers stay) before `write_html()`.
- Errors are `ValueError` with a lab-facing message (`sre check` prints them as `FAIL`).

10.9 Progress reporting

`sre start` emits JSON lines to **stderr** so the GUI can render progress:

```
{"phase": "pull",   "status": "start",    "image": "sre/base:1.10"}
{"phase": "pull",   "status": "progress", "image": "sre/base:1.10", "percent": 42}
{"phase": "pull",   "status": "end",      "image": "sre/base:1.10"}
{"phase": "deploy", "status": "start",    "total": 3}
{"phase": "deploy", "status": "progress", "current": 1, "total": 3}
{"phase": "deploy", "status": "end",      "total": 3}
```

10.10 Security model

Concern	Mechanism
Allowed UIDs	`sre.py` checks `os.geteuid()` against `sre_uid` and 0; exits otherwise
Privilege drop	Root calls `_drop_permanently()`, which `sys.exit()`s on failure
Lab path whitelist	`utils.py` checks `current_srelab_file` is under `authorized_src_dir`
Eval lock	Lock file opened with `O_CREAT | O_WRONLY | O_NOFOLLOW`; `flock(LOCK_EX)` prevents concurrent evals and symlink attacks
Host commands drop privileges	`preexec_drop_to_sre()` is `preexec_fn` for every host-side `subprocess.run()` (eval, state transitions)
Shell injection	`tls.py` quotes with `shlex.quote()`; `state_helpers.py` writes passwords to `/tmp/.sre_chpasswd` (mode `0o600`); usernames passed via env vars
`data.json` secrecy	Mode `0o600` in `.private/` (mode `0o700`)
Unknown `__type__` deserialization	`Data0.from_dict()` / `unpack()` / `Flavor0` raise `ValueError`
Single GUI instance	PID file at `/tmp/sysreseval-{uid}.pid` kills the previous instance
Student command restrictions	`user_not_allowed()` / `user_not_allowed_in_exam_mode()`
State access control	`@sre_state(user_allowed=False)` blocks students from re-applying protected states
Switch console	In user mode only the commands of `params.switch_user_commands` are sent to a managed switch (no port creation / removal, plugin loading, shutdown); `allow_connection` closes a console; the management socket itself is not readable by students

10.11 Archive format

Every evaluation writes a `zstandard`-compressed `msgpack` file with a `.zst` extension to `params.archive_dirs` (default `['/var/lib/sre/archives']`; labs may extend the list via a module-level `archive_dirs` attribute).

Filename: `{eval_date}_{running_lab_name}.zst` — `eval_date` is `YYYYmmddHHMMSS`, `running_lab_name`

is `{timestamp}@@@{lab_name}@@@{username}`. The leading timestamp sorts chronologically with `ls`.

```
archive = {
    "running_lab_name": "{timestamp}@@@{lab_name}@@@{username}",
    "eval_date":        "20260615110042",
    "data_json":        "<JSON string>",        # serialized Data
    "tests": {
        ("router", 1): {("ping -c1 8.8.8.8", 5): ("64 bytes ...\n", 0), ...},
        ...
    },
    "errors":     ["error message", ...],
    "answers": {
        "<question_hash>":     "<student answer>",
        "hostname":            "pc-101",
        "login":               "alice",
        "fullname":            "Alice Martin",      # from GECOS, always present
        "email":               "alice@example.com", # when email_in_gecos_last_field=True
        "language":            "fr",
        "answers_updated_at":  "20260615110000",
        "auto_eval_count":     2,                   # self-evals before this one
        "exam_mode":           true,                # exam-mode only
        "exam_started_at":     "2026-06-15T09:00",  # exam-mode only
        "exam_duration":       120,                 # exam-mode only
        "exam_time_remaining": 3600,                # exam-mode only
    },
    "grade_list": [
        {"title": "Connectivity", "max_grade": 4, "grade": 4,
         "grade_letter": null, "description": ""},
        ...
    ],
    "total_grade":  14.0,
    "total_max":    20.0,
}
```

Archives are written by `sre eval`, `eval-all`, `eval-exam`, `end-exam`, `re-eval`, and consumed by `sre cat`, `sheet`, `outline`, `check-eval`, `watch` — see *CLI Reference*.

10.12 Save file format

`sre save` writes, and `sre restore` reads, a single `.sre` file (`src/SRE/save_archive.py`). The layout is sequential so that `sre restore -` can consume it from stdin:

```
SRESAVE1\n                          magic (params.save_file_magic)
{"format_version": 1, …}\n          one-line cleartext JSON header (keys =␣
↪params.save_meta_*)
<payload>                           tar stream - encrypted when the lab defines save_key
```

Header fields: `format_version`, `sre_version`, `lab_name` (the `lab@path` form), `running_lab_name`, `srelab_file`, `username`, `saved_at`, `debug_project`, `full_images`, `shared_path`, `encrypted`, `kdf_salt`, `kdf_iterations`.

Payload members:

Member	Content
data.json	.private/data.json of the saved instance
kathara.tar	The patched Kathara save_lab archive: manifest.json (topology, save_mode, per-device original_image, deletions, saved image tag kathara_save_<hash>:<device>) and images/<device>.diff.tar (filesystem diff, default) or images/<device>.tar (full image, --full-images)
answers	Student answers and cheat.json
files/	.private/files (host-side files of cp_to_host() / host_cmd())
user_public/	Contents of the user public dir (/home/sre/<lab>[_N]/): shared/ and the relative-path volume directories
mnt/	.private/mnt (private volumes), when present

Not saved: info.json (regenerated), records/, the eval lock, and what was changed on the console of a managed switch after the start (the restored switches get their type and the VLANs the lab declares; the lab's restore state can re-apply more with switch_cmd()). Kathara's diff excludes /etc/hosts, /etc/hostname, /etc/resolv.conf, /dev, /proc, /sys, /run, /tmp, /hosthome, /shared, /hostlab; bind-mounted directories are covered by user_public/ and mnt/ instead.

Encryption (save_key): the payload tar stream is cut into params.save_cipher_chunk_size chunks, each sealed with AES-256-GCM (key = PBKDF2-HMAC-SHA256 of save_key with the header's salt). Frame: [4-byte length][1-byte last flag][12-byte nonce][ciphertext+tag]; the associated data is sha256(header) + chunk index + last flag, so header tampering, reordering and truncation are detected. The cryptography package is imported lazily (only needed for encrypted files).

Restore (src/SRE/command/restore.py): read the header → import the lab module (allow_save_restore required, save_key checked) → extract the payload into /var/lib/sre/tmp with tarfile.data_filter → recreate the project directories with the start.py helpers (ProjectSetup, create_project_directories), copy the saved project files back and give them to the student (answers 0o666, shared/ and volume dirs 0o777) → build the Kathara lab from the restored Data (get_new_lab_from_scheme(): fresh host ports, X11 cookie, volumes, /shared) → Kathara.restore_lab(kathara.tar, lab=…) (images rebuilt from the base image + diff, deployed with the same pull/deploy progress events as start) → refresh exetests.py → finalize_project (lifecycle state restore, data.json, info.json). Any failure triggers rollback_project (undeploy + directory removal). Restored images (kathara_save_<hash>:<device>) are not garbage-collected; docker image prune removes the unused ones.